SYSTEM OWNER'S HANDBOOK

Data Model (Part 4)

Multi-tenant clinic management platform

Status as of 2026-10-28 · code is the source of truth · written for a software-engineer owner

How to read this document

This is part of the **System Owner's Handbook** for a multi-tenant clinic management product, called "the Platform" throughout. It is written for the product owner, a software engineer, and every chapter can be read on its own.

Status labels. Every feature carries exactly one label, based on the code as of 2026-10-28:

- **[Implemented]**: built end to end.
- **[Partial]**: some of it exists; the text says exactly what is missing.
- **[Not implemented]**: planned, seeded, disabled or only mentioned. It is never described as working.
- **[Unverified]**: the code was read but the behaviour could not be confirmed; the text says what is unconfirmed.

Source of truth. The code is the source of truth. Where older documentation disagreed with the code, the handbook follows the code. Nothing was executed while writing: behaviour is described from reading the code.

Figures. Every figure has a number, a caption and a paragraph that describes it in words. Figures captioned "screenshot pending" are placeholders; the paragraph after them describes the screen from the code. The few real screenshots were taken on 2026-10-23 and show a former working name of the product.

Names. "Organization" is one clinic business (the tenant); "location" is one of its branches. File paths and code names appear in a short "Technical references" line at the end of each feature. Example people and places are fictional.

Table of contents

Part 4 — Data model

Chapter 12 — Data model overview 4
Chapter 13 — Identity and sessions 11
Chapter 14 — Organizations, locations, subscriptions and settings 16
Chapter 15 — Access and features 22
Chapter 16 — Staff and payroll 26
Chapter 17 — Patients 30
Chapter 18 — Appointments, queue and encounters 37
Chapter 19 — Prescriptions and plans 45
Chapter 20 — Billing core 53
Chapter 21 — Billing ledgers and closing 60
Chapter 22 — Compensation 65
Chapter 23 — Inventory 70
Chapter 24 — Notifications 75
Chapter 25 — Labs 79
Chapter 26 — Workflow engine and common tables 92
Chapter 27 — Overlaps and ambiguities 98

Chapter 12 — Data model overview

This chapter explains how the Platform's database is organised as a whole: who owns the schema, which naming and money conventions the tables follow, and how tenant data is kept apart. It is written for the product owner who needs to reason about the data before changing a feature or answering a support question. Chapters 13 to 27 then walk through each business domain table by table; this chapter is the map they share.

12.1 The database in numbers

The Platform stores all of its data in one PostgreSQL database. The backend defines **153 entity classes, one per table**, spread across **31 owning modules** (a module "owns" a table when the table's entity class lives in that module's folder). The schema has 278 enforced foreign keys. It has no views, no table inheritance and no many-to-many junction tables generated by the ORM; every link between two tables is an ordinary column.

The field-level reference for every table is the generated entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`. It lists every column, type, default, index, constraint and foreign key, and it was produced from the same metadata the ORM uses to build the schema. This handbook names only the key columns of each table; when you need the full column list, open the catalog. The catalog has two known defects: the "Notes" column of the ten inventory tables is shifted one row down, and one note claims a trigger on `form_schema_versions` that does not exist.

12.2 Who owns the schema

12.2.1 Entities are the source of truth

Status: [Implemented]

The shape of every table — its columns, primary key, foreign keys, unique keys, indexes, CHECK constraints and EXCLUDE constraints — is declared in TypeORM entity classes in the backend code. Nobody writes `CREATE TABLE` statements by hand for normal tables.

- **In development**, the API process builds and updates the schema itself on start-up. The setting in `app.module.ts` is `synchronize: true` whenever the process owns the schema and `NODE_ENV` is not `production`. The ORM compares the entities with the database and applies the difference.
- **In production**, `synchronize` is off. An engineer runs `npm run schema:log:prod` to print the SQL the entity changes would need, reviews every statement (watching especially for `DROP COLUMN`, `DROP TABLE` and type changes), takes a backup, and then applies it with `npm run schema:sync:prod`.

Only the API process owns the schema. The background worker process never synchronizes and never runs migrations.

Two entity classes are not registered by any running module: `chairs` and `inventory`. The running app only knows tables that some module registers, so in development these two tables are not created. The command-line data source used by `schema:sync:prod` loads entity files by file pattern, so it does include them. A development database and a production database can therefore differ by exactly these two tables.

12.2.2 Migrations: three files, two that run

Status: [Implemented]

The folder `backend/src/migrations/` holds three files. Migrations do the work that `synchronize` cannot do: triggers, SQL functions and data backfills.

File	What it does	Runs?
`1776356529788-create-table`	A partial schema diff taken from an early development database. Describes 17 tables in a shape that no longer exists.	Never. Start-up records it as already applied without executing it.
`1792403456699-doctor-compensation`	Creates the commission tables if they are missing, installs the append-only trigger on `commission_entries`, and backfills data.	Yes, once. Hand-written and idempotent.
`1792490000000-lab-management-foundation`	Creates `audit_logs`, `status_history`, `number_sequences`, `labs`, `lab_members` and `lab_clinic_links` if missing, installs `uuid_generate_v7()` and the append-only triggers on `audit_logs` and `status_history`.	Yes, once. Hand-written and idempotent.

The table lists the three migration files, what each contains and whether it executes. The first file is the "legacy baseline": it is kept in the folder, recorded as applied by a helper called `recordLegacyBaseline`, and must never be edited or deleted. The other two are written so that they give the same result whether or not `synchronize` already created the tables; they skip table creation when the table exists and always (re)install their triggers.

On API start-up the order is fixed: first the required database extensions and functions are created, then `synchronize` runs (development only), then pending migrations run, each in its own transaction. Because 136 of the 153 tables appear in no migration at all, the migration folder cannot be used to rebuild the schema; the entities are the only complete description.

12.2.3 Database objects that live outside the entities

A few database objects cannot be expressed in an entity class. They are created by start-up code or by the two running migrations:

- `btree_gist` **extension.** Needed by the GiST EXCLUDE constraints that compare uuids, such as the rule that stops one doctor from having two overlapping appointments (`appointments.no_doctor_overlap`). Created at start-up before `synchronize`.
- `uuid_generate_v7()` **SQL function.** Produces time-ordered uuids; it is the primary-key default of the newer tables. Created at start-up and also by the lab-management migration.
- `commission_entries` **append-only trigger.** Blocks every DELETE and blocks UPDATE except setting or releasing the settlement link while the settlement is still pending.
- `audit_logs` **and** `status_history` **append-only triggers.** Block every UPDATE and DELETE.
- `migrations` **table.** TypeORM's bookkeeping of which migrations ran.

There are no views, no row-level security policies and no other triggers anywhere in the backend. Several other tables are described as "append-only" or "immutable" in code comments (for example `queue_events`, `patient_consents`, `prescription_history`, `ledger_entries`), but for those the rule is kept by the application only; the database would accept an update.

12.3 Conventions

12.3.1 Primary keys

Kind of primary key	Tables	Where it is used
uuid v4, generated by TypeORM (`uuid_generate_v4()`)	84	The older modules: identity, organizations, locations, staff, patients, appointments, queue, prescriptions, billing, inventory and others.
uuid v7, default `uuid_generate_v7()`	55	The newer modules: every lab table, the workflow engine, notifications, the general-ledger journals, `audit_logs` , `status_history` and `config_values` .
bigint identity	2	`outbox` and `queue_events` , where strict insertion order matters.
Supplied by the application (natural or composite key)	12	For example `idempotency_keys` (organization + key), `location_billing_settings` (the location id), `notification_events` (an event code), `patient_comm_prefs` (the patient id).

The table counts the four kinds of primary key across all 153 tables and names where each appears. A uuid v7 starts with a timestamp, so new rows sort by creation time and indexes stay compact; a uuid v4 is fully random. Both look the same to the application. One naming quirk: the four identity tables (`users` , `oauth_providers` , `security_logs` , `user_sessions`) call their primary-key column `uuid` ; every other table calls it `id` .

12.3.2 Column naming

Columns use `snake_case` (`organization_id` , `created_at`) in every module except one. Ten of the eleven tables of the inventory module use `camelCase` column names such as `organizationId` , `locationId` and `inventoryItemId` . Queries written by hand against these tables must quote the names. The eleventh table, the unregistered `inventory` table, uses snake_case. The Inventory chapter (Chapter 23) covers those tables.

12.3.3 Money

The Platform stores money in two ways, and a reader must check which one a column uses:

- **Integer paisa** (`bigint` or `integer` columns whose names end in `_paisa`). One rupee is 100 paisa, so 1,250.50 PKR is stored as `125050` . The newer money-handling code uses this form: payroll breakdowns, commissions and compensation plans, the general-ledger journals, and every lab billing and payables table.
- **Numeric with two decimals** (`numeric(12,2)` or `numeric(10,2)`). The amount is stored in rupees with two decimal places. Billing v2 (charges, invoices, payments, refunds), subscriptions, inventory, insurance, treatment plans and the legacy `treatments.cost` use this form.

Some tables carry both. `payroll_records` keeps `base_salary_paisa` , `commission_paisa` and `total_paisa` in paisa and also `salary_amount` in rupees, which mirrors the total for older screens. "PKR" is the default currency of a new organization, not a limit: `organizations.currency` and `locations.currency` hold the currency code.

12.3.4 Timestamps and dates

Most tables have `created_at` and many have `updated_at` , filled automatically by the ORM. Older tables store these as `timestamp without time zone` (about 50 tables); newer tables use `timestamp with time zone` (about 70 tables). Business moments that matter to the clinic — appointment start and end, check-in, payment time — are `timestamp with time zone` throughout. Calendar days that belong to a clinic day, such as `queue_sessions.service_date` , are stored as `date` and are computed in the **location's** timezone (`locations.timezone`), not the server's.

12.3.5 Soft delete

The Platform hides records in three ways:

- `is_active` **flag.** Organizations, locations, employees, assignments, roles, catalogs and templates are switched off with `is_active = false`; the row stays. Deactivating an organization or a location is always a soft delete.
- `deleted_at` **column.** Three tables use TypeORM's soft-delete column: `patients`, `patient_documents` and `patient_treatment_plans`. A row with `deleted_at` set is excluded from normal ORM reads.
- **Hard delete.** Some actions really delete rows and rely on `ON DELETE CASCADE` to remove children. For example, permanently deleting a patient deletes the patient row and every child row whose foreign key cascades. Child rows whose foreign key is `RESTRICT` block the delete instead.

`patients` has both a `deleted_at` column and an `is_active` flag. The user-facing activate/deactivate switch uses `is_active`.

12.3.6 Tenant columns

Every row that belongs to one clinic carries `organization_id`, and every row that belongs to one branch also carries `location_id`. The column names are the same in every snake_case module. Whether the column has a foreign key varies by module:

- The older modules (organizations, staff, patients, appointments, encounters, treatments, prescriptions) declare real foreign keys to `organizations` and `locations`, almost always with `ON DELETE CASCADE`.
- The queue tables, nearly all billing v2 tables, the lab tables, `plan_templates`, `audit_logs` and `status_history` keep `organization_id` (and often `location_id`) **without** a foreign key. The database accepts any uuid there; the application is responsible for writing the right one.

12.4 Tenant scope rules

The catalog classifies every table by which tenant columns it carries. The classes are:

Scope class	Meaning	Examples
Global — platform	Shared by all tenants; no tenant column.	`users`, `user_sessions`, `feature_catalog`, `specialties`, `notification_events`, `workflow_effects`
Global — person or lab	Shared across organizations by design.	`employees` (one person can work for several clinics), `labs`, `lab_members`, `lab_services`
Organization	`organization_id` only.	`patients`, `employment_roles`, `org_feature_flags`, `medicine_directory`, `services`
Organization + location	Both columns.	`appointments`, `queue_sessions`, `charges`, `invoices`, `payroll_records`, `prescriptions`
Location only	`location_id` only; the organization is reached through `locations`.	`chairs` (unregistered)
Cascading scope	A `scope_type` + `scope_id` pair that points to the platform, an organization or a location.	`config_values`
Platform or organization	Nullable `organization_id`: null means a platform default shared by all, a value means a clinic's own copy.	`form_schemas`, `workflow_templates`, `stage_templates`, `notification_templates`

Scope class	Meaning	Examples
No tenant column	A child table scoped only through its parent.	the ten tables listed after this table

The table shows each scope class, what it means and typical tables. Ten tenant-data tables have no organization column and are isolated only through their parent: `treatment_plan_items` (through `patient_treatment_plans`), `prescription_history` and `prescription_medicines` (through `prescriptions`), `treatment_plan_template_medicines` (through `treatment_plan_templates`), `cash_close_lines` (through `cash_closes`), `commission_rules` (through `compensation_plans`), and the inventory children `consumption_template_items`, `patient_consumption_items` and `purchase_order_items`. Any query on these tables must join the parent to stay inside one tenant.

The rule that keeps tenants apart at runtime is not in the database. The guard chain resolves the active organization and location from the request headers, and every service filters by `organization_id` (and `location_id` where relevant). The multi-tenant model is described in Chapter 8; this chapter only describes the columns that make it possible.

12.5 The domains and how they connect

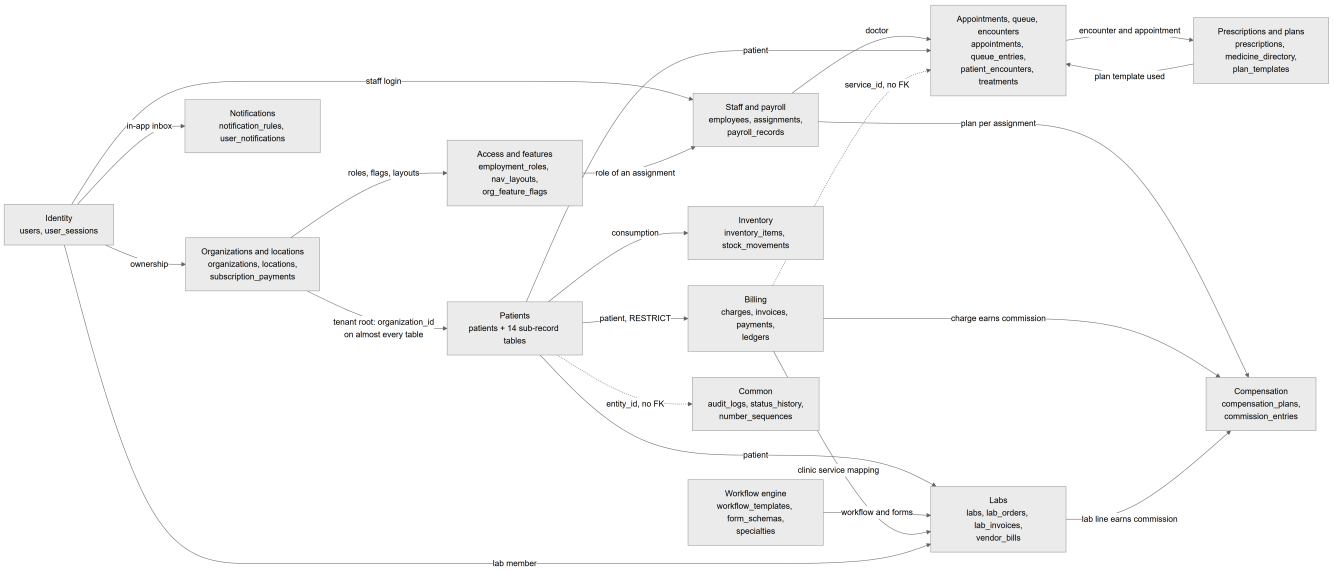


Figure 12.1 — Domains of the data model and the key links between them.

The figure draws the 153 tables as fourteen domain boxes, each labelled with the domain name and two to four of its most important tables. Arrows point from the side that is referenced (the parent) to the side that holds the reference (the child). Solid arrows are links backed by a foreign key; the two dotted arrows are logical links that have no foreign key. The left side holds the foundation: Identity, Organizations and locations, and Access and features. The middle holds the clinic's daily work: Staff and payroll, Patients, Appointments-queue-encounters, Prescriptions and plans, Billing, Inventory, Common and the Workflow engine. The right side holds Compensation and Labs, which consume data from several middle domains. Notifications sits near Identity because its main link is the user's in-app inbox. Every link drawn is explained in the next section.

12.5.1 Every link in the figure

- **Identity → Organizations and locations ("ownership").** `organization_ownership.user_id` links a user account to the organization it owns. Deleting the user or the organization deletes the ownership row (CASCADE).

- **Identity → Staff and payroll ("staff login").** `employee_org_location_assignments.user_id` links an employee's assignment to the user account they log in with. The link is optional; deleting the user sets it to null.
- **Organizations and locations → Patients ("tenant root").** `patients.organization_id` is a foreign key with CASCADE. The same kind of link, `organization_id` and often `location_id`, exists on almost every tenant table in every domain; the figure draws it once to keep the picture readable.
- **Organizations and locations → Access and features ("roles, flags, layouts").** `employment_roles`, `org_feature_flags` and `nav_layouts` each belong to one organization (CASCADE); `location_feature_overrides` belongs to one location (CASCADE).
- **Access and features → Staff and payroll ("role of an assignment").** `employee_org_location_assignments.employment_role_id` names the role a person has at a location. Deleting the role sets it to null.
- **Staff and payroll → Appointments, queue, encounters ("doctor").** `appointments.doctor_id` and `queue_sessions.doctor_id` point to `employees`, both with RESTRICT, so a doctor who has appointments or queue sessions cannot be hard-deleted.
- **Patients → Appointments, queue, encounters ("patient").** `appointments`, `queue_entries`, `patient_encounters` and `treatments` all carry `patient_id` with CASCADE.
- **Appointments, queue, encounters → Prescriptions and plans ("encounter and appointment").** `prescriptions.encounter_id` and `prescriptions.appointment_id` are optional links; deleting the encounter or appointment sets them to null.
- **Prescriptions and plans → Appointments, queue, encounters ("plan template used").** `treatments.plan_template_id` points to `plan_templates`; deleting the template sets it to null.
- **Patients → Billing ("patient, RESTRICT").** `charges`, `invoices`, `payments` and `refunds` carry `patient_id` with RESTRICT. A patient who has any billing row cannot be hard-deleted.
- **Staff and payroll → Compensation ("plan per assignment").** `compensation_plans.assignment_id` ties a salary-and-commission plan to one assignment (CASCADE). In the other direction, `commission_settlements.payroll_record_id` links a settlement to the payroll record that paid it (RESTRICT).
- **Billing → Compensation ("charge earns commission").** `commission_entries.charge_id` points to the charge that earned a commission (RESTRICT), and `charges.commission_plan_id` records which plan applied.
- **Labs → Compensation ("lab line earns commission").** `commission_entries.lab_invoice_line_id` points to a lab invoice line (RESTRICT).
- **Patients → Labs ("patient").** `lab_orders`, `lab_invoices`, `lab_payments` and `lab_refunds` carry `patient_id` with RESTRICT.
- **Workflow engine → Labs ("workflow and forms").** `lab_services` and `lab_orders` point to workflow templates, workflow template versions, stage templates and form schemas (all RESTRICT).
- **Billing → Labs ("clinic service mapping").** `service_mappings.clinic_treatment_id` maps a clinic billing service in `services` to a lab service (RESTRICT).
- **Patients → Inventory ("consumption").** `patient_consumptions.patientId` records materials used for a patient (CASCADE).
- **Identity → Notifications ("in-app inbox").** `user_notifications.user_id` delivers an in-app notification to a user (CASCADE).
- **Identity → Labs ("lab member").** `lab_members.user_id` makes a user a member of a lab (RESTRICT).
- **Patients ⇢ Common ("entity_id, no FK", dotted).** `audit_logs` and `status_history` record history for many kinds of rows through an `entity_type` + `entity_id` pair (and a `subject_id` that groups a patient's sub-records under the patient). The pair is polymorphic, so no foreign key is possible.

- **Billing ⇢ Appointments, queue, encounters ("service_id, no FK", dotted).** `treatments.service_id` names the catalog service billing uses to price the treatment, without a foreign key. `charges.encounter_id` and `charges.appointment_id` are also logical links without foreign keys.

12.6 Where to read next

Each domain has its own chapter with an entity-relationship diagram and every relationship explained: identity (Chapter 13), organizations, locations and subscriptions (Chapter 14), access and features (Chapter 15), staff and payroll (Chapter 16), patients (Chapter 17), appointments, queue and encounters (Chapter 18), prescriptions and plans (Chapter 19), billing (Chapters 20 and 21), compensation (Chapter 22), inventory (Chapter 23), notifications (Chapter 24), labs (Chapter 25), and the workflow engine and common tables (Chapter 26). Overlapping concepts — two prescription tables, four "visit" tables, two outboxes, two sequence mechanisms — are compared in Chapter 27.

Chapter 13 — Identity and sessions

This chapter describes the four tables that hold the people who can sign in to the Platform and the record of what happened when they did. It is for the product owner who needs to know where a login account lives, how a session is kept, and how sign-in events are logged. These tables sit underneath every other domain: a staff login, an owner, a super admin and a lab member are all rows in `users`.

13.1 Context

The identity domain is global. Its tables carry no `organization_id` or `location_id`, because one person has one account and may work for several organizations. What a user may do inside a clinic is not stored here; it comes from ownership (Chapter 14) and from the user's assignments and roles (Chapters 15 and 16). The tables are owned by three small modules: `user` owns `users`, `auth` owns `oauth_providers` and `security_logs`, and `session` owns `user_sessions`. Every column of every table is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

13.2 The tables

13.2.1 `users` — user account

Status: [Implemented]

One row per login account on the whole Platform. The key columns are:

- `uuid` — the primary key. This table, and the other three in this chapter, name the key column `uuid` instead of `id`.
- `email` — unique across the whole Platform. Two accounts can never share an email address.
- `password` — the stored password hash; the column is nullable.
- `global_role` — null for normal users; the value `super_admin` marks a super admin. No other value exists.
- `status` — `active`, `suspended` or `deleted`, with `suspended_at`, `suspension_reason` and `deleted_at` recording when and why. The account is never removed by this flag; it is a status.
- `email_verification_token`, `email_verification_expires`, `email_verified_at` — used only for self-signup accounts, so accounts created by an administrator are never asked to verify.
- `last_accessed_organization_id`, `last_accessed_location_id` — the organization and location the user last worked in, so the next login can return there.
- `provider` — `local`, `google` or `github`; defaults to `local`.
- `refresh_token` — a deprecated single refresh token kept for backward compatibility. Sessions now keep their own refresh tokens in `user_sessions`.

There is an index on `status` for listing accounts by state.

13.2.2 `oauth_providers` — linked external sign-in

Status: [Not implemented] (table exists, nothing reads or writes it)

Designed to record external sign-in identities (Google, GitHub) linked to an account: `user_id`, `provider`, `provider_id` (the provider's own id for the person), and the email, display name, avatar and raw profile the provider returned. A user can have at most one row per provider (unique `user_id` + `provider`), and lookups by

`provider` + `provider_id` are indexed.

The auth module registers the table, so it is created in the database, but no service injects or queries it. External sign-in itself is disabled: the Google and GitHub strategies exist as files but are not registered, and the module comment says Google/GitHub sign-in is disabled. Expect this table to be empty.

13.2.3 `security_logs` — security event log

Status: [Implemented]

One row per security-relevant event: a successful or failed login, logout, logout of all devices, password change or reset, email verification, session created, revoked or expired, token refreshed, account suspended, deleted or restored, a login from a new device, and suspicious activity. Key columns:

- `user_id` — the account concerned; nullable, because a failed login for an unknown email has no account.
- `event_type` — one of twenty event codes such as `LOGIN_SUCCESS`, `LOGIN_FAILED`, `SESSION_REVOKED`.
- `success` and `error_message` — whether the event succeeded and why not.
- `session_id` — the session involved, when the event concerns a session.
- `metadata` — free-form JSON with event-specific detail.
- The table also records the network address and browser string of the request. Those values are personal data and are not reproduced in this handbook.

Indexes on `created_at` and on `user_id` + `event_type` support "recent events" and "events of this kind for this user" queries. The table has only `created_at`; rows are never updated by design, but the database does not enforce that.

13.2.4 `user_sessions` — login session

Status: [Implemented]

One row per signed-in device or browser. Each session has its own refresh token, so a user can be signed in on several devices and sign one out without affecting the others. Key columns:

- `user_id` — the account.
- `refresh_token_hash` — a hash of this session's refresh token; the token itself is never stored.
- `expires_at` — after this moment the session is expired.
- `is_revoked` — a soft revocation flag: when true the session is no longer valid, and the row stays as history.
- `device`, `browser`, `os` and a free-text `location` describe the device; an `is_current` flag is also stored.
- `last_active_at` — updated on every save of the row.

An index on `user_id` + `is_revoked` makes "this user's live sessions" fast.

13.3 Entity-relationship diagram

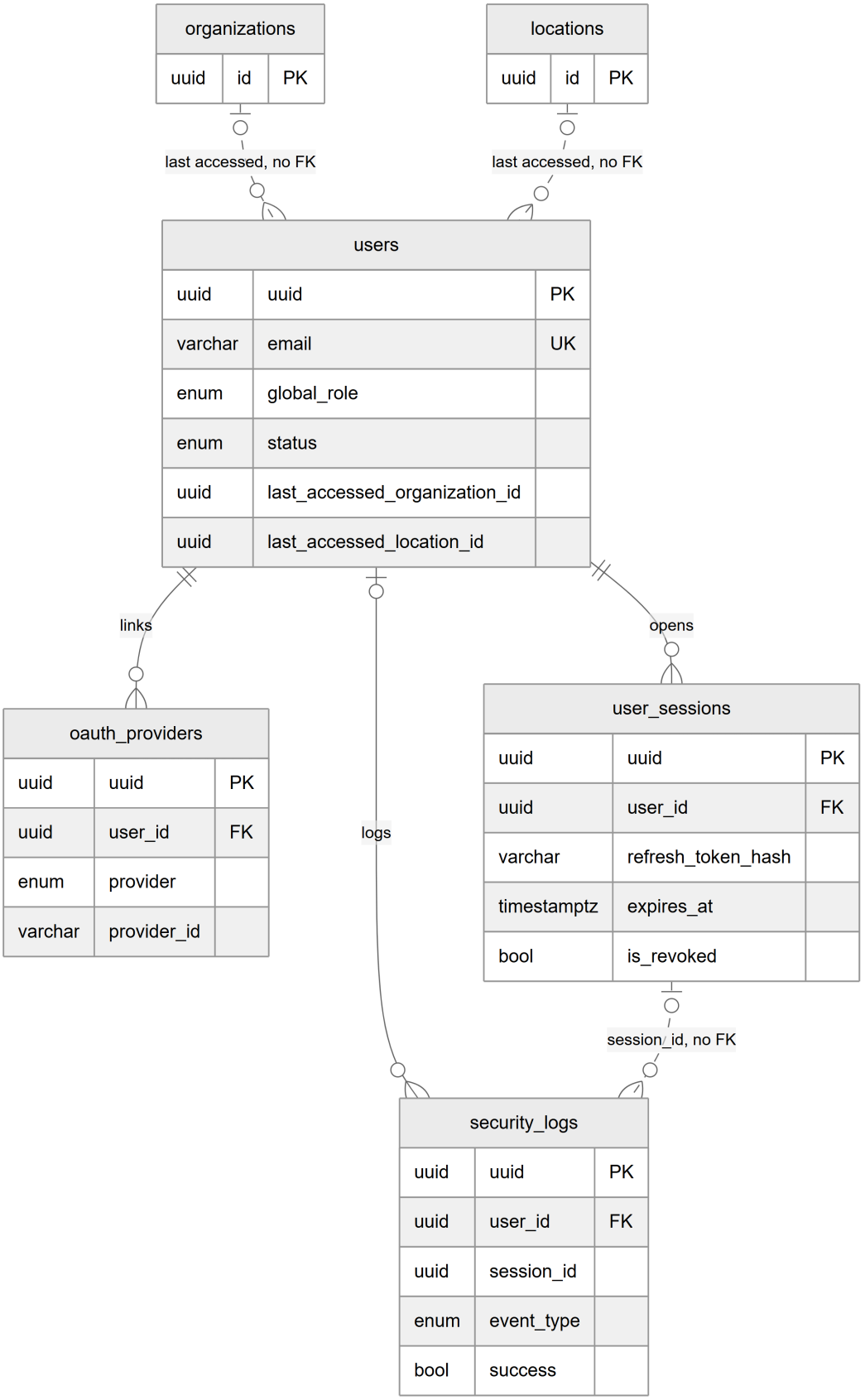


Figure 13.1 — Identity and session tables.

The diagram has `users` in the centre, with the key columns `uuid` (primary key), `email` (unique), `global_role`, `status`, `last_accessed_organization_id` and `last_accessed_location_id`. Under it are three child tables. On the left, `oauth_providers` (`uuid`, `user_id`, `provider`, `provider_id`) hangs from `users` with a solid line

labelled "links": one user, zero or more provider rows, and every provider row must have a user. In the middle, `security_logs` (`uuid` , `user_id` , `session_id` , `event_type` , `success`) hangs from `users` with a solid line labelled "logs" whose user end is optional. On the right, `user_sessions` (`uuid` , `user_id` , `refresh_token_hash` , `expires_at` , `is_revoked`) hangs from `users` with a solid line labelled "opens". A dashed line labelled "session_id, no FK" runs from `user_sessions` to `security_logs` . At the top, `organizations` and `locations` are drawn as context boxes with only their `id` ; each has a dashed line to `users` labelled "last accessed, no FK". Solid lines are foreign keys; dashed lines are logical references the database does not enforce.

13.4 Relationships explained

Foreign-key relationships:

- **`oauth_providers.user_id` → `users` .** Each provider row belongs to exactly one user; a user has zero or more (at most one per provider). Deleting the user deletes their provider rows (ON DELETE CASCADE).
- **`security_logs.user_id` → `users` .** Each log row belongs to zero or one user; a user has zero or more log rows. Deleting the user keeps the log rows and sets `user_id` to null (ON DELETE SET NULL), so the security history survives the account.
- **`user_sessions.user_id` → `users` .** Each session belongs to exactly one user; a user has zero or more sessions. Deleting the user deletes their sessions (ON DELETE CASCADE).

Logical relationships (no foreign key; the database accepts any value):

- **`security_logs.session_id` → `user_sessions` .** The session an event concerns. Zero or one session per log row. If the session row disappears, the log keeps a dangling id.
- **`users.last_accessed_organization_id` → `organizations`** and **`users.last_accessed_location_id` → `locations` .** A convenience pointer for the next login. If the organization or location is removed, the pointer is not cleared by the database.

Links into `users` from other domains (described in their own chapters):

- `organization_ownership.user_id` (CASCADE) makes a user an owner of an organization (Chapter 14).
- `employee_org_location_assignments.user_id` (SET NULL) gives an employee's assignment a staff login (Chapter 16).
- `lab_members.user_id` (RESTRICT) makes a user a member of a lab (Chapter 25). Because of RESTRICT, a user who is a lab member cannot be hard-deleted.
- `user_notifications.user_id` (CASCADE) is the user's in-app inbox (Chapter 24).
- Many tables keep "actor" columns such as `created_by` , `updated_by` , `approved_by` or `recorded_by` . They hold a `users.uuid` but have no foreign key (see Chapter 12).

13.5 Tenant scope

All four tables are global: no `organization_id` , no `location_id` . A user's link to clinics is indirect, through `organization_ownership` and through assignments. This is why `users.email` is unique across the whole Platform rather than per organization. Only the super admin has a platform-wide role, stored in `users.global_role` .

13.6 Ownership and ambiguities

- **`oauth_providers` is registered but unused.** The table is created, but no code path reads or writes it, and external sign-in is disabled. It is a leftover of a planned feature.

- **Two places for a refresh token.** `users.refresh_token` is marked deprecated; the live mechanism is one hashed token per row in `user_sessions`.
- `security_logs.session_id` **has no foreign key**, so a log row can point to a session that no longer exists. This is acceptable for a log but means joins must be outer joins.
- **Append-only by convention.** Nothing in the database prevents an update or delete of `security_logs` rows.

Chapter 14 — Organizations, locations, subscriptions and settings

This chapter describes the tables that define a tenant: the organization (one clinic business), its locations (branches), who owns it, how it pays for the Platform, and where its settings are stored. It is for the product owner who needs to know what a "clinic" is in the database and which table to look at for a given setting. Almost every other table in the Platform points back to `organizations`, and most point to `locations` as well.

14.1 Context

Seven tables belong to this domain. `organizations`, `organization_ownership` and `organization_settings` are owned by the `organizations` module; `locations` and `location_settings` by the `locations` module; `subscription_payments` by the `subscriptions` module; and `config_values` by the `settings-registry` module. The business behaviour of organizations and locations is covered in Chapter 9 and subscriptions in Chapter 45; this chapter covers only the data. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

14.2 The tables

14.2.1 `organizations` — organization (the tenant)

Status: [Implemented]

One row per clinic business. This is the root of tenancy: no other table sits over it. Key columns:

- `name` and `slug` — the display name and a short machine name. `slug` is unique across the Platform.
- `clinic_type` and `specialty_tags` — what kind of clinic this is. `specialty_tags` is a JSON list such as `["general"]` or `["dental"]` and is used to decide which features are relevant.
- `timezone` (default `Asia/Karachi`) and `currency` (default `PKR`) — the organization-wide defaults.
- `subscription_plan` — `free`, `starter`, `professional` or `enterprise`.
- `trial_ends_at` and `paid_until` — the subscription window. When both are null the clinic is "unmanaged" (created by a super admin before subscriptions existed) and has no expiry.
- `max_locations` and `max_users` — optional limits.
- `onboarding_pending` — true for a self-signup clinic that has not yet finished the setup wizard.
- `access_version` — a counter increased by one on every access change in the organization (roles, assignments, feature flags, ownership). Access caches use it as their key; it is written only by the access-version service.
- `is_active` — the soft-delete flag. Deactivating an organization sets it to false; the application does not hard-delete organizations.

Indexes: unique `slug`, and an index on `is_active`.

14.2.2 `organization_ownership` — owner link

Status: [Implemented]

Links a user account to an organization it owns: `user_id`, `organization_id` and `is_primary_owner`. The pair (`user_id`, `organization_id`) is unique, so a user is listed at most once per organization. An owner holds every permission in the organization (Chapter 11). An organization can have more than one owner row; `is_primary_owner` marks the main one.

14.2.3 `organization_settings` — organization-wide settings

Status: [Partial] — stored and editable; the feature switches only hide sections on the location settings screen

One row per organization at most (`organization_id` is unique). It holds:

- Booking defaults: `default_slot_duration` (30 minutes), `advance_booking_days` (30), `cancellation_notice_hours` (24).
- `allow_location_override` — whether locations may set their own booking values. The location settings screen shows the organization's values instead when this is false.
- Six older feature switches: `feature_appointments`, `feature_inventory`, `feature_finance`, `feature_reminders` (all default true), `feature_whatsapp` and `feature_patient_portal` (default false).

The feature switches predate the feature-flag system. Today the location settings screen reads them only to hide its appointments, reminders and finance sections. The guards that actually allow or block a feature read `org_feature_flags` (Chapter 15), not these columns. The row is created with defaults the first time the settings are read.

14.2.4 `locations` — location (branch)

Status: [Implemented]

One row per branch of an organization. Key columns:

- `organization_id` — the owning organization.
- `name` and `code` — the code is unique within the organization (`organization_id` + `code`).
- `token_prefix` — a short prefix for daily tokens (for example "MAIN" giving "MAIN-007"); falls back to `code` when empty.
- `timezone` (default `UTC`) and `currency` (default `USD`). The location's timezone is the one the Platform uses to decide what "today" is for appointments, queues and reports at that branch.
- Address fields, contact fields, `operating_hours` (JSON), `capacity_per_day`, `total_chairs`, `business_model`.
- `is_main_branch` — marks the main branch.
- `manager_id` — a reference to the branch manager with no foreign key; the code does not say whether it holds an employee id or a user id.
- `is_active` — the soft-delete flag; deactivating a location sets it to false.

Indexes: unique (`organization_id`, `code`) and (`organization_id`, `is_active`).

Note that the column defaults for a new location (`UTC`, `USD`) differ from the defaults of a new organization (`Asia/Karachi`, `PKR`). Whatever the creating screen sends overrides the default.

14.2.5 `location_settings` — per-location settings

Status: [Implemented]

One row per location at most (`location_id` is unique). It holds booking values (`appointment_slot_duration`, `max_appointments_per_day`, `advance_booking_days`, `cancellation_notice_hours`), reminder values (`reminder_enabled`, `reminder_days_before`, `reminder_hours_before`, `reminder_channels`), money values

(`tax_rate` and `late_fee_percentage` as `numeric(5,2)` percentages, `payment_gateway`) and branding (`logo_url` , `primary_color` , `theme`). Billing reads `tax_rate` as the sales-tax rate of the active location, and the appointments service reads the booking values. The row is created with defaults the first time the settings are read. `organization_id` is nullable here, unlike most tenant tables, and the auto-create path in the locations service sets only `location_id` , so rows created that way have no organization.

14.2.6 `subscription_payments` — subscription payment

Status: [Implemented]

A clinic pays for the Platform outside the app; a super admin then records the payment here. One row per recorded payment: `organization_id` , `plan` , `months` , `amount` (numeric, in rupees), `method` , an optional `reference` , `paid_at` , the covered period `period_start` and `period_end` , `notes` , and `recorded_by` (the super admin's user id, no foreign key). In the same transaction the service sets `organizations.paid_until` to the period end and `organizations.subscription_plan` to the paid plan. An index on (`organization_id` , `paid_at`) supports the payment history list.

14.2.7 `config_values` — settings registry value

Status: [Implemented] for the keys the code registers

A generic store for settings that cascade from the Platform to an organization to a location. Each row is one setting key at one scope:

- `scope_type` — `platform` , `org` or `location` .
- `scope_id` — the organization id or location id; null for the platform.
- `key` — the setting name, for example `theme.brand_color` .
- `value` — JSON; null means "nothing set here, inherit from the wider scope". A row can therefore exist only to carry a lock.
- `is_locked` — freezes the effective value at this scope so lower scopes cannot override it.
- `version` — increased on every write.

Two unique indexes keep one row per scope and key: (`scope_type` , `scope_id` , `key`), and a partial unique index on `key` where `scope_type = 'platform'` . The partial index is needed because PostgreSQL treats nulls as distinct, so two platform rows with a null `scope_id` would otherwise both be allowed. The keys registered in code today are the theme keys (`theme.app_name` , `theme.brand_color` , `theme.sidebar_style` , `theme.default_mode` , `theme.logo_url` , `theme.logo_dark_url` , `theme.favicon_url`) and `subscription.payment_instructions` . The default of `theme.app_name` is "Clinic Portal". The primary key uses the time-ordered `uuid_generate_v7()` default.

14.3 Entity-relationship diagram

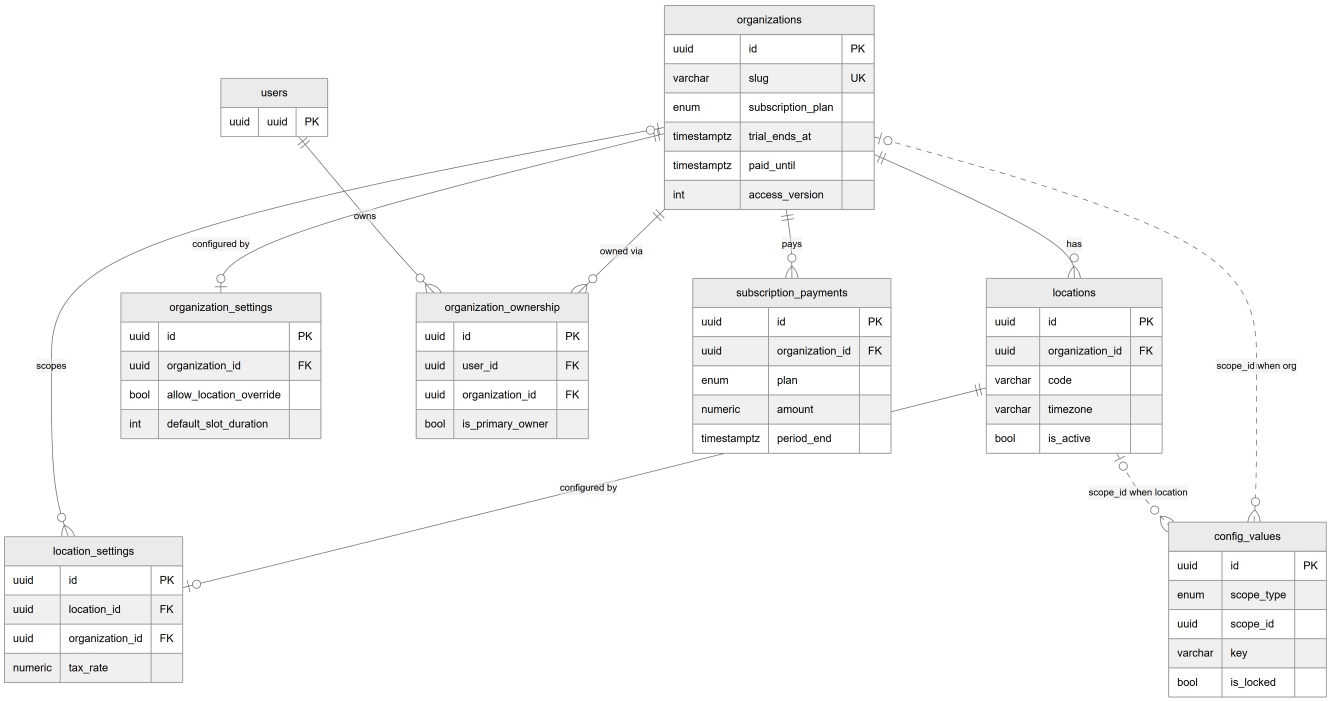


Figure 14.1 — Organizations, locations, ownership, subscriptions and settings.

The diagram places `organizations` at the top, showing `id`, the unique `slug`, `subscription_plan`, `trial_ends_at`, `paid_until` and `access_version`. A small `users` box with only its `uuid` key sits to its left. Solid lines run from `organizations` to five children: `organization_settings` ("configured by", at most one per organization), `organization_ownership` ("owned via"), `subscription_payments` ("pays"), `locations` ("has") and `location_settings` ("scopes", optional on the settings side). `users` also has a solid line to `organization_ownership` labelled "owns". `locations` has a solid line to `location_settings` labelled "configured by", at most one per location. At the bottom right, `config_values` (`id`, `scope_type`, `scope_id`, `key`, `is_locked`) is connected by two dashed lines, one from `organizations` labelled "scope_id when org" and one from `locations` labelled "scope_id when location". Dashed lines mean there is no foreign key.

14.4 Relationships explained

Foreign-key relationships:

- **`locations.organization_id` → `organizations`.** Every location belongs to exactly one organization; an organization has zero or more locations. Deleting the organization deletes its locations (CASCADE).
- **`organization_ownership.organization_id` → `organizations`** and **`organization_ownership.user_id` → `users`.** Each ownership row joins exactly one user to exactly one organization; a user can own several organizations and an organization can have several owners. Deleting either side deletes the ownership row (CASCADE).
- **`organization_settings.organization_id` → `organizations`.** One-to-one: each organization has zero or one settings row, and each settings row belongs to exactly one organization. Deleting the organization deletes the row (CASCADE).
- **`location_settings.location_id` → `locations`.** One-to-one: each location has zero or one settings row. Deleting the location deletes it (CASCADE).
- **`location_settings.organization_id` → `organizations`.** Optional: a settings row may carry its organization. Deleting the organization deletes the row (CASCADE).

- **`subscription_payments.organization_id` → `organizations`.** Each payment belongs to exactly one organization; an organization has zero or more payments. Deleting the organization deletes its payment history (CASCADE).

Logical relationships (no foreign key):

- **`config_values.scope_id` → `organizations` or `locations`.** Polymorphic: `scope_type` says which table the id belongs to, so no single foreign key is possible. Deleting an organization or location leaves its `config_values` rows behind.
- **`locations.manager_id` → `employees` or `users`** (the target is not stated in the code).
- **Actor columns** — `locations.created_by`, `locations.updated_by`, `subscription_payments.recorded_by` and `config_values.updated_by` — hold a user id without a foreign key.
- **`users.last_accessed_organization_id` / `last_accessed_location_id`** point into this domain without a foreign key (Chapter 13).

Links into `organizations` and `locations` from other domains:

- Almost every tenant table in the older modules has `organization_id` → `organizations` with CASCADE, and often `location_id` → `locations` with CASCADE.
- Some newer links use RESTRICT instead: `lab_clinic_links`, `lab_orders` (organization and location), `magic_links`, `order_templates`, `service_mappings`, `lab_service_prices`, `notification_outbox`, and the clinic-specific rows of `form_schemas`, `workflow_templates` and `stage_templates`. An internal lab also points to its owner organization with RESTRICT.
- The queue, billing v2 and lab tables mostly keep `organization_id` with no foreign key at all (Chapter 12).

Because of the RESTRICT links, a hard delete of an organization that has lab, notification or engine rows would be refused by the database; a hard delete of one without them would cascade into most of its data. The application never hard-deletes an organization or a location: it only sets `is_active` to false.

14.5 Tenant scope

- `organizations` is the tenant root and has no tenant column itself.
- `organization_ownership`, `organization_settings`, `locations` and `subscription_payments` are organization-scoped.
- `location_settings` is organization + location scoped.
- `config_values` uses its own cascading scope (`scope_type` + `scope_id`) and is classified as global in the catalog, because platform rows belong to no tenant.

14.6 Ownership and ambiguities

- **Two kinds of feature switch.** `organization_settings.feature_*` columns and `org_feature_flags` rows both describe "is this feature on". Only `org_feature_flags` is enforced (Chapter 15).
- **Two kinds of settings storage.** Typed settings tables (`organization_settings`, `location_settings`, `queue_policies`, `location_billing_settings`) coexist with the generic `config_values` registry. The project's rule is that typed tables stay as they are and new long-tail settings go into the registry.
- **Default mismatch.** A new location defaults to `UTC` and `USD`, while a new organization defaults to `Asia/Karachi` and `PKR`.
- **Specialties stored twice.** `organizations.specialty_tags` and `organizations.clinic_type` describe the clinic's specialties, and the engine's `organization_specialties` table does too (Chapter 26).

- **Legacy baseline.** The never-executed baseline migration describes `organization_admins` and `user_locations` tables that no longer exist; they were replaced by `organization_ownership` and by assignments.

Chapter 15 — Access and features

This chapter describes the tables behind two questions the Platform asks on every request: "is this feature switched on for this clinic and this location?" and "what may this person do here?". It is for the product owner who configures roles and features for clinics, or who needs to explain why a screen is hidden. The behaviour built on these tables — the guards, the permission catalog, the role editor — is explained in Chapters 10 and 11; this chapter covers the stored data.

15.1 Context

Five tables belong to this domain. `feature_catalog`, `org_feature_flags` and `location_feature_overrides` are owned by the `feature-flags` module. `nav_layouts` is owned by the `access` module. `employment_roles` is defined in the `employees` module folder, although the permission model that reads it lives in the `access` module; three modules register it (`employees`, `access` and `feature-flags`). Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

15.2 The tables

15.2.1 `feature_catalog` — feature catalog entry

Status: [Implemented]

The master list of every feature the Platform offers, one row per feature. It is global: the same list for all clinics. Key columns:

- `key` — the feature key, unique, for example `billing.invoices`.
- `name` and `description` — the human label.
- `module` — a grouping such as `billing`, `appointments` or `patients`.
- `specialty_tags` — the clinic types the feature is relevant to (for example `["general"]`, `["dental"]`).
- `enabled_by_default` — whether a clinic gets it switched on when its flags are created.
- `sort_order` and `is_active` — display order, and whether the feature is still offered at all.

The catalog is seeded from code (`backend/src/feature-flags/feature-catalog.seed.ts`); seeding inserts missing keys and does not overwrite existing rows.

15.2.2 `org_feature_flags` — organization feature state

Status: [Implemented]

One row per organization per feature, unique on (`organization_id`, `feature_key`). It holds two independent switches:

- **Entitled** — `is_entitled`, set by the super admin: the organization may use the feature. It defaults to true so that organizations created before entitlements existed kept every feature they had. `entitled_until` turns an entitlement into a trial: from that moment the feature is no longer entitled. `entitled_by` and `entitled_at` record who and when.
- **Enabled** — `is_enabled`, set by the owner within what is entitled. `enabled_by` records who last switched it.

A feature is **active** for an organization only when its catalog entry is active, it is entitled (and the trial has not expired) and it is enabled. `feature_key` is a copy of `feature_catalog.key`, not a foreign key. An index on (`organization_id`, `is_enabled`) supports "all enabled features of this clinic".

15.2.3 `location_feature_overrides` — location override

Status: [Implemented]

The owner's way to switch an organization-active feature off at one location. One row per location per feature, unique on (`location_id`, `feature_key`). `is_disabled = true` turns the feature off at that location. There is no way to turn a feature on at a location if it is off for the organization; the table can only narrow. `organization_id` is copied from the location for tenant-scoped reads and has no foreign key; `feature_key` is a copy of the catalog key; `updated_by` is the user who last changed it.

15.2.4 `employment_roles` — role

Status: [Implemented]

One row per role in an organization: the five system roles seeded from templates (Admin, Doctor, Receptionist, Assistant, Accountant) and any custom roles the clinic creates. Key columns:

- `name` and `slug` — the slug is unique within the organization (`organization_id` + `slug`).
- `permissions` — a JSON object of granted permissions only, `{ permission: 'own' | 'location' | 'org' | true }`. A missing key means no access. The value is the scope at which the permission applies.
- `template_key` — the system template the role was seeded from, such as `doctor` or `receptionist`; empty for custom roles.
- `is_system_role` — true for seeded roles (the column default is true).
- `home_route` — where members of the role land after login, for example `/queue/doctor`.
- `description` and `is_active`.

An index on (`organization_id`, `is_active`) lists a clinic's live roles. The role a person has is not stored on the person: it is the `employment_role_id` of each of their assignments (Chapter 16), so one person can hold different roles at different locations.

15.2.5 `nav_layouts` — sidebar layout

Status: [Implemented] for the clinic default layout; [Not implemented] for per-role layouts

A clinic's arrangement of the sidebar: which navigation keys go into which section, renamed labels and hidden keys, stored as JSON in `layout`. It only arranges keys that the web app's navigation registry defines; it never grants access. `version` increases on every save and `updated_by` records the user.

`role_id` null means the clinic default layout, which is the only kind the current version writes. A row with a `role_id` would be a layout for one role. Two unique indexes keep at most one default per organization (a partial unique index on `organization_id` where `role_id` is null) and at most one layout per role (`organization_id` + `role_id`).

15.3 Entity-relationship diagram

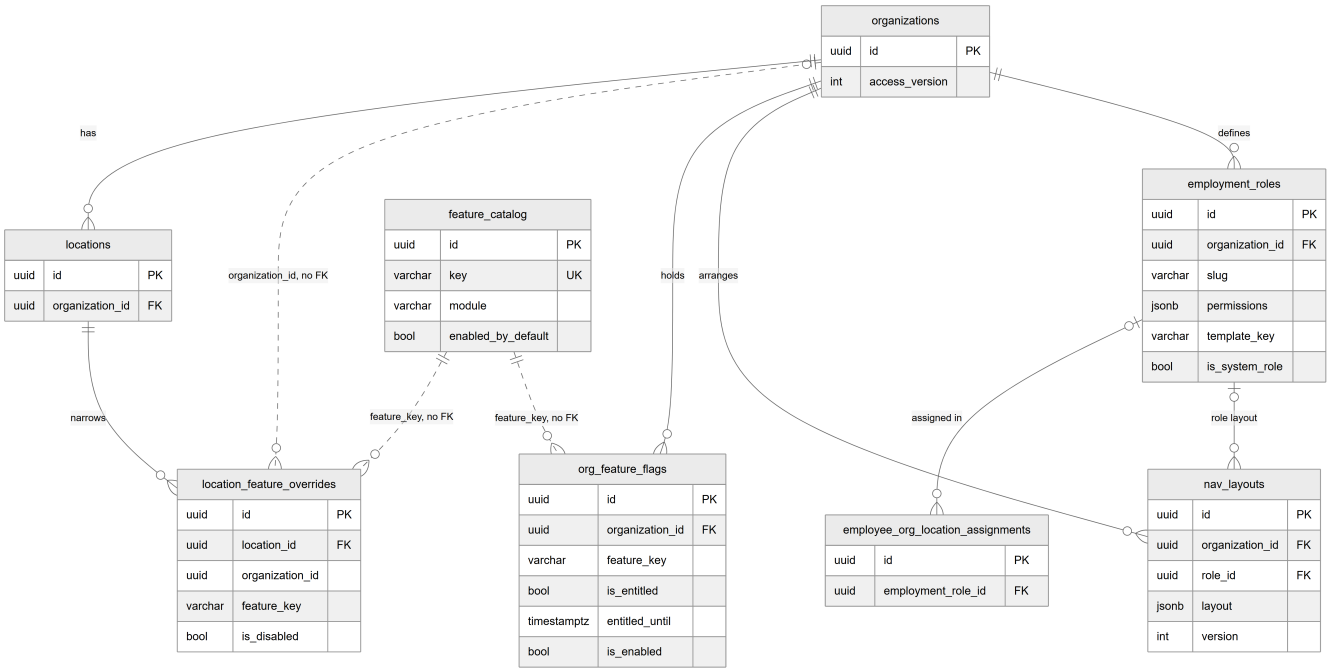


Figure 15.1 — Access and feature tables.

The diagram puts `organizations` (with `id` and `access_version`) at the top. Solid lines run from it to `locations` ("has"), `org_feature_flags` ("holds"), `nav_layouts` ("arranges") and `employment_roles` ("defines"). `employment_roles` (`id`, `organization_id`, `slug`, `permissions`, `template_key`, `is_system_role`) has two solid lines going down: to `nav_layouts` labelled "role layout" and to a context box `employee_org_location_assignments` (only `id` and `employment_role_id`) labelled "assigned in"; both are optional on the role side. `locations` has a solid line to `location_feature_overrides` labelled "narrows". In the middle, `feature_catalog` (`id`, unique `key`, `module`, `enabled_by_default`) has two dashed lines labelled "feature_key, no FK", one to `org_feature_flags` and one to `location_feature_overrides`. A dashed line labelled "organization_id, no FK" also runs from `organizations` to `location_feature_overrides`. Dashed lines are logical links without a foreign key.

15.4 Relationships explained

Foreign-key relationships:

- **`employment_roles.organization_id` → `organizations`.** Each role belongs to exactly one organization; an organization has zero or more roles. Deleting the organization deletes its roles (CASCADE).
- **`nav_layouts.organization_id` → `organizations`.** Each layout belongs to exactly one organization. Deleting the organization deletes its layouts (CASCADE).
- **`nav_layouts.role_id` → `employment_roles`.** Optional: a layout belongs to zero or one role (zero means the clinic default). Deleting a role deletes its own layout (CASCADE); the default layout is unaffected.
- **`employee_org_location_assignments.employment_role_id` → `employment_roles`.** Optional: an assignment has zero or one role; a role is held by zero or more assignments. Deleting a role keeps the assignments and sets their role to null (SET NULL), leaving those people with no role permissions at that location.
- **`org_feature_flags.organization_id` → `organizations`.** Each flag belongs to exactly one organization. Deleting the organization deletes its flags (CASCADE).
- **`location_feature_overrides.location_id` → `locations`.** Each override belongs to exactly one location. Deleting the location deletes its overrides (CASCADE).

Logical relationships (no foreign key):

- **`org_feature_flags.feature_key` → `feature_catalog.key` and `location_feature_overrides.feature_key` → `feature_catalog.key`.** The flag and the override refer to a catalog entry by its key text. The database does not stop a flag for a key that is not in the catalog, and removing a catalog row would not remove its flags.
- **`location_feature_overrides.organization_id` → `organizations`.** A copy of the location's organization, for filtering.
- **Actor columns** — `org_feature_flags.enabled_by` and `entitled_by`, `location_feature_overrides.updated_by`, `nav_layouts.updated_by` — hold a user id without a foreign key.

15.5 Tenant scope

- `feature_catalog` is global (one list for the whole Platform).
- `org_feature_flags`, `employment_roles` and `nav_layouts` are organization-scoped.
- `location_feature_overrides` is organization + location scoped.

Every change to roles, assignments, feature flags or ownership increases `organizations.access_version` (Chapter 14). The resolver that turns these tables into a user's permissions caches its result per user, organization, location and access version, so a change takes effect as soon as the version moves.

15.6 Ownership and ambiguities

- **Where `employment_roles` lives.** The entity file is in `employees/`, the permission logic that reads `permissions` is in `access/`, and the project's own overview once described "employee role definitions" as part of `organizations/`. Treat `access` as the functional owner and `employees` as the place the file sits.
- **Feature keys are text, not foreign keys.** A typo in a key would not be caught by the database.
- **Older feature switches.** `organization_settings` has six `feature_*` columns that the guards do not read (Chapter 14).
- **Per-role sidebar layouts** are supported by the schema (`role_id`, its foreign key and its unique index) but the current version only writes the clinic default.

Chapter 16 — Staff and payroll

This chapter describes the tables that record the clinic's staff: the person, where and in which role they work, their weekly hours, leave and day-by-day availability, and the monthly payroll record. It is for the product owner who needs to know how one person can work at several clinics, where a doctor's working hours come from, and what a payroll row contains. The employee and payroll modules are described in Chapters 37 and 38; salary and commission plans are in Chapter 22.

16.1 Context

The domain separates the **person** from the **job**. `employees` holds the person once for the whole Platform. Each place the person works — one organization and one location, with one role — is an **assignment** in `employee_org_location_assignments`. Schedules, leave, availability overrides and payroll all hang from the person and name the location they apply to. Six of the tables are owned by the `employees` module (it also owns `employment_roles`, which Chapter 15 describes) and `payroll_records` is owned by the `payroll` module. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

16.2 The tables

16.2.1 `employees` — employee (the person)

Status: [Implemented]

One row per person who works at any clinic on the Platform. It holds only personal data: `full_name`, `phone`, an optional national identity number (`cnic`), `gender`, `dob`, `profile_picture_url` and `is_active`. There is no `organization_id`: the same person record is shared by every organization they work for. For that reason `phone` and `cnic` are **unique across the whole Platform**, not per clinic.

16.2.2 `employee_org_location_assignments` — assignment

Status: [Implemented]

One row per person per location they work at. This is where the job lives. Key columns:

- `employee_id`, `organization_id`, `location_id` — who and where. (`employee_id`, `location_id`) is unique: a person has at most one assignment per location.
- `employment_role_id` — the role at this location (Chapter 15); nullable.
- `user_id` — the staff login used for this assignment; nullable, because not every employee logs in.
- `has_portal_access` — whether this person may log in at this location.
- `designation` and `employment_type` (`full_time`, `part_time`, `visiting`, `contract`).
- `employee_code` — unique within the organization (`organization_id` + `employee_code`).
- `joining_date`, `is_primary` and `is_active` (the soft-delete flag for this job).
- `token_prefix` — the doctor's queue-token prefix at this location (for example "A" giving "A-007"); empty means the first letter of the doctor's name.
- `gross_salary` — deprecated. Salary now lives in versioned compensation plans (Chapter 22); the column is kept for old rows and no longer written once a plan exists.

Indexes support lookups by user, by location and active flag, and by employee and organization.

16.2.3 `employee_schedules` — weekly working hours

Status: [Implemented]

The regular week of a person at one location: one row per weekday (`day_of_week`), with `is_working` , `start_time` , `end_time` and `slot_duration` (default 30 minutes). (`employee_id` , `location_id` , `day_of_week`) is unique, so there is one row per weekday per location. The appointments service reads these rows when it works out a doctor's availability.

16.2.4 `employee_leaves` — leave request

Status: [Implemented]

One row per leave request: `leave_type` (`annual` , `sick` , `unpaid` , `maternity` , `paternity` , `other`), `start_date` , `end_date` , `total_days` , `status` (`pending` , `approved` , `rejected` , `cancelled`), the reason, a review note, `approved_by` and `reviewed_at` . Indexes support "leave in this date range", "pending leave at this location" and "this person's leave by status".

16.2.5 `doctor_availability_overrides` — availability exception

Status: [Partial] — recorded and edited on the employee's availability tab; the appointments and queue services do not read it

An exception to the weekly schedule for one date at one location: `date` , `is_available` (default false, meaning "not available that day"), a reason and optional `override_hours` (JSON) for different hours that day. (`employee_id` , `location_id` , `date`) is unique: one exception per person, location and day. Only the employees module reads this table; the appointments service builds availability from `employee_schedules` without consulting these exceptions.

16.2.6 `payroll_records` — payroll record

Status: [Implemented]

One row per person per location per month. (`employee_id` , `location_id` , `month` , `year`) is unique, so the same person can have a separate record at each location for the same month. Key columns:

- `month` , `year` and `period_label` .
- `base_salary_paisa` , `commission_paisa` , `total_paisa` — the breakdown in integer paisa, where total = base salary + commission.
- `salary_amount` — the total in rupees (`numeric(10,2)`), kept for older screens.
- `status` (`pending` , `paid` , `on_hold`), `paid_at` , `payment_method` (`cash` , `bank_transfer` , `cheque`), `payment_reference` , `paid_by` , `notes` .

The unique key changed over time. The doctor-compensation migration drops the older key (`employee_id` , `month` , `year`) and adds the location-aware one; the entity matches the new key.

16.3 Entity-relationship diagram

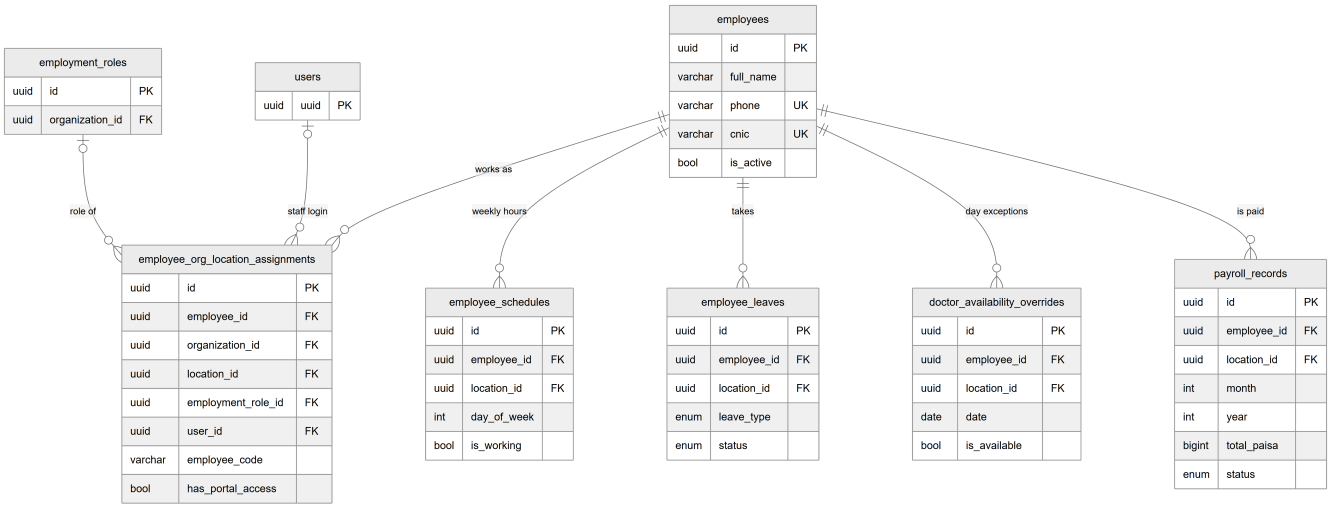


Figure 16.1 — Staff and payroll tables.

The diagram has `employees` at the top centre, showing `id`, `full_name`, `phone` and `cnic` (both marked unique) and `is_active`. Five solid lines fan out from it to its children, each "one employee to zero or more rows": `employee_org_location_assignments` ("works as"), `employee_schedules` ("weekly hours"), `employee_leaves` ("takes"), `doctor_availability_overrides` ("day exceptions") and `payroll_records` ("is paid"). The assignment box, on the left, lists `id`, `employee_id`, `organization_id`, `location_id`, `employment_role_id`, `user_id`, `employee_code` and `has_portal_access`; over it two context boxes, `employment_roles` and `users`, connect to it with optional lines labelled "role of" and "staff login". Each child box shows its `employee_id` and `location_id` foreign keys plus its key business columns (`day_of_week` and `is_working`; `leave_type` and `status`; `date` and `is_available`; `month`, `year`, `total_paisa` and `status`). The links from these tables to `organizations` and `locations` are not drawn, to keep the picture readable; they are listed in the next section.

16.4 Relationships explained

Foreign-key relationships inside the domain:

- **`employee_org_location_assignments.employee_id` → `employees`.** Each assignment belongs to exactly one person; a person has zero or more assignments. Deleting the person deletes the assignments (CASCADE).
- **`employee_org_location_assignments.employment_role_id` → `employment_roles`.** Optional, zero or one role. Deleting the role sets it to null (SET NULL).
- **`employee_org_location_assignments.user_id` → `users`.** Optional, zero or one staff login; a user can be linked to several assignments. Deleting the user sets it to null (SET NULL).
- **`employee_schedules.employee_id`, `employee_leaves.employee_id`, `doctor_availability_overrides.employee_id`, `payroll_records.employee_id` → `employees`.** Each row belongs to exactly one person; a person has zero or more of each. Deleting the person deletes them all (CASCADE).

Tenant foreign keys (not drawn in the figure):

- **`organization_id` → `organizations`** on assignments, schedules, leaves, availability overrides and payroll records: mandatory, CASCADE.
- **`location_id` → `locations`** on the same five tables: mandatory, CASCADE. Deactivating a location never triggers this, because the application only soft-deletes locations.

Logical relationships (no foreign key): the actor columns `created_by` (on assignments, schedules, leaves, overrides and payroll), `approved_by` on leaves and `paid_by` on payroll hold a user id.

Links into this domain from other domains:

- **Into `employees`:** `appointments.doctor_id` and `queue_sessions.doctor_id` (RESTRICT), `charges.doctor_id`, `service_prices.doctor_id` and `lab_orders.doctor_id` (RESTRICT), `prescriptions.doctor_id` (SET NULL), `expenses.recorded_by` and `patient_credits.purchased_by` (SET NULL), and the inventory columns `patient_consumptions.performedBy`, `purchase_orders.createdBy` and `stock_movements.performedBy` (NO ACTION, which behaves like RESTRICT).
- **Into `employee_org_location_assignments`:** `compensation_plans.assignment_id` (CASCADE), and `charges.performing_assignment_id`, `commission_entries.assignment_id` and `commission_settlements.assignment_id` (RESTRICT).
- **Into `payroll_records`:** `commission_settlements.payroll_record_id`, one-to-one and RESTRICT: a payroll record that settled commissions cannot be deleted while the settlement exists.

Because of these RESTRICT links, a person who is the doctor on any appointment, queue session, charge, special price or lab order, or who appears in inventory records, cannot be hard-deleted; the database refuses. The employee service catches that refusal and replies that the employee "has linked inventory records" and should be deactivated instead — the message names inventory even when the blocking row is an appointment or a charge.

16.5 Tenant scope

- `employees` is global: one person record for the whole Platform. The tenant link lives on the assignment.
- `employee_org_location_assignments`, `employee_schedules`, `employee_leaves`, `doctor_availability_overrides` and `payroll_records` are organization + location scoped, with real foreign keys on both columns.

16.6 Ownership and ambiguities

- **Platform-wide uniqueness of personal data.** Because `employees.phone` and `employees.cnic` are unique across the Platform, the database refuses a second person record with the same phone or national identity number, even in a different clinic. The design intends one person record shared by many clinics; how the employee screens handle a person already known elsewhere is covered in Chapter 37.
- **Code comment versus schema.** The comment on the employee delete method says that appointments and invoices keep their rows with the doctor set to null. The schema says otherwise: `appointments.doctor_id`, `queue_sessions.doctor_id` and `charges.doctor_id` are RESTRICT, so such a delete is refused.
- **Deprecated salary column.** `employee_org_location_assignments.gross_salary` is superseded by `compensation_plans`.
- **Two money forms on one row.** `payroll_records` keeps the total in paisa and in rupees; the paisa columns are authoritative for new code.

Chapter 17 — Patients

This chapter describes the patient record and the fourteen sub-record tables that hang from it: clinical history, charts, vitals, plans, documents, consents, contacts, recalls, referrals, feedback, insurance and the per-location visit summary. It is for the product owner who needs to know where a given piece of patient information is stored and what happens to it when a patient is deactivated or deleted. The patient module itself — screens, import, export and permissions — is described in Chapter 28.

17.1 Context

All fifteen tables are owned by the `patients` module. The patient is an organization-level record: one clinic business has one record per patient, shared by all of its locations. Most sub-records also remember the location where they were created. One of the fourteen sub-record tables, `patient_prescriptions`, is the legacy prescription table; it is summarised here and described in full in Chapter 19 next to the current prescription tables. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

17.2 The patient record

17.2.1 `patients` — patient

Status: [Implemented]

One row per patient per organization. Key columns:

- `organization_id` — the owning organization.
- `patient_number` — the clinic's own patient number, unique within the organization (`organization_id` + `patient_number`).
- `full_name`, `dob`, `gender`, `blood_group`, `marital_status`, `occupation`.
- Contact fields: `phone` (required, **not** unique — two patients can share a family phone), a WhatsApp number, email, address and city. A national identity number column also exists. These hold personal data and are never reproduced in this handbook.
- `preferred_language` (`urdu` or `english`, default `urdu`) and `communication_pref` (`whatsapp`, `sms`, `call`, `email`, default `whatsapp`).
- `primary_location_id` — the patient's home branch; nullable.
- `source` (`walk_in`, `referral`, `online`, `social_media`, `google`, `other`) and a free-text `referred_by_text`.
- Minor and guardian fields, `medical_history` and `allergies` as free text, emergency-contact fields, `internal_notes` and `tags` (a JSON list).
- `is_active` — the activate/deactivate switch used by the patient screens.
- `is_blacklisted` and `blacklist_reason`.
- `deleted_at` — a TypeORM soft-delete column. The permanent-delete action does not use it; it removes the row.
- `user_id` — a reference to a user account for the patient, with no foreign key. The code does not say what fills it.

Indexes support search by name, phone and national identity number, the active list (`organization_id` + `is_active`), the blacklist filter (`organization_id` + `is_blacklisted`) and lookups by primary location.

17.3 Clinical sub-records

17.3.1 `patient_dental_info` — dental profile

Status: [Implemented]

At most one row per patient (`patient_id` is unique): last check-up date, gum condition, sensitivity, brushing and flossing habits, smoking and tobacco chewing, orthodontic history, prosthetics, previous extractions and surgeries, X-ray notes and dentist notes. It carries `organization_id` but no location: the profile belongs to the patient, not to a branch.

17.3.2 `patient_teeth_chart` — teeth chart snapshot

Status: [Implemented]

One row per charting. `chart_data` is JSON keyed by tooth number in FDI notation (for example `"11"`), each holding condition, treatment, surface and notes; `periodontal_chart` is optional JSON. `is_initial_chart` marks the first chart, `recorded_at` the moment. `dentist_id` records who charted (target table not stated in the code) and `appointment_id` the visit, both without a foreign key.

17.3.3 `patient_vitals` — vitals reading

Status: [Implemented]

One row per measurement: blood pressure (`bp_systolic` , `bp_diastolic`), pulse, temperature, oxygen saturation, weight, height, BMI, risk flags (diabetic, hypertensive, pregnant, on blood thinners, on antibiotics), anaesthesia type and dose, notes and `recorded_at` . `recorded_by` and `anesthesia_administered_by` are user ids without a foreign key; `appointment_id` links the visit without a foreign key.

17.3.4 `patient_treatment_plans` — treatment plan

Status: [Partial] — backend endpoints exist and billing reads the table; no web screen calls the create or update endpoints

A planned course of work for a patient: `plan_number` (unique within the organization), `plan_name` , description, `estimated_cost` , `discount_amount` and `final_estimated_cost` (numeric, rupees), `status` (`draft` , `active` , `completed` , `cancelled` , `on_hold`), start, expected end and actual end dates, `patient_accepted` and `patient_accepted_at` , notes, and `dentist_id` (no foreign key). It has a `deleted_at` soft-delete column. Planned work is different from performed work, which is stored in `treatments` (Chapter 18).

17.3.5 `treatment_plan_items` — treatment plan item

Status: [Partial] — as for its parent plan

One procedure inside a plan: `procedure_name` , `procedure_code` , `tooth_number` , `tooth_surface` , `estimated_sessions` and `completed_sessions` , `estimated_cost` , `status` (`pending` , `in_progress` , `completed` , `skipped` , `postponed`), `sequence_order` , `assigned_to` (a user id, no foreign key) and notes. This table has **no organization column**; it is isolated only through its plan. An index on (`plan_id` , `sequence_order`) returns items in order.

17.3.6 `patient_documents` — patient document

Status: [Implemented]

One row per uploaded file: `document_type` (X-ray, CBCT scan, report, prescription, consent form, insurance card, identity-card copy, before photo, after photo, other), file name, file URL, thumbnail URL, size, MIME type, `tooth_number`, `taken_date`, `is_visible_to_patient`, notes and `uploaded_by` (a user id, no foreign key). It has a `deleted_at` soft-delete column.

17.3.7 `patient_consents` — signed consent

Status: [Implemented]

One row per consent signed: `consent_type` (general treatment, extraction, implant, root canal, orthodontic, sedation, photo release), `consent_version`, the full `consent_text`, `is_agreed`, `agreed_at`, `signed_by_name`, `signed_by_type` (default `patient`), an optional signature image URL and `witnessed_by`. The entity comment declares the table immutable: a revoked consent is a new row with `is_agreed = false`, never an update. The database does not enforce this; it is an application rule.

17.3.8 `patient_prescriptions` — legacy quick prescription

Status: [Implemented] (legacy; still readable and writable from the patient's Clinical tab)

The older, simpler prescription table, with medicines stored as a JSON list. The patient's Clinical tab shows it in a collapsed "Legacy quick-Rx history" section next to the current prescription list. Its columns and its relation to the current `prescriptions` table are described in Chapter 19.

17.4 Relationship and administrative sub-records

17.4.1 `patient_visits` — per-location visit summary

Status: [Partial] — the table exists and a read endpoint returns it; no code writes it and no screen calls the endpoint

Designed as one summary row per patient per location (`patient_id` + `location_id` unique): `first_visit_date`, `last_visit_date`, `visit_count`, `total_spent` (numeric, rupees) and `is_preferred_location`. The encounter entity's comment calls it a patient-to-location aggregate that is "left untouched". No service inserts or updates rows, so on a new database it stays empty. The clinical record of each consultation is `patient_encounters` (Chapter 18), not this table.

17.4.2 `patient_communications` — communication log

Status: [Implemented]

One row per contact with the patient: `type` (`sms`, `whatsapp`, `call`, `email`, `in_person`), `direction` (inbound or outbound), `purpose` (reminder, confirmation, cancellation, recall reminder, follow-up, billing, general, marketing, emergency), subject, message body, `status` (`pending`, `sent`, `delivered`, `read`, `failed`, `received`), sent, delivered and read times, `sent_by`, `is_automated`, `automation_trigger` and an external message id. Rows are written only through the patient module's create endpoint; nothing on the Platform sends these messages. Notifications on the Platform are in-app only and use their own tables (Chapter 24).

17.4.3 `patient_recalls` — recall

Status: [Partial] — backend endpoints exist; no web screen calls them

A reminder that the patient is due back: `recall_type` (six-month check-up, annual cleaning, post-treatment follow-up, implant review, orthodontic review, custom), `custom_name`, `due_date`, up to three reminder-sent timestamps, `status` (`scheduled`, `reminder_sent`, `contacted`, `booked`, `completed`, `ignored`, `overdue`), notes, `dentist_id` and `appointment_id` (no foreign keys).

17.4.4 `patient_referrals` — patient referral

Status: [Partial] — the patient screen lists referrals; no screen creates them

Records that one patient referred another: `referring_patient_id` (required), `referred_patient_id` (set once the referred person is registered), the referred person's name and phone before registration, `referral_date`, reward fields (`reward_given`, `reward_amount` in numeric rupees, `reward_type`, `reward_given_at`) and `status` (`pending`, `registered`, `rewarded`, `cancelled`). It is organization-scoped with no location.

17.4.5 `patient_feedback` — patient feedback

Status: [Partial] — the patient screen lists feedback; no screen creates it

One row per rating: `rating` (integer), `category` (doctor, service, facility, billing, waiting time, general), text, `is_anonymous`, `is_published`, a staff response with `responded_by` and `responded_at`, and `dentist_id` and `appointment_id` without foreign keys.

17.4.6 `patient_insurance` — insurance policy

Status: [Partial] — backend endpoints exist; no web screen calls them

One row per policy: provider name, plan name, policy number, policy holder, group number, coverage details and percentage, `max_annual_benefit` and `used_benefit` (numeric, rupees), valid-from and valid-until dates, card images and `is_active`. It is organization-scoped with no location.

17.5 Entity-relationship diagrams

The fifteen tables are drawn in two diagrams so that each stays readable.

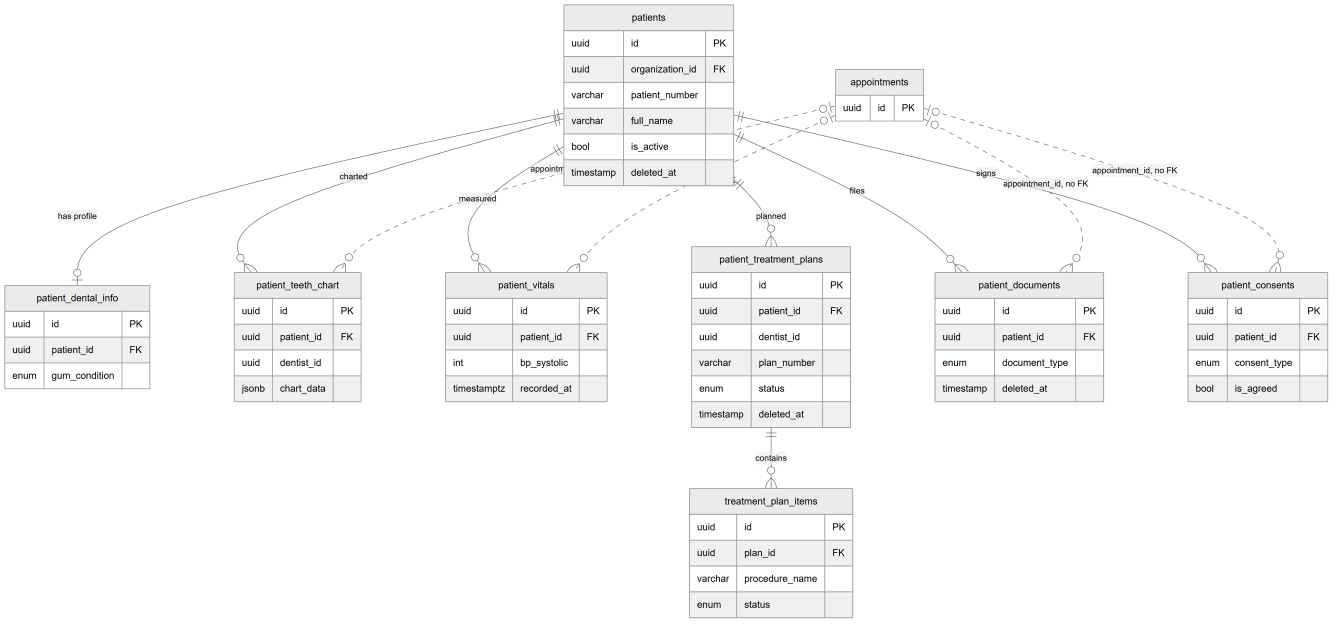


Figure 17.1 — Patient clinical sub-records.

The first diagram has `patients` at the top centre with `id`, `organization_id`, `patient_number`, `full_name`, `is_active` and `deleted_at`. Six solid lines go down from it to `patient_dental_info` ("has profile", at most one per patient), `patient_teeth_chart` ("charted"), `patient_vitals` ("measured"), `patient_treatment_plans` ("planned"), `patient_documents` ("files") and `patient_consents` ("signs"). Under `patient_treatment_plans`, a solid line labelled "contains" leads to `treatment_plan_items` (`id`, `plan_id`, `procedure_name`, `status`). A small `appointments` context box with only `id` sits to the right of `patients`; four dashed lines labelled "appointment_id, no FK" run from it to the teeth chart, vitals, documents and consents boxes. Each child box shows its `patient_id` foreign key and one to three business columns such as `chart_data`, `bp_systolic`, `plan_number`, `document_type` or `consent_type`. The legacy `patient_prescriptions` table is drawn in Figure 19.3, and the tenant links to `organizations` and `locations` are not drawn.

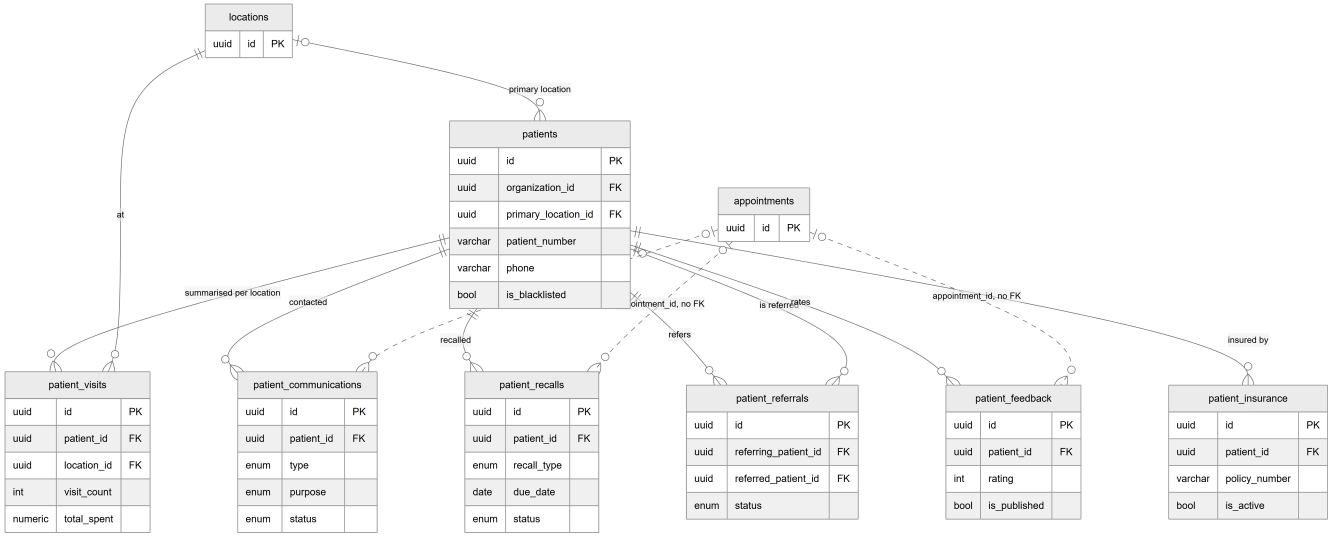


Figure 17.2 — Patient relationship and administrative sub-records.

The second diagram again centres on `patients`, this time showing `primary_location_id`, `phone` and `is_blacklisted`. A `locations` context box at the top left has an optional solid line to `patients` labelled "primary location" and a solid line to `patient_visits` labelled "at". From `patients`, solid lines go down to `patient_visits` ("summarised per location", with `visit_count` and `total_spent`), `patient_communications` ("contacted"), `patient_recalls` ("recalled"), `patient_referrals` (two lines: "refers" for the referring patient and an optional "is referred" for the referred patient), `patient_feedback` ("rates") and `patient_insurance` ("insured by"). An `appointments` context box connects with dashed "appointment_id, no FK" lines to communications, recalls and feedback.

17.6 Relationships explained

Foreign-key relationships inside the domain:

- **`patient_dental_info.patient_id` → `patients`.** One-to-one: a patient has zero or one dental profile. Deleting the patient deletes it (CASCADE).
- **`patient_teeth_chart`, `patient_vitals`, `patient_treatment_plans`, `patient_documents`, `patient_consents`, `patient_prescriptions`, `patient_visits`, `patient_communications`, `patient_recalls`, `patient_feedback`, `patient_insurance` → `patients` (`patient_id`).** Each row belongs to exactly one patient; a patient has zero or more rows in each. Deleting the patient deletes them all (CASCADE).
- **`treatment_plan_items.plan_id` → `patient_treatment_plans`.** Each item belongs to exactly one plan; a plan has zero or more items. Deleting the plan row deletes its items (CASCADE). A soft delete of the plan (`deleted_at`) does not touch the items.

- `patient_referrals.referring_patient_id` → `patients`. Mandatory. Deleting the referring patient deletes the referral (CASCADE).
- `patient_referrals.referred_patient_id` → `patients`. Optional. Deleting the referred patient keeps the referral and sets the link to null (SET NULL).
- `patients.primary_location_id` → `locations`. Optional. Deleting the location sets it to null (SET NULL).
- `patient_visits.location_id` → `locations`. Mandatory; deleting the location deletes its summary rows (CASCADE). The pair (`patient_id`, `location_id`) is unique.

Tenant foreign keys (not drawn):

- `organization_id` → `organizations` on every table except `treatment_plan_items`: mandatory, CASCADE.
- `location_id` → `locations` on teeth charts, vitals, treatment plans, documents, consents, legacy prescriptions, visits, communications, recalls and feedback: mandatory, CASCADE. `patients`, `patient_dental_info`, `patient_referrals` and `patient_insurance` have no location column.

Logical relationships (no foreign key):

- `appointment_id` → `appointments` on communications, consents, documents, feedback, legacy prescriptions, recalls, teeth charts and vitals. The database accepts an appointment id that does not exist.
- `dentist_id` on teeth charts, treatment plans, recalls and feedback. The code does not say whether it holds an employee id or a user id.
- `patients.user_id` → `users`.
- **Actor columns** — `created_by`, `sent_by`, `witnessed_by`, `uploaded_by`, `responded_by`, `prescribed_by`, `recorded_by`, `anesthesia_administered_by`, `assigned_to` — hold user ids.

Links into `patients` from other domains:

- **CASCADE** (deleted with the patient): `appointments`, `queue_entries`, `patient_encounters`, `treatments`, `prescriptions`, `patient_credits`, `patient_consumptions` (inventory) and `patient_comm_prefs` (notifications, one-to-one).
- **RESTRICT** (block the delete): `charges`, `invoices`, `payments`, `refunds`, `lab_orders`, `lab_invoices`, `lab_payments` and `lab_refunds`.
- **No foreign key:** `ledger_entries.patient_id`, `payment_allocations.patient_id`, `ledger_journal_lines.patient_id`, `lab_credit_notes.patient_id`, and the polymorphic history columns of `audit_logs` and `status_history`.

What a permanent delete does. Deleting a patient removes the patient row and every cascading child listed in this section, inside one transaction that first writes an audit row with the patient's last state and a count of linked records. If the patient has any billing or lab-billing row, the RESTRICT links make the database refuse the delete. Deactivating (`is_active = false`) removes nothing.

17.7 Tenant scope

- `patients`, `patient_dental_info`, `patient_referrals` and `patient_insurance` are organization-scoped.
- The other ten sub-record tables are organization + location scoped.
- `treatment_plan_items` has no tenant column and is scoped only through `patient_treatment_plans`; queries must join the plan.

17.8 Ownership and ambiguities

- **Four "visit" concepts.** `patient_visits` (an unused per-location summary), `appointments` (bookings), `queue_entries` (a place in the queue) and `patient_encounters` (the clinical record of a consultation) all describe visits. Chapter 27 compares them.
- **Three planning tables.** `patient_treatment_plans` holds planned work, `treatments` holds performed work, and `plan_templates` holds reusable plans (Chapter 19).
- **Code comment versus schema.** The comment on the patient delete method says invoices and payments are removed by cascade. In the schema, `invoices`, `charges`, `payments` and `refunds` reference the patient with RESTRICT, so a patient with billing history cannot be deleted. No code catches this refusal; what the user sees is [Unverified].
- **Uncertain targets.** The `dentist_id` columns may hold employee ids or user ids.
- **Immutability by convention.** `patient_consents` is declared immutable only in a comment.
- **Two soft-delete mechanisms on one table.** `patients` has both `is_active` and `deleted_at`; the screens use `is_active`.

Chapter 18 — Appointments, queue and encounters

This chapter describes the tables that follow a patient through one visit: the booking, the place in a doctor's queue, the realtime and safety tables the queue needs, the clinical record of the consultation, and the treatments performed. It is for the product owner who needs to know how a visit is stored at each step and which database rules stop double bookings or two patients being with one doctor at once. The behaviour is described in Chapters 29 (appointments), 30 (queue) and 31 (consultation); this chapter covers the data.

18.1 Context

Nine tables belong to this domain, owned by four modules: `appointments` owns `appointments`; `queue` owns `queue_sessions`, `queue_entries`, `queue_events`, `queue_policies`, `outbox` and `idempotency_keys`; `encounters` owns `patient_encounters`; and `treatments` owns `treatments`. A visit moves through them in order: an appointment is booked, at check-in (or walk-in registration) a queue entry is created in the doctor's queue session for that day, when the doctor starts the consultation an encounter is created, and treatments performed are recorded against the encounter. All queue changes go through one service, the `QueueOrchestrator`, which writes the queue tables, an event and an outbox message in one transaction. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

18.2 The tables

18.2.1 `appointments` — appointment

Status: [Implemented]

One row per booking. Key columns:

- `organization_id`, `location_id`, `patient_id`, `doctor_id` (an employee).
- `type` (`consultation`, `follow_up`, `treatment`, `checkup`, `emergency`, `cleaning`) and `is_emergency`.
- `status` — `scheduled`, `confirmed`, `checked_in`, `completed`, `cancelled` or `no_show`.
- `start_time`, `end_time` (with time zone) and `duration_minutes`.
- `reason_for_visit` (shown in the queue row) and `notes`.
- `cancel_reason_code` (`no_show`, `patient_left`, `other`) and a free-text `cancel_reason`.
- Moment columns: `checked_in_at`, `completed_at`, `cancelled_at`, `no_show_at`.

The key database rule is the EXCLUDE constraint `no_doctor_overlap`: for the same `doctor_id`, no two appointments may have overlapping time ranges `[start_time, end_time)`, unless the appointment is `cancelled` or `no_show`. It needs the `btree_gist` extension. This is why a doctor can never be double-booked, even if two receptionists book at the same moment. Indexes support the doctor's calendar (`doctor_id`, `start_time`, `end_time`), the location's day (`location_id`, `start_time`), and filters by status, patient and organization.

18.2.2 `queue_sessions` — queue session

Status: [Implemented]

One doctor's queue for one service day at one location. Key columns:

- `organization_id`, `location_id`, `doctor_id`.
- `service_date` — the location-local date of the service day.
- `status` — `waiting_for_doctor`, `open`, `paused`, `closed`, with `pause_reason`, `started_at` and `closed_at`.
- `last_token_number` — the token counter (CHECK: never negative). Because every queue change first locks this row, tokens are gap-free and never duplicated.
- `token_prefix` — the doctor's token prefix, copied when the session is created.
- `smart_queue_enabled` — copied from the queue policy when the session is created; afterwards it changes only through the mode endpoint.
- `last_call_reason`, `event_seq` (the counter that numbers `queue_events`) and `version` (for optimistic checks).

(`organization_id`, `location_id`, `doctor_id`, `service_date`) is unique: one session per doctor per location per day.

18.2.3 `queue_entries` — queue entry

Status: [Implemented]

A patient's place in one queue session. It is created only at check-in of a booked appointment or at walk-in registration — never at booking. Key columns:

- `queue_session_id`, `patient_id`, and `appointment_id` (set for booked visits, null for walk-ins).
- `token_number` and `queue_type` (`appointment` or `walk_in`).
- `status` — `waiting`, `called`, `in_consultation`, `on_hold`, `skipped`, `completed`, `cancelled`, `transferred`, `aborted`.
- `priority_class` (`normal` or `authorized_urgent`) with `urgent_since` and `urgent_reason`.
- `checked_in_at` and `sort_anchor_at` (the first-in-first-out position: the check-in time, or the rejoin time after a rejoin at the end).
- Moment columns: `called_at`, `consultation_started_at`, `hold_started_at`, `returned_from_hold_at`, `skipped_at`, `finished_at`.
- `converted_from_appt_id` (the appointment a walk-in was converted from) and `transferred_from_id` (the entry this one replaced on a transfer to another doctor).
- `version` for optimistic checks.

The database rules on this table are the strongest in the Platform:

- **Unique token per session** — (`queue_session_id`, `token_number`).
- **One active entry per appointment** — a partial unique index on `appointment_id` where the status is waiting, called, in consultation, on hold or skipped.
- **One active entry per patient per session** — a partial unique index on (`queue_session_id`, `patient_id`) for the same statuses.
- **At most one patient with the doctor** — a partial unique index on `queue_session_id` where the status is `called` or `in_consultation`.
- **CHECK: token number greater than zero.**
- **CHECK: type matches appointment** — `queue_type = 'appointment'` exactly when `appointment_id` is set.
- **CHECK: urgent needs a reason** — `priority_class` is `normal`, or both `urgent_since` and `urgent_reason` are set.

A partial index on (`queue_session_id`, `sort_anchor_at`) for waiting entries makes "who is next" fast.

18.2.4 `queue_events` — queue event

Status: [Implemented]

An audit log of every queue change, written in the same transaction as the change. Key columns: `queue_session_id`, `queue_entry_id`, `seq` (from the session's `event_seq`, unique per session, so screens can apply events in order and notice gaps), `event_type` (for example `CHECKED_IN`, `CALLED`, `SKIPPED`, `MODE_CHANGED`), `from_status`, `to_status`, reason code and text, `suggested_entry_id` (when staff selected a patient manually, the entry the selection engine would have called), `actor_id`, `actor_role`, `metadata` and `occurred_at`. The primary key is a bigint identity. The entity comment calls the table append-only (corrections are new events); the database does not enforce it.

18.2.5 `queue_policies` — queue policy

Status: [Implemented]

Queue settings. A row with `location_id` null is the organization default; a row with a location overrides it for that location. Two partial unique indexes keep one default per organization and one override per location. Columns and their CHECK rules:

- `smart_queue_enabled` — smart or manual selection for new sessions.
- `early_window_min` (CHECK 0 to 240), `late_grace_min` (CHECK at least 0), `max_late_min` (CHECK at least `late_grace_min`) — how early and how late a booked patient may check in.
- `walkin_starvation_min` (CHECK greater than 0) — the walk-in waiting threshold, in minutes, used by the selection engine (Chapter 30 explains the rule).
- `skipped_rejoin_policy` — `end`, `original_position` or `manual`.
- `call_timeout_sec` — null turns automatic skip-after-call off; otherwise at least 30 seconds.
- `manual_select_reason_required`.
- `service_day_cutoff` — the local time a service day starts, for example "04:00" for a clinic that works past midnight.

The timezone is not stored here; it is always the location's own `locations.timezone`.

18.2.6 `outbox` — realtime outbox

Status: [Implemented]

The transactional outbox for realtime screens. A queue change writes its realtime message here in the same transaction; a relay later publishes it to the socket room named in `topic` (for example a session room or a location's display room) and sets `published_at`. Screens therefore never see a change that rolled back and never miss one that committed. A partial index on `id` where `published_at` is null lets the relay find unpublished rows quickly. A cleanup job deletes published rows older than 7 days, every day at 03:15 server time. This table is separate from `notification_outbox` (Chapter 24).

18.2.7 `idempotency_keys` — idempotency record

Status: [Implemented]

Stores the result of a mutation keyed by the client's `Idempotency-Key` header, so a retried request returns the saved answer instead of doing the work twice (for example, a check-in whose reply was lost returns the same token). The primary key is (`organization_id`, `key`); other columns are `request_hash`, `response_status`,

`response_body` and `created_at`. The queue writes it, and the shared idempotency service used by the lab modules and the workflow engine writes to the same table with keys namespaced as `scope:key`. The same daily 03:15 cleanup job deletes every row older than 24 hours.

18.2.8 `patient_encounters` — encounter

Status: [Implemented]

The clinical record of one consultation, created when the doctor starts the consultation on a queue entry. `queue_entry_id` is unique: exactly one encounter per queue entry. `appointment_id` is set for booked visits and null for walk-ins. Other key columns: `patient_id`, `organization_id`, `location_id`, `encounter_date`, `reason` (why the patient actually came, which can differ from the booking), `chief_complaint`, `examination`, `diagnosis`, `advice` and `follow_up_date`.

18.2.9 `treatments` — treatment (work performed)

Status: [Implemented]

A treatment actually carried out. It always belongs to a patient and may be linked to the encounter it was performed in. Key columns: `patient_id`, `encounter_id`, `plan_template_id` (the plan template it was based on), `treatment_name`, `notes`, `date`, `service_id` (the catalog service billing uses to price the charge) and `cost` (a legacy fee in numeric rupees, used by billing only when no service is set). Planned work lives in `patient_treatment_plans` (Chapter 17), not here.

18.3 Entity-relationship diagrams

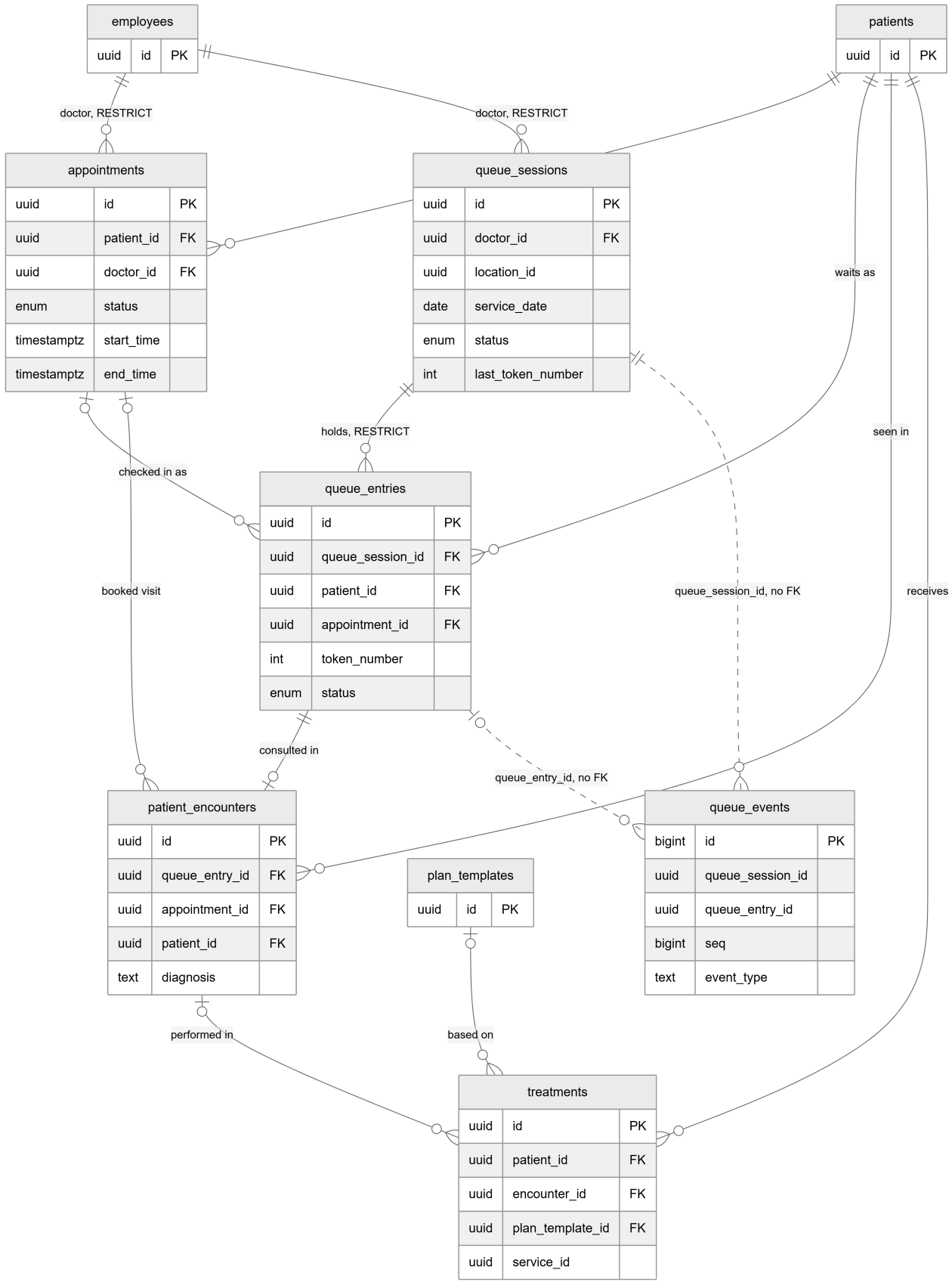


Figure 18.1 — From booking to treatment.

The diagram has two context boxes at the top, `employees` on the left and `patients` on the right, each showing only `id`. From `employees`, two solid lines labelled "doctor, RESTRICT" go to `appointments` and `queue_sessions`. From `patients`, solid lines go to `appointments` ("books"; this label is hidden behind the `queue_sessions` box in

the rendered image), `queue_entries` ("waits as"), `patient_encounters` ("seen in") and `treatments` ("receives"). `appointments` (`id` , `patient_id` , `doctor_id` , `status` , `start_time` , `end_time`) has optional solid lines to `queue_entries` ("checked in as") and to `patient_encounters` ("booked visit"). `queue_sessions` (`id` , `doctor_id` , `location_id` , `service_date` , `status` , `last_token_number`) has a solid line labelled "holds, RESTRICT" to `queue_entries` (`id` , `queue_session_id` , `patient_id` , `appointment_id` , `token_number` , `status`). `queue_entries` has a solid line labelled "consulted in" to `patient_encounters` (at most one encounter per entry). `queue_events` (`id` , `queue_session_id` , `queue_entry_id` , `seq` , `event_type`) is joined by two dashed lines, "queue_session_id, no FK" from `queue_sessions` and "queue_entry_id, no FK" from `queue_entries` . At the bottom, `treatments` (`id` , `patient_id` , `encounter_id` , `plan_template_id` , `service_id`) has optional solid lines from `patient_encounters` ("performed in") and from a `plan_templates` context box ("based on"). Tenant links are not drawn.

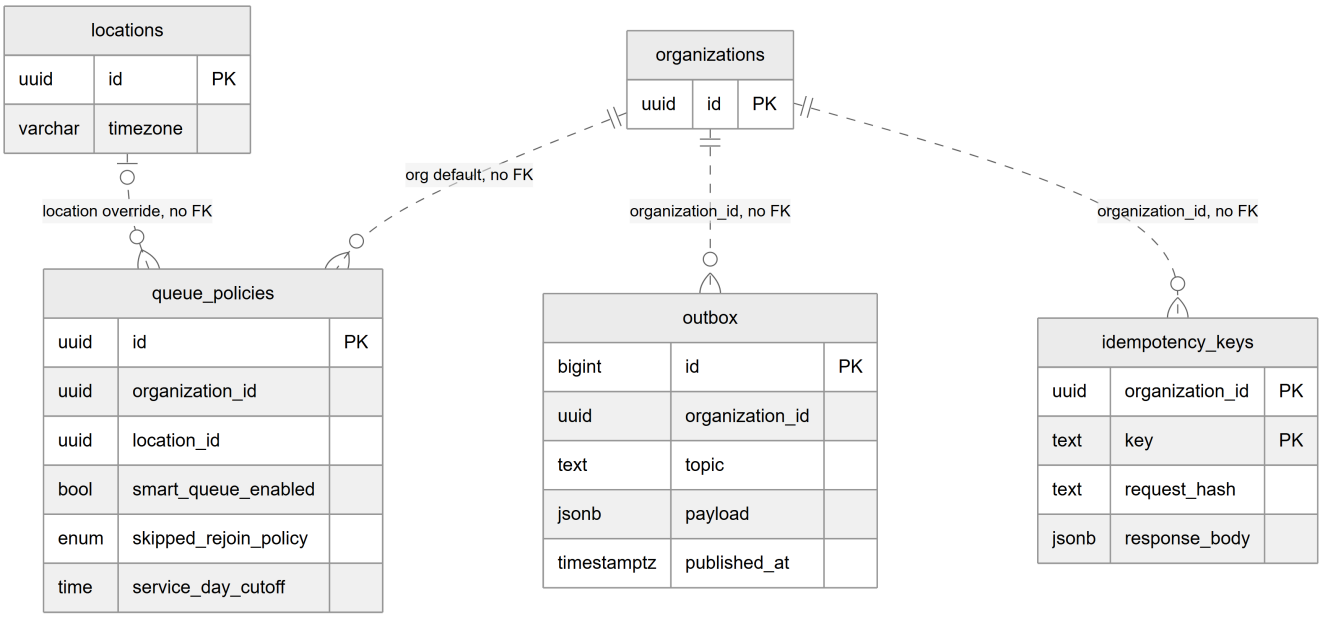


Figure 18.2 — Queue support tables.

The second diagram shows the three queue tables that have no foreign key at all. `organizations` (with `id`) is joined by dashed lines to `queue_policies` ("org default, no FK"), `outbox` ("organization_id, no FK") and `idempotency_keys` ("organization_id, no FK"). `locations` (with `id` and `timezone`) is joined by a dashed line to `queue_policies` labelled "location override, no FK". The boxes show `queue_policies` with `organization_id` , `location_id` , `smart_queue_enabled` , `skipped_rejoin_policy` and `service_day_cutoff` ; `outbox` with a bigint `id` , `organization_id` , `topic` , `payload` and `published_at` ; and `idempotency_keys` with its two-part key `organization_id` + `key` , `request_hash` and `response_body` .

18.4 Relationships explained

Foreign-key relationships:

- **`appointments.patient_id` → `patients` .** Each appointment has exactly one patient; a patient has zero or more. Deleting the patient deletes the appointments (CASCADE).
- **`appointments.doctor_id` → `employees` .** Each appointment has exactly one doctor. A doctor with appointments cannot be deleted (RESTRICT).
- **`appointments.organization_id` → `organizations`** and **`appointments.location_id` → `locations` .** Mandatory, CASCADE.
- **`queue_sessions.doctor_id` → `employees` .** Each session has exactly one doctor. RESTRICT.

- `queue_entries.queue_session_id` → `queue_sessions`. Each entry belongs to exactly one session; a session has zero or more entries. A session with entries cannot be deleted (RESTRICT).
- `queue_entries.patient_id` → `patients`. Mandatory. CASCADE.
- `queue_entries.appointment_id` → `appointments`. Optional; an appointment has zero or more entries over time (only one active, by the partial unique index). Deleting the appointment deletes its entries (CASCADE).
- `patient_encounters.queue_entry_id` → `queue_entries`. Each encounter has exactly one queue entry; an entry has zero or one encounter (unique). Deleting the entry deletes the encounter (CASCADE).
- `patient_encounters.appointment_id` → `appointments`. Optional. Deleting the appointment keeps the encounter and sets the link to null (SET NULL).
- `patient_encounters.patient_id` → `patients`, `organization_id` → `organizations`, `location_id` → `locations`. Mandatory, CASCADE.
- `treatments.patient_id` → `patients`. Mandatory, CASCADE.
- `treatments.encounter_id` → `patient_encounters`. Optional; an encounter has zero or more treatments. Deleting the encounter keeps the treatments with the link set to null (SET NULL).
- `treatments.plan_template_id` → `plan_templates`. Optional. SET NULL.
- `treatments.organization_id` → `organizations` and `location_id` → `locations`. Mandatory, CASCADE.

Logical relationships (no foreign key):

- `queue_events.queue_session_id` → `queue_sessions`, `queue_entry_id` → `queue_entries`, `suggested_entry_id` → `queue_entries`, `actor_id` → `users`.
- `queue_entries.converted_from_appt_id` → `appointments` and `transferred_from_id` → `queue_entries` (a link to another entry of the same table).
- `treatments.service_id` → `services` (billing's catalog, Chapter 20).
- **Tenant columns without a foreign key:** `organization_id` on `queue_sessions`, `queue_entries`, `queue_events`, `queue_policies` and `outbox`; `location_id` on `queue_sessions` and `queue_policies`. `idempotency_keys.organization_id` is part of its primary key, also without a foreign key.
- **Actor columns** — `created_by` and `updated_by` on appointments and encounters, `created_by` on treatments.

Links into this domain from other domains:

- `prescriptions.appointment_id` and `prescriptions.encounter_id` (SET NULL) — Chapter 19.
- `patient_consumptions.appointmentId` (SET NULL) — Chapter 23.
- Logical only: `charges.encounter_id` and `charges.appointment_id`, `lab_orders.appointment_id` and `lab_orders.treatment_id`, `order_item_stages.appointment_id`, and the `appointment_id` columns of the patient sub-records.

18.5 Tenant scope

- `appointments`, `queue_sessions`, `queue_policies`, `patient_encounters` and `treatments` are organization + location scoped.
- `queue_entries`, `queue_events`, `outbox` and `idempotency_keys` carry only `organization_id`; an entry's location is the location of its session.
- The queue tables keep their tenant columns without foreign keys; the orchestrator writes them.

18.6 Ownership and ambiguities

- **Four "visit" tables.** An appointment is the booking, a queue entry is a place in line, an encounter is the clinical record, and `patient_visits` is an unused summary (Chapter 17). Chapter 27 compares them.
- **Two outboxes.** `outbox` carries realtime screen updates; `notification_outbox` carries notifications.
- **Shared idempotency table.** `idempotency_keys` lives in the queue module, but the shared idempotency service used by labs and the workflow engine writes to it too, and the queue's daily cleanup removes those rows after 24 hours as well.
- **Append-only by convention.** `queue_events` is declared append-only only in a comment.
- **RESTRICT on the doctor.** Because sessions and appointments block deletion of their doctor, a doctor who has worked cannot be hard-deleted; deactivation is the only option (Chapter 16).

Chapter 19 — Prescriptions and plans

This chapter describes the tables behind prescriptions and the clinic's reusable libraries: the prescription itself with its medicine lines and version history, the medicine directory, prescription presets, print layouts, doctors' saved signatures, plan templates, and the older prescription table that still exists. It is for the product owner who needs to know where a printed prescription's data comes from and why several tables have "plan" or "template" in their name. The prescription module is described in Chapter 32 and treatment plans in Chapter 33.

19.1 Context

Ten tables are covered here. Eight are owned by the `prescriptions` module: `prescriptions`, `prescription_medicines`, `prescription_history`, `prescription_templates`, `doctor_signatures`, `medicine_directory`, `treatment_plan_templates` and `treatment_plan_template_medicines`. `plan_templates` is owned by the `plan-templates` module. The legacy `patient_prescriptions` table is owned by the `patients` module. The patient's own planned work, `patient_treatment_plans` and `treatment_plan_items`, also belongs to the patients domain and is described in Chapter 17.

The names are easy to confuse, so here is the mapping from screen to table. In the web app's prescription library, **Medicines** is `medicine_directory`, **Presets** (ready-made medicine sets) is `treatment_plan_templates` despite its name, and **Print layouts** is `prescription_templates`. The sidebar entry **Treatment Plans** opens the plan template library, which is `plan_templates`. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

19.2 The prescription record

19.2.1 `prescriptions` — prescription

Status: [Implemented]

The prescription a doctor writes, signs and shares. Key columns:

- `prescription_number` — formatted `RX-<year>-<5 digits>`, for example `RX-2026-00001`.
- `organization_id`, `location_id`, `patient_id` (all required).
- `appointment_id`, `encounter_id`, `doctor_id` — the visit and the doctor; all optional.
- `prescription_template_id` (the print layout) and `treatment_plan_template_id` (the preset it was pre-filled from); both optional.
- Clinical text: `diagnosis`, `chief_complaint`, `further_plan`, and instructions in English and in Urdu.
- `vitals` — a JSON copy of the readings (blood pressure, pulse, temperature, oxygen, weight, height, BMI, respiratory rate).
- `signature_image_url` and `signed_at`.
- `status` — `DRAFT`, `FINALIZED` or `AMENDED`; `version` starts at 1.
- `pdf_url` — a cached generated PDF, cleared when the record is amended.

`prescription_number` is unique across the whole Platform, while the number generator counts per organization (it reads the organization's latest number and adds one). Indexes support lookups by patient, appointment, encounter, doctor and by (`organization_id`, `location_id`).

19.2.2 `prescription_medicines` — prescription medicine line

Status: [Implemented]

One line per medicine on a prescription: `medicine_name` and its Urdu name, `type`, `dosage`, `frequency`, `frequency_timing` (a JSON list such as `["morning","evening"]`), `meal_relation` (before, after or with a meal, empty stomach, none), `duration`, special instructions in English and Urdu, and `sort_order`. `medicine_directory_id` keeps a link to the directory entry it came from, but the name and every other value are copied onto the line, so a finalized prescription stays a faithful record even if the directory entry is renamed or switched off later. This table has no tenant column; it is isolated through its prescription.

19.2.3 `prescription_history` — prescription change history

Status: [Implemented]

One row per change event: `version`, `change_type` (`CREATED`, `AMENDED`, `FINALIZED`, `PRINTED`, `SHARED`), `changed_by` (a user id), `change_notes`, and `previous_data` — a full JSON snapshot of the prescription and its medicines before the change (null for `CREATED`). Any past version can be shown from this table alone. The entity comment calls it immutable; the database does not enforce that. It has no tenant column and is isolated through its prescription.

19.3 The libraries

19.3.1 `medicine_directory` — medicine directory

Status: [Implemented] (screen label "Medicines")

The clinic's own medicine catalogue, which powers the search while writing a prescription. Each row: `name`, `generic_name`, `name_urdu`, `type` (tablet, capsule, syrup, injection, cream, drops and others), `strength`, `manufacturer`, default dosage, frequency, duration and meal relation, Urdu instructions, and `is_active`. Entries are switched off with `is_active`, never hard-deleted, so old prescriptions still resolve. Indexes cover (`organization_id`, `name`) and (`organization_id`, `is_active`).

19.3.2 `treatment_plan_templates` — prescription preset

Status: [Implemented] (screen label "Presets")

A reusable prescription protocol, for example "Root canal aftercare": `name`, `description`, `category` (forty categories such as pain relief, post-extraction, implant, pregnancy-safe), `specialty_tags`, `further_plan`, instructions in English and Urdu, and `is_active`. Choosing a preset in the prescription builder pre-fills the medicines and texts; the doctor can still edit before finalizing. Despite the name, it is not a treatment plan.

19.3.3 `treatment_plan_template_medicines` — preset medicine line

Status: [Implemented]

The medicine lines of a preset, with the same columns as `prescription_medicines` and the same denormalized copy of the directory values. It has no tenant column and is isolated through its preset.

19.3.4 `prescription_templates` — print layout

Status: [Implemented] (screen label "Print layouts")

The look of the printed prescription: `name`, `is_default`, `page_size` (`A4`, `A5`, `LETTER`, `CUSTOM`), `orientation`, custom width and height in millimetres, and JSON blocks for `header` (logo, clinic and doctor names in English and Urdu, qualifications, registration number), `footer` (address, timings, disclaimer) and `styling` (colours, fonts, toggles, margins). `location_id` null means an organization-wide layout usable at any location; a location can also have its own default, and several layouts per location are allowed. Nothing in the database limits a location to one default; an index on (`location_id`, `is_default`) only speeds the lookup.

19.3.5 `doctor_signatures` — saved doctor signature

Status: [Implemented]

A doctor's saved signature image (`signature_image_url`, optional `signature_data`), attached to prescriptions they finalize unless they draw a new one. `employee_id` is unique: one signature per owner, and saving again updates the row in place. Despite its name, the column holds the signed-in **user's** id, not an employee id — the controller passes the user id from the login token. Because the uniqueness is on that id alone, a user who works for two organizations has one signature row, stored with the organization where it was first saved, and the read path does not filter by organization.

19.3.6 `plan_templates` — plan template

Status: [Implemented] (sidebar label "Treatment Plans")

A predefined treatment plan a doctor can pick, for example a root canal with its usual steps: `name`, `category` (seventeen dental categories such as endodontics, prosthodontics, orthodontics, emergency), `description`, `items` (a JSON list of procedures) and `is_active` (inactive templates are hidden from the picker but kept). When a treatment is recorded from the patient screen, the chosen template is saved in `treatments.plan_template_id`. `organization_id` has no foreign key.

19.3.7 `patient_prescriptions` — legacy quick prescription

Status: [Implemented] (legacy; shown and still creatable in the "Legacy quick-Rx history" section of the patient's Clinical tab)

The older prescription table, which coexists with `prescriptions` while consumers move to the newer schema. Each row has `prescription_number` (unique within the organization), `prescription_date`, chief complaint, diagnosis, `medications` as one JSON list, special instructions, `follow_up_date`, print tracking (`is_printed`, `printed_at`, `print_count`), `prescribed_by` (a user id) and `appointment_id`, both without foreign keys. It is organization + location scoped with foreign keys to `patients`, `organizations` and `locations`, all CASCADE.

19.4 Entity-relationship diagrams

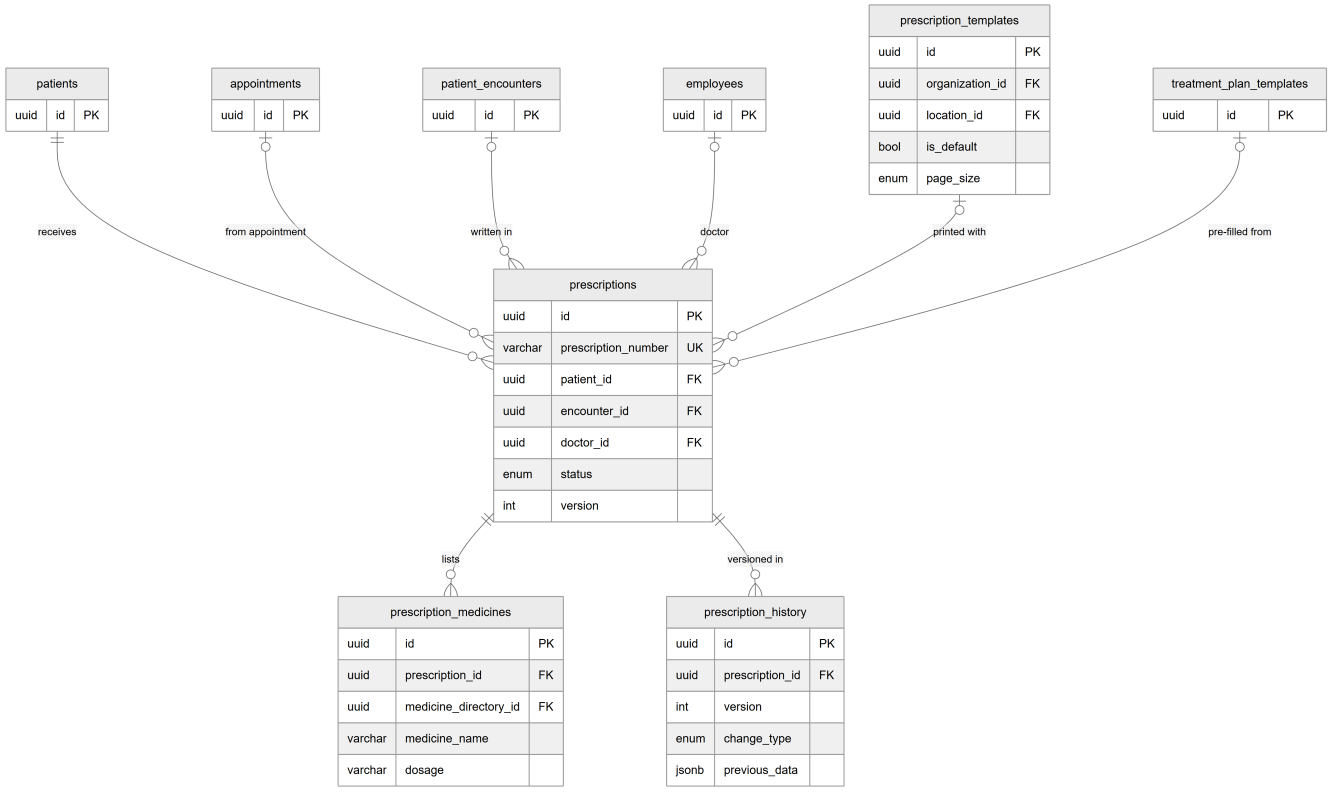


Figure 19.1 — The prescription record.

The diagram centres on `prescriptions`, showing `id`, the unique `prescription_number`, `patient_id`, `encounter_id`, `doctor_id`, `status` and `version`. Six context boxes sit in a row over it, each with an optional or mandatory solid line into `prescriptions`: `patients` ("receives", mandatory), `appointments` ("from appointment"), `patient_encounters` ("written in"), `employees` ("doctor"), `prescription_templates` ("printed with", shown with `organization_id`, `location_id`, `is_default` and `page_size`) and `treatment_plan_templates` ("pre-filled from"). Under `prescriptions`, two solid lines lead to its children: `prescription_medicines` ("lists", with `prescription_id`, `medicine_directory_id`, `medicine_name` and `dosage`) and `prescription_history` ("versioned in", with `prescription_id`, `version`, `change_type` and `previous_data`).

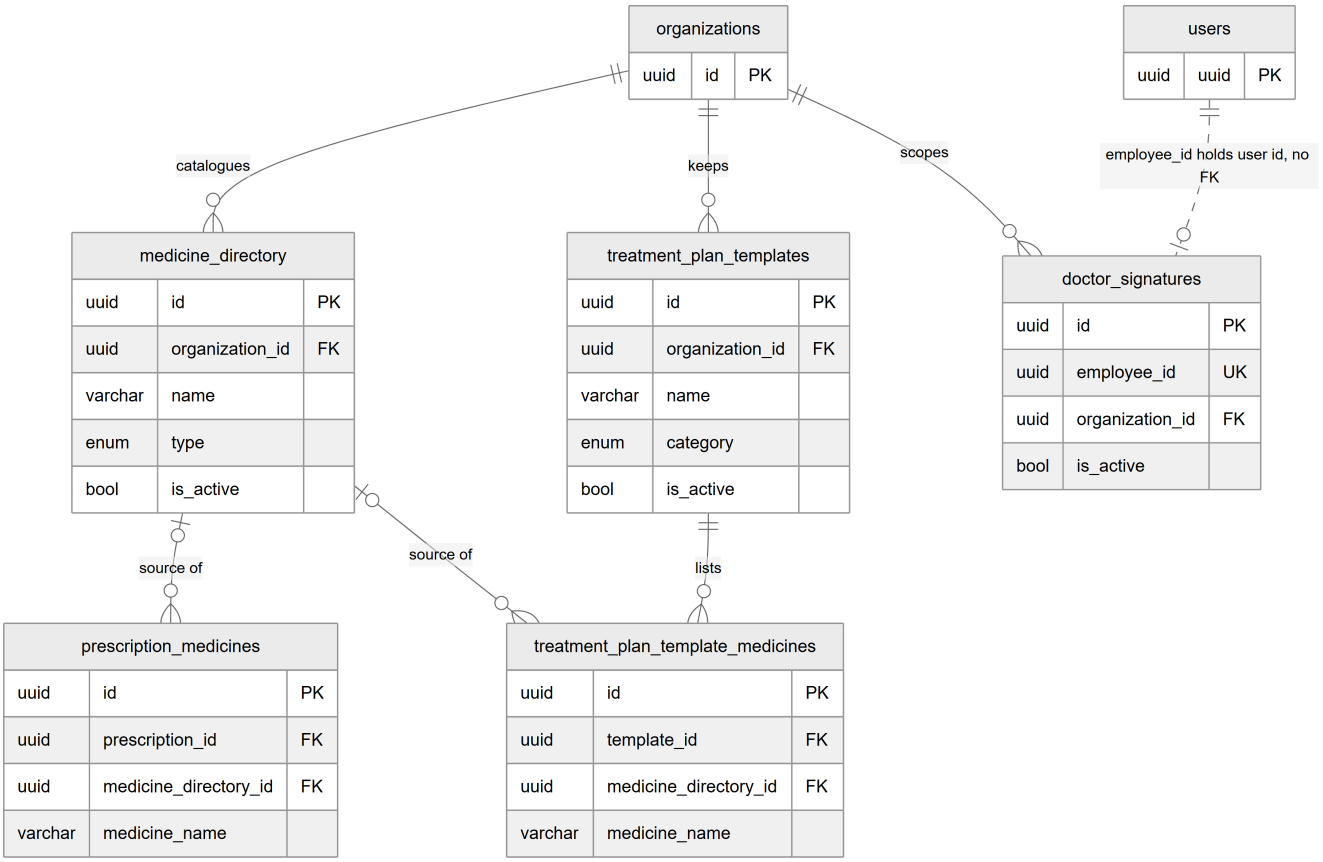


Figure 19.2 — Prescription libraries.

The second diagram shows the libraries. `organizations` has solid lines to `medicine_directory` ("catalogues"), `treatment_plan_templates` ("keeps") and `doctor_signatures` ("scopes"). `medicine_directory` (`id` , `organization_id` , `name` , `type` , `is_active`) has two optional solid lines labelled "source of", one to `prescription_medicines` and one to `treatment_plan_template_medicines` . `treatment_plan_templates` has a solid line labelled "lists" to `treatment_plan_template_medicines` . A `users` box is joined to `doctor_signatures` (`id` , the unique `employee_id` , `organization_id` , `is_active`) by a dashed line labelled "employee_id holds user id, no FK".

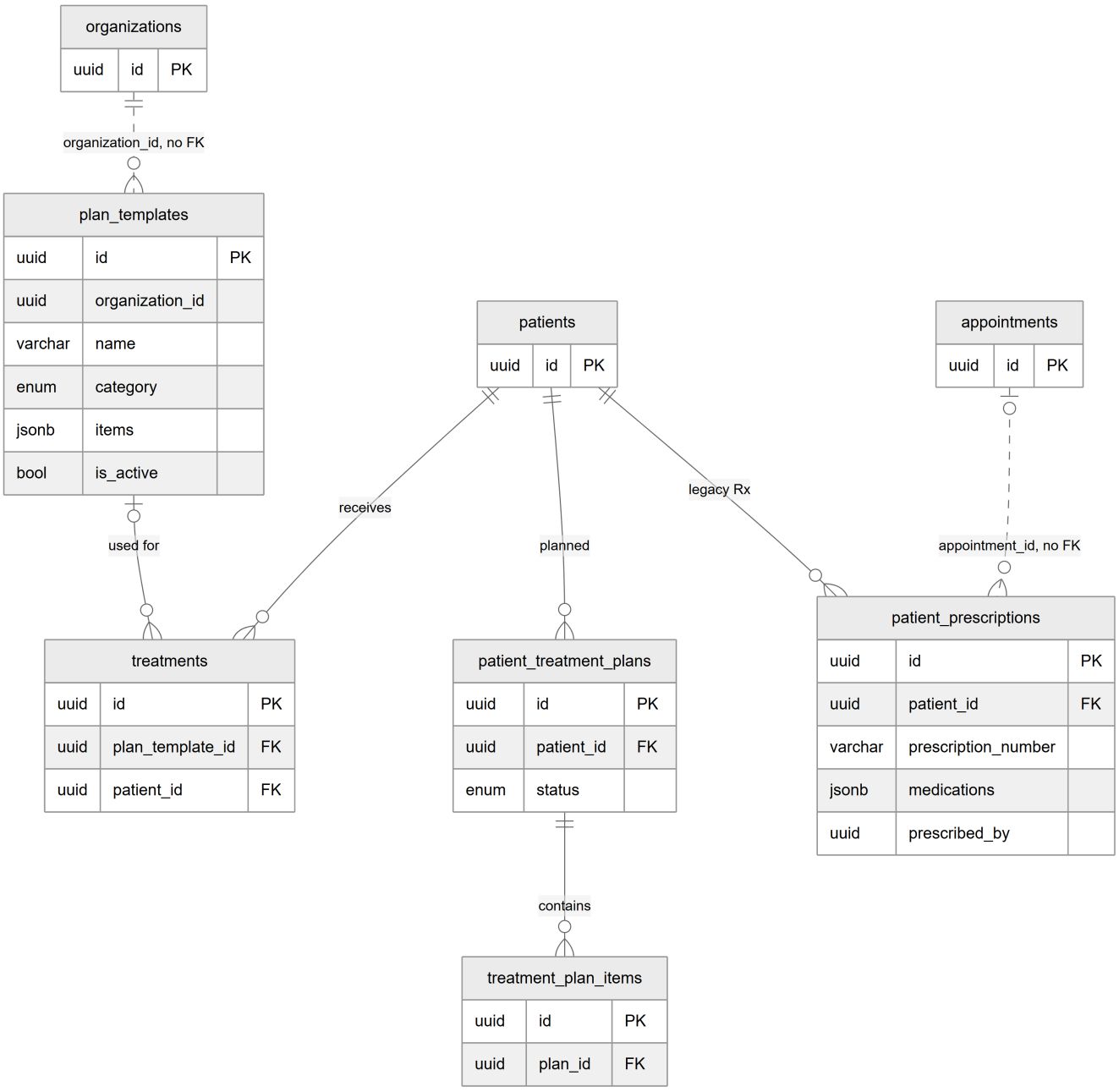


Figure 19.3 — Plan templates and the legacy prescription table.

The third diagram groups the remaining tables. `organizations` is joined to `plan_templates` (`id` , `organization_id` , `name` , `category` , `items` , `is_active`) by a dashed line labelled "organization_id, no FK". `plan_templates` has an optional solid line labelled "used for" to `treatments` . `patients` has solid lines to `treatments` ("receives"), to `patient_treatment_plans` ("planned") and to `patient_prescriptions` ("legacy Rx", showing `prescription_number` , `medications` and `prescribed_by`). `patient_treatment_plans` has a solid line labelled "contains" to `treatment_plan_items` . `appointments` is joined to `patient_prescriptions` by a dashed line labelled "appointment_id, no FK".

19.5 Relationships explained

Foreign-key relationships:

- **`prescriptions.patient_id`** → **`patients`.** Each prescription has exactly one patient; deleting the patient deletes the prescriptions (CASCADE).
- **`prescriptions.organization_id`** → **`organizations`** and **`location_id`** → **`locations`.** Mandatory, CASCADE.

- `prescriptions.appointment_id` → `appointments`, `encounter_id` → `patient_encounters`, `doctor_id` → `employees`, `prescription_template_id` → `prescription_templates`, `treatment_plan_template_id` → `treatment_plan_templates`. All optional (zero or one); deleting the referenced row keeps the prescription and sets the link to null (SET NULL).
- `prescription_medicines.prescription_id` → `prescriptions`. Each line belongs to exactly one prescription; deleting the prescription deletes its lines (CASCADE).
- `prescription_medicines.medicine_directory_id` → `medicine_directory`. Optional; deleting the directory entry keeps the line with the link set to null (SET NULL). The copied name keeps the line readable.
- `prescription_history.prescription_id` → `prescriptions`. Mandatory; deleting the prescription deletes its history (CASCADE).
- `medicine_directory.organization_id`, `treatment_plan_templates.organization_id`, `doctor_signatures.organization_id` → `organizations`. Mandatory, CASCADE.
- `treatment_plan_template_medicines.template_id` → `treatment_plan_templates`. Mandatory, CASCADE.
- `treatment_plan_template_medicines.medicine_directory_id` → `medicine_directory`. Optional, SET NULL.
- `prescription_templates.organization_id` → `organizations` (mandatory, CASCADE) and `location_id` → `locations` (optional; deleting the location deletes its layouts, CASCADE).
- `treatments.plan_template_id` → `plan_templates`. Optional, SET NULL.
- `patient_prescriptions.patient_id` → `patients`, `organization_id` → `organizations`, `location_id` → `locations`. Mandatory, CASCADE.

Logical relationships (no foreign key):

- `doctor_signatures.employee_id` → `users` (holds the user id despite the column name).
- `plan_templates.organization_id` → `organizations`.
- `patient_prescriptions.prescribed_by` → `users` and `appointment_id` → `appointments`.
- **Actor columns:** `prescription_history.changed_by`, `created_by` on `prescription_templates`, `treatment_plan_templates` and `plan_templates`.

19.6 Tenant scope

- `prescriptions`, `prescription_templates` and `patient_prescriptions` are organization + location scoped.
- `medicine_directory`, `treatment_plan_templates`, `doctor_signatures` and `plan_templates` are organization-scoped.
- `prescription_medicines`, `prescription_history` and `treatment_plan_template_medicines` have no tenant column and are isolated through their parent; queries must join it.

19.7 Ownership and ambiguities

- **Two prescription tables.** `prescriptions` is current; `patient_prescriptions` is legacy but still writable from the patient screen. Their numbers come from two different generators, and a prescription can exist in either.
- **Number uniqueness mismatch.** `prescriptions.prescription_number` is unique Platform-wide while numbering restarts in each organization, so the second organization to issue a given number would hit the unique key. The legacy table's number is unique per organization and does not have this problem. The runtime effect has not been tested and is [Unverified].
- **Names that mislead.** `treatment_plan_templates` are prescription presets, not treatment plans; `plan_templates` are treatment plan templates; `prescription_templates` are print layouts.

- **Plan template use differs from its comment.** The `plan_templates` entity comment says using a template copies its procedures into the patient's own plan. In the code, the link that is written is `treatments.plan_template_id`; no screen creates `patient_treatment_plans` rows (Chapter 17).
- **Signature owner.** `doctor_signatures.employee_id` holds a user id, is unique across organizations, and is read without an organization filter.
- **Immutability by convention.** `prescription_history` is declared immutable only in a comment.

Chapter 20 — Billing core

This chapter describes the tables that hold a clinic's money for treatment work: the service catalog, prices, charges, invoices, payments, allocations, refunds, discount types, payment accounts and per-location billing settings. It is written for the product owner who needs to know where every rupee is recorded and which database rules protect it. The design is called "billing v2" in the code; it replaced an older single-table invoice model. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference; this chapter explains meaning, keys and relationships only.

20.1 The idea in one paragraph

A **charge** is one billable line for a patient, for example one consultation or one filling. It is the atomic unit of billing. An **invoice** is an issued document that groups some of the patient's charges; there are no draft invoices. A **payment** is money received from the patient. It belongs to the patient, not to an invoice. An **allocation** says how much of a payment went to which charge. Money that has been received but not allocated is the patient's **advance**. A **refund** always gives back advance money, and refund sources record which payments funded it. Prices, discounts and the service type are copied onto each charge when it is created, so later catalog edits never change old bills.

Money convention. Every amount in this chapter is a `numeric(12,2)` column in rupees (major units), not paisa. The commission snapshot columns on `charges` are the exception: they use basis points and paisa, because they feed the commission ledger (the Compensation chapter, Chapter 22, explains it).

Key generation. All tables in this chapter use random UUID (version 4) primary keys generated by the database, except `location_billing_settings` , whose key is the location id supplied by the application.

Status: [Implemented]. All tables in this chapter are written by the billing module (`backend/src/billing/`) and shown on billing screens (billing desk, invoices, payments, billing settings).

20.2 Catalog and pricing tables

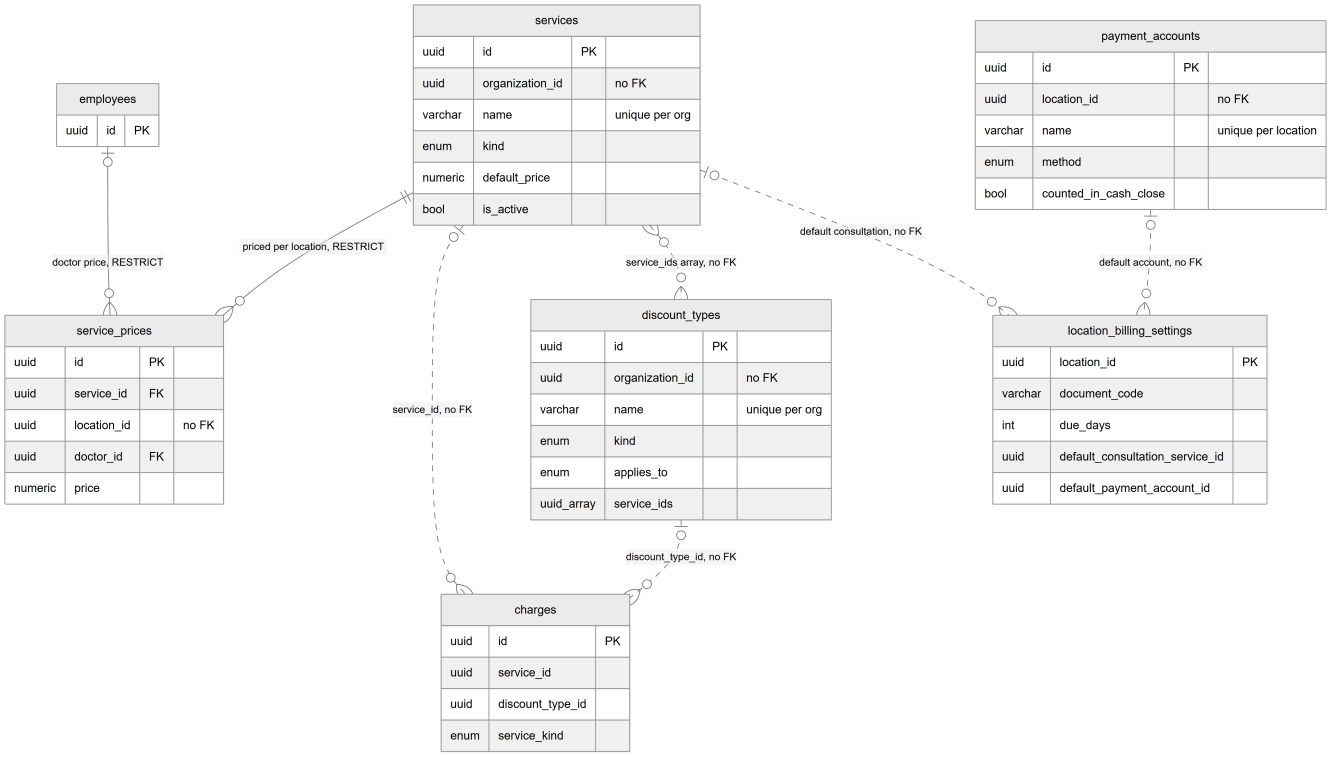


Figure 20.1 — The service catalog, location prices, discount types, payment accounts and location billing settings.

The figure shows seven boxes. `services` is at the centre. A solid line runs from `services` to `service_prices` labelled "priced per location, RESTRICT", and a second solid line runs from `employees` to `service_prices` labelled "doctor price, RESTRICT". Dotted lines, which mean the column has no foreign key, run from `services` to `charges` (service_id), from `discount_types` to `charges` (discount_type_id), from `services` to `discount_types` (the `service_ids` array, a many-to-many link), and from `services` and `payment_accounts` to `location_billing_settings` (the default consultation service and default payment account). Each box lists its primary key and a few key columns; the notes "unique per org" and "unique per location" mark business-unique names, and "no FK" marks tenant columns without a database constraint.

`services` — the service catalog

Code class name: `CatalogService` (the table name and class name differ). One row is something the clinic sells: a consultation, a procedure or "other". It belongs to an organization, not a location. Key columns: `name`, optional `code`, `kind` (CONSULTATION, PROCEDURE, OTHER), `default_price`, `is_active` (soft delete) and `sort_order`.

Constraints that matter:

- The name is unique per organization (`uq_services_org_name`).
- The code is unique per organization when present (`uq_services_org_code`, a partial unique index that ignores rows with no code).

`service_prices` — location and doctor price overrides

One row overrides a service's default price at one location, and optionally for one doctor at that location. Doctors are identified by their employee record, the same way queue sessions identify them.

Constraints that matter:

- At most one location-wide price per service and location (`uq_service_prices_loc` , partial: only rows with no doctor).
- At most one doctor-specific price per service, location and doctor (`uq_service_prices_loc_doc` , partial: only rows with a doctor).

`discount_types` — discount presets

An owner-defined discount, such as "Staff family 20%" or "Senior citizen PKR 500 off". It replaced the older coupon idea. Key columns: `kind` (PERCENTAGE or FIXED), `value` , an optional cap `max_amount` , and `applies_to` (ALL, SERVICE_KINDS or SERVICES) with the matching `service_kinds` or `service_ids` arrays. `reception_editable` , `min_value` and `max_value` let the front desk adjust the value within a range. `requires_note` forces a written reason. `valid_from` and `valid_to` limit the period. `is_active` is the soft-delete flag.

Constraints that matter: the name is unique per organization, and the code is unique per organization when present.

`payment_accounts` — ways a location receives money

One row is one way a location can take money: cash drawer, a bank account, a mobile wallet. Key columns: `name` , `method` (CASH, CARD, BANK_TRANSFER, CHEQUE, ONLINE_WALLET), optional bank details, `requires_reference` (the cashier must type a transaction reference), `counted_in_cash_close` (the account is counted at the end-of-day cash close), `is_active` and `sort_order` .

Constraint that matters: the name is unique per organization and location (`uq_payment_accounts_loc_name`).

`location_billing_settings` — one settings row per location

The primary key is the location id itself, so each location has at most one row. The row is created with defaults the first time it is needed. Key columns:

- `document_code` : the 2–6 character location code printed inside invoice, receipt and refund numbers. It lives here, not on `locations.code` , because the location code is free-form and up to 50 characters.
- `due_days` (default 7): days from issue to due date.
- `invoice_print_format` and `receipt_print_format` (A4 or THERMAL_80).
- Display switches: `show_logo` , `show_previous_balance` , `show_outstanding_at_check_in` , and a `footer_note` .
- `default_consultation_service_id` and `default_payment_account_id` .
- `cash_variance_alert_amount` : the cash-close difference that triggers an alert.

20.3 Charges, invoices, payments and refunds

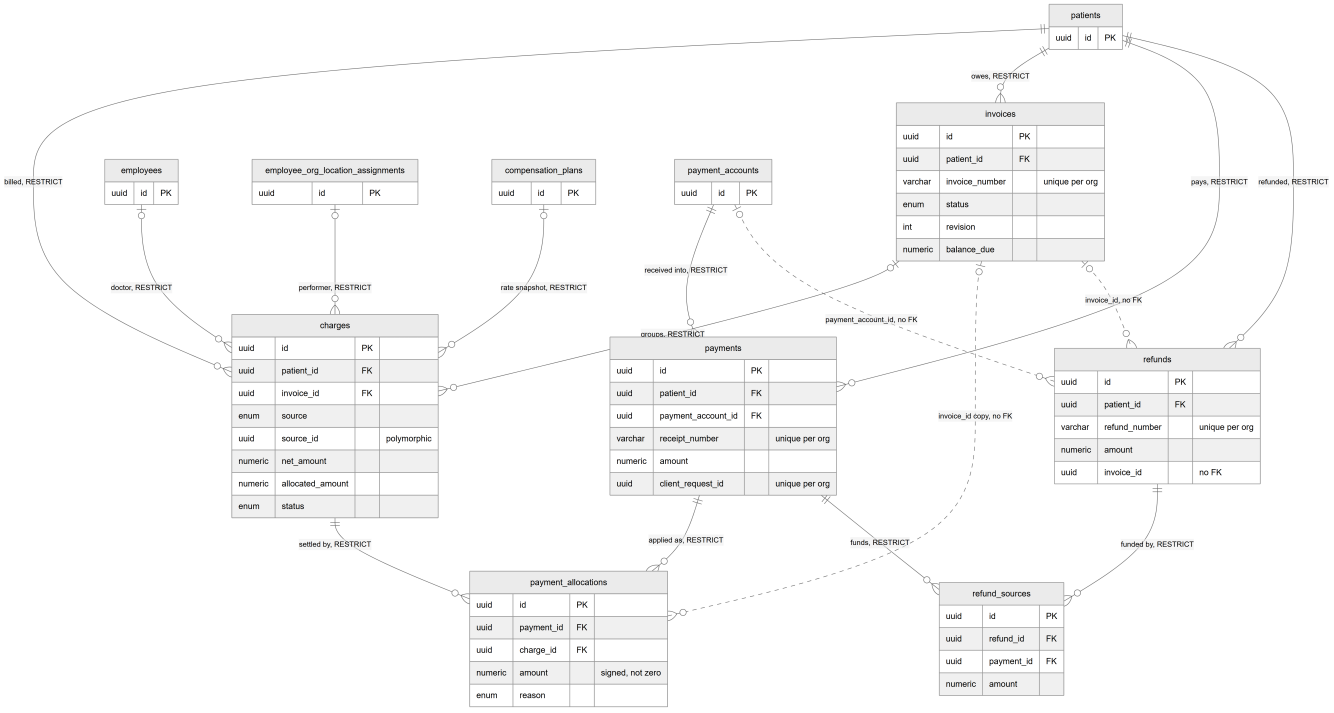


Figure 20.2 — How charges, invoices, payments, allocations, refunds and refund sources connect.

The figure shows `patients` at the top with four solid lines down to `charges` ("billed"), `invoices` ("owes"), `payments` ("pays") and `refunds` ("refunded"), all marked RESTRICT. `invoices` connects to `charges` with "groups, RESTRICT". Three small boxes on the left, `employees`, `employee_org_location_assignments` and `compensation_plans`, each connect to `charges` with "doctor", "performer" and "rate snapshot" (all RESTRICT). `payment_accounts` connects to `payments` with "received into, RESTRICT". At the bottom, `payment_allocations` hangs from both `payments` ("applied as") and `charges` ("settled by"), and `refund_sources` hangs from both `refunds` ("funded by") and `payments` ("funds"). Dotted lines show the logical links without foreign keys: `invoices` to `payment_allocations` (a copied invoice id), `invoices` to `refunds` (invoice_id) and `payment_accounts` to `refunds` (payment_account_id).

`charges` — billable lines

One row is one billable line for one patient at one location. Charges come from four sources (`source`): CONSULTATION, TREATMENT, TREATMENT_PLAN and MANUAL; `source_id` points at the originating record. Key columns:

- Pricing snapshot: `service_id`, `service_kind`, `description`, `quantity`, `unit_price`, `gross_amount`.
- Discount snapshot: `discount_type_id`, `discount_name`, `discount_kind`, `discount_value`, `discount_amount`, `discount_note`, `is_custom_discount`.
- Results: `net_amount`, `tax_amount` (this charge's share of its invoice's sales tax; zero while unbilled) and `allocated_amount` (a cached sum of allocations).
- Billing state: `invoice_id` (null means the charge is not yet invoiced), `status` (ACTIVE or VOID), void details, `needs_pricing` and `source_removed_at` (set when the doctor removed the source treatment after billing).
- Commission snapshot: `doctor_id`, `performing_assignment_id`, `commission_plan_id`, `commission_rate_bps`, `commission_fixed_paisa`.
- Links to the visit: `encounter_id` and `appointment_id`.

The balance of a charge is never stored. The code computes it as net + tax − allocated.

Constraints that matter:

- `uq_charges_source` (partial unique index): an active charge can exist only once per organization, source and source id. This stops the same consultation or treatment being charged twice. Voided charges are excluded, so a corrected charge can be created after a void.
- `chk_charges_commission_rate`: the rate is null or between 0 and 10,000 basis points (0–100%).

`invoices` — issued documents

One row is one issued invoice for one patient at one location. There is no draft state: an invoice is created and issued in one step. Key columns: `invoice_number`, `status` (ISSUED or CANCELLED), `revision` (an edit after issue increases it), `issue_date`, `due_date`, and the cached money columns `subtotal`, `discount_total`, `tax_rate` (copied from the location's settings at issue), `tax_amount`, `total_amount`, `paid_amount`, `balance_due`, `released_amount` and `refunded_amount`. Only the billing totals service writes the cached columns.

Constraint that matters: the invoice number is unique per organization (`uq_invoices_org_number`). Numbers are produced by the document numbering table described in the Billing ledgers chapter (Chapter 21).

`payments` — money received

One row is money received from one patient. It belongs to the patient, not to an invoice. Apart from the cached `allocated_amount` and `refunded_amount`, a payment row never changes. The patient's advance from a payment is amount − allocated − refunded. Key columns: `receipt_number`, `amount`, `payment_account_id`, snapshots of the account's `method` and name, `reference_number`, `payment_date`, `received_at`, `received_by` and `client_request_id`.

Constraints that matter:

- The receipt number is unique per organization.
- `client_request_id` is unique per organization (`uq_payments_org_client_request`). The web app sends a fresh id with each "collect payment" action, so a double click or a network retry cannot record the same payment twice.

`payment_allocations` — which payment paid which charge

One row says "this much of payment P was applied to charge C" (positive amount) or "this much was released back to advance" (negative amount). The table is designed as append-only: rows are never updated or deleted, and corrections are new rows with the opposite sign. The `reason` column records why: PAYMENT, ADVANCE_APPLIED, INVOICE_EDIT_RELEASE or UNAPPLY. `invoice_id` is a copy of the charge's invoice at the time of writing, kept for fast invoice queries.

Constraint that matters: `chk_payment_allocations_amount_nonzero` forbids zero-amount rows. The append-only rule itself is enforced by the application, not by a database trigger.

`refunds` — money returned

One row is money returned to a patient. A refund always comes out of advance money. If `invoice_id` is set, it is an invoice refund; if it is null, it is a plain advance refund. Key columns: `refund_number`, `amount`, `payment_account_id` with method and name snapshots, `reason` (required), `refund_date`, `refunded_at`, `refunded_by` and `client_request_id`.

Constraints that matter: the refund number and the client request id are each unique per organization.

`refund_sources` — which payments funded a refund

One row says how much of one payment's advance was used for one refund. The rows of one refund add up to the refund amount; the application enforces this sum.

20.4 Relationships explained

Foreign-key relationships (the database enforces these):

- **Service → service prices.** Each service price belongs to exactly one service; a service has zero or more prices. A service with prices cannot be deleted (ON DELETE RESTRICT).
- **Doctor → service prices.** A service price may name one doctor (an employee); a doctor can have many prices. Deleting an employee who has prices is blocked (RESTRICT).
- **Patient → charges, invoices, payments, refunds.** Each of these rows belongs to exactly one patient, and a patient can have many of each. A patient with any billing history cannot be deleted (RESTRICT on all four).
- **Invoice → charges.** A charge belongs to zero or one invoice; an invoice groups one or more charges. An invoice that still has charges cannot be deleted (RESTRICT).
- **Doctor → charges.** A charge may name the doctor (employee) who did the work; deleting that employee is blocked (RESTRICT).
- **Assignment → charges.** A charge may name the performing assignment, which is the doctor's link to the organization and location; deleting that assignment is blocked (RESTRICT).
- **Compensation plan → charges.** A charge may record the compensation plan whose rate was copied onto it; that plan cannot be deleted (RESTRICT).
- **Payment account → payments.** Each payment was received into exactly one payment account; an account with payments cannot be deleted (RESTRICT). This is why accounts are switched off with `is_active` instead.
- **Payment → allocations** and **charge → allocations.** Each allocation row belongs to exactly one payment and exactly one charge. A payment or charge with allocations cannot be deleted (RESTRICT).
- **Refund → refund sources** and **payment → refund sources.** Each refund source belongs to exactly one refund and exactly one payment; neither parent can be deleted while sources exist (RESTRICT).

Logical relationships (an id column with no foreign key; the database accepts ids that do not exist):

- `charges.service_id` → `services`; `charges.discount_type_id` → `discount_types`; `charges.encounter_id` → `patient_encounters`; `charges.appointment_id` → `appointments`. Each charge refers to at most one of each.
- `charges.source_id` is polymorphic: its meaning depends on `source` (for example a treatment or an encounter).
- `discount_types.service_ids` is an array of service ids, a many-to-many link with no constraint.
- `location_billing_settings.default_consultation_service_id` → `services` and `default_payment_account_id` → `payment_accounts`.
- `payment_allocations.invoice_id` and `payment_allocations.patient_id` are copies for fast reads.
- `refunds.invoice_id` → `invoices` and `refunds.payment_account_id` → `payment_accounts`. Note that, unlike payments, refunds have no foreign key to the payment account.
- Actor columns (`created_by`, `voided_by`, `discount_applied_by`, `issued_by`, `last_edited_by`, `cancelled_by`, `received_by`, `refunded_by`) hold a user id with no foreign key.

20.5 Tenant scope

- **Organization and location:** `charges`, `invoices`, `payments`, `payment_allocations`, `refunds`, `service_prices`, `payment_accounts`, `location_billing_settings`.
- **Organization only:** `services`, `discount_types` (both shared by every location), and `refund_sources`.
- **No foreign key on the tenant columns.** In every table of this chapter, `organization_id` and `location_id` are plain columns without a foreign key to `organizations` or `locations` (the one exception is that `location_billing_settings.location_id` is the primary key). Tenant isolation therefore depends entirely on the application always filtering by the active organization and location. The Overlaps chapter (Chapter 27) discusses this choice.

20.6 What an owner should remember

- Deleting a patient who has any billing history is impossible by design. Patients are deactivated instead.
- A charge's price and discount never change when the catalog changes, because they are copied at creation.
- The patient's balance and advance are always derived from charges, allocations and refunds; the cached totals are recomputed by one service.
- Duplicate protection exists at three levels: one active charge per source, unique document numbers per organization, and one payment or refund per client request id.

Chapter 21 — Billing ledgers and closing

This chapter describes the record-keeping tables that sit around the core billing tables: the patient statement, the double-entry general ledger, the end-of-day cash close, document numbering, the billing audit log, clinic expenses and prepaid credit packages. It is for the product owner who needs to know which tables are the books of the clinic, which are history, and which features are only partly built. The core billing tables (charges, invoices, payments, allocations, refunds) are covered in the Billing core chapter (Chapter 20). The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

21.1 Two different "ledgers"

The Platform has two tables with "ledger" in their names, and they are not the same thing:

- **`ledger_entries` is the patient statement.** It is one-sided, in rupees (`numeric(12,2)`), and has one line per invoice, adjustment, cancellation, payment or refund for a patient. It answers "what does this patient owe us?".
- **`ledger_journals` with `ledger_journal_lines` is the general ledger.** It is true double-entry bookkeeping in paisa (`bigint`), with debits and credits on named accounts. It answers "what are the clinic's balances by account?".

Both are described in this chapter. The Overlaps chapter (Chapter 27) explains why both exist.

21.2 Patient statement, cash close, numbering and audit

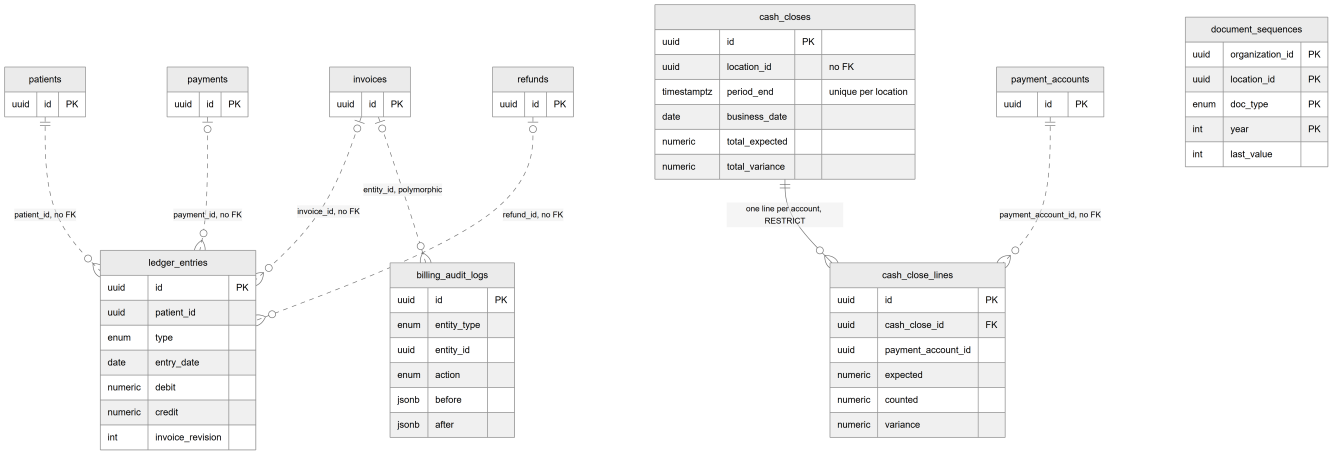


Figure 21.1 — The patient statement, the cash close with its lines, the document counter and the billing audit log.

The figure shows `ledger_entries` connected by dotted lines (no foreign key) to `patients` (patient_id), `invoices` (invoice_id), `payments` (payment_id) and `refunds` (refund_id). `cash_closes` connects with a solid line to `cash_close_lines` , labelled "one line per account, RESTRICT", and `payment_accounts` connects to `cash_close_lines` with a dotted line (payment_account_id, no FK). `billing_audit_logs` receives a dotted line from `invoices` labelled "entity_id, polymorphic", which stands for all the record types the log can point at. `document_sequences` stands alone: its four-part primary key (organization, location, document type, year) and a `last_value` counter are shown, and it has no links.

`ledger_entries` — the patient statement

Status: [Implemented]. Written by the billing ledger service; read by the patient statement endpoints and the printed statement.

One row is one line on a patient's statement at one location. `type` is INVOICE_ISSUED, INVOICE_ADJUSTED, INVOICE_CANCELLED, PAYMENT or REFUND. Exactly one of `debit` and `credit` is above zero. A positive running balance means the patient owes money; a negative one means the clinic holds the patient's advance. `invoice_revision` records which invoice revision an adjustment line belongs to. The table is designed as append-only, but no database trigger enforces it; the application never updates or deletes rows.

`cash_closes` and `cash_close_lines` — end-of-day count

Status: [Implemented]. Written from the cash close sheet on the payments page.

A cash close is an immutable count of a location's counter since the previous close. `cash_closes` holds the period (`period_start` , `period_end`), the clinic `business_date` , the totals `total_expected` , `total_counted` and `total_variance` , an optional `note` and who closed it. `cash_close_lines` holds one line per payment account in that close, with that account's payment and refund totals and counts, `expected` , `counted` and `variance` . The account's name is copied onto the line, so renaming an account later does not change old closes.

Constraint that matters: `uq_cash_closes_loc_end` makes `period_end` unique per organization and location, so two closes cannot end at the same moment.

`document_sequences` — invoice, receipt and refund numbers

Status: [Implemented].

One row is a counter for one organization, location, document type (INVOICE, RECEIPT or REFUND) and year. The primary key is those four columns, and there is no generated id: the application supplies all four values. The numbering service increments `last_value` with a single atomic statement inside the caller's transaction, so two cashiers can never get the same number. The resulting number has the form `<location code>-<INV|REC|RFD>-<year>-<five-digit counter>` , where the location code is `document_code` from `location_billing_settings` and the year is the clinic's year in the location's timezone. Because the counter is per location and per year, numbering restarts at 1 each year at each location. If a location has no document code yet, numbering refuses with the error `BILLING_LOCATION_CODE_MISSING` ("Set a document code for this location in billing settings first").

`billing_audit_logs` — billing's own audit trail

Status: [Implemented]. Read by the billing activity screen.

One row records one billing state change. `entity_type` names what changed (CHARGE, INVOICE, PAYMENT, REFUND, DISCOUNT_TYPE, SERVICE, SERVICE_PRICE, SETTINGS, PAYMENT_ACCOUNT or CASH_CLOSE), `entity_id` points at that record, and `action` names the change (for example CREATED, VOIDED, DISCOUNT_APPLIED, ISSUED, EDITED, CANCELLED, ALLOCATED, REFUNDED, CLOSED). `before` and `after` hold JSON snapshots; for an edited invoice they hold the full snapshot used for invoice revisions. `amount` records the money moved, for activity totals, and `actor_role` records the role of the person who did it. `location_id` is null only for organization-level records such as services and discount types.

The table is described as append-only, but no database trigger protects it. The Platform also has a generic `audit_logs` table (the Workflow engine and common tables chapter, Chapter 26), which billing does not use for these events.

21.3 The general ledger

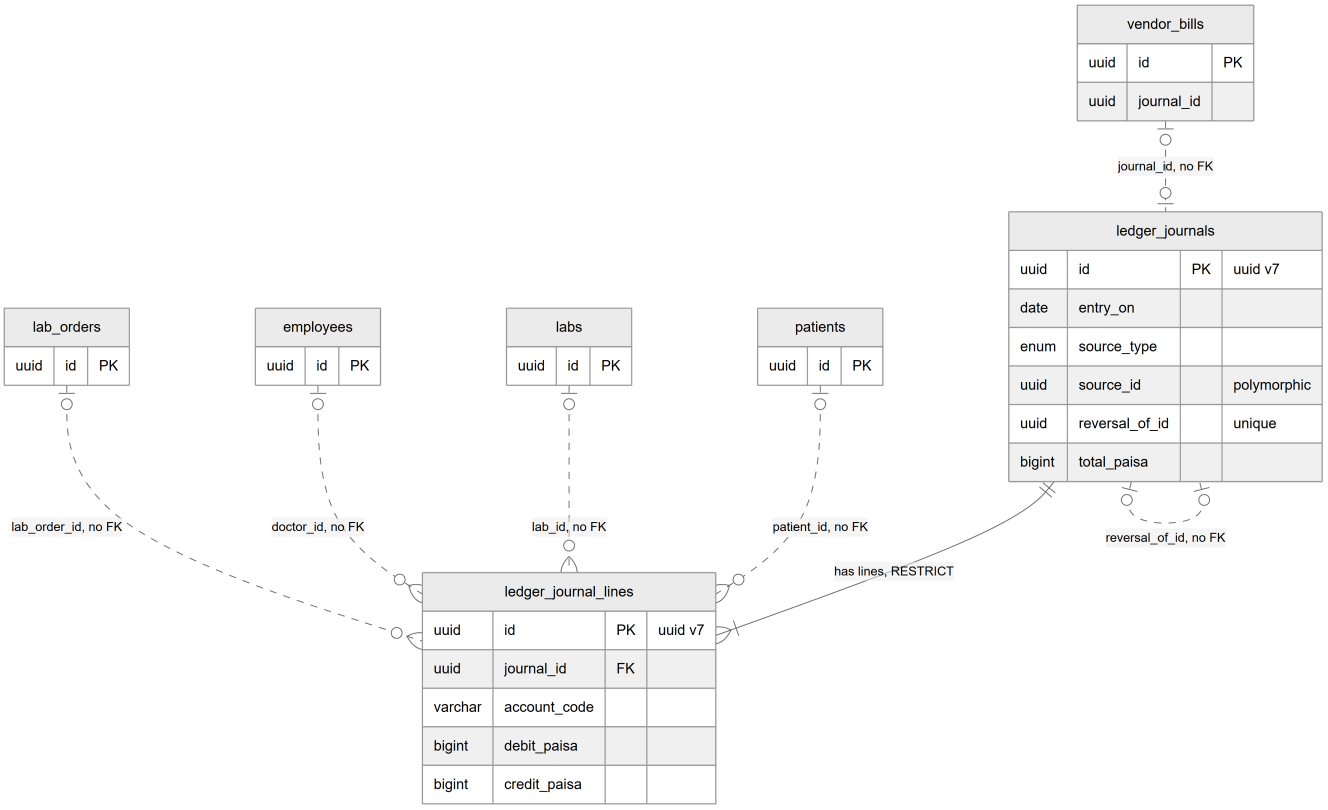


Figure 21.2 — Journals, journal lines and their logical links.

The figure shows `ledger_journals` connected to `ledger_journal_lines` by a solid line labelled "has lines, RESTRICT"; the crow's foot shows that a journal has one or more lines. A dotted loop from `ledger_journals` back to itself is labelled "reversal_of_id, no FK", meaning a journal can be the reversal of one earlier journal. `vendor_bills` points to `ledger_journals` with a dotted line (journal_id, no FK). Four dotted lines run into `ledger_journal_lines` from `patients`, `labs`, `employees` and `lab_orders`; these are optional tags on a line. Both ledger boxes note that their keys are UUID version 7.

`ledger_journals` and `ledger_journal_lines`

Status: [Partial]. The tables and the posting service exist, but in the code only lab money posts journals: lab patient invoices, lab payments, lab credit notes, lab refunds, commission on lab invoice lines, vendor bills and vendor payments. Clinic (non-lab) invoices and payments from Chapter 20 do not post journals, the `expense` source type is never posted, and no screen or report reads the general ledger.

A journal is one balanced double-entry posting. `entry_on` is the accounting date, `source_type` and `source_id` name the business record that caused it (vendor_bill, vendor_payment, expense, invoice, payment, credit_note, refund or commission), `memo` describes it, and `total_paisa` equals the sum of debits, which equals the sum of credits. A mistake is never edited: the application writes a mirror journal whose `reversal_of_id` points at the original.

A journal line is one side of a journal. `account_code` is a code from a fixed chart of accounts kept in the code, not in a table: CASH, BANK, LAB_COST, LAB_PAYABLE, LAB_ADVANCES, PATIENT_RECEIVABLE, PATIENT_ADVANCES, LAB_REVENUE, TAX_PAYABLE, COMMISSION_EXPENSE and COMMISSION_PAYABLE. Each line also carries optional tags: `patient_id`, `lab_id`, `doctor_id` and `lab_order_id`.

Constraints that matter:

- `chk_ledger_journal_lines_side`: each line is either a debit or a credit, never both and never zero.
- `uq_ledger_journals_reversal` (partial unique index): a journal can be reversed at most once.
- The rule "debits equal credits" is checked by the posting service, not by the database.
- No trigger stops updates or deletes; append-only is an application rule.

21.4 Expenses and credit packages

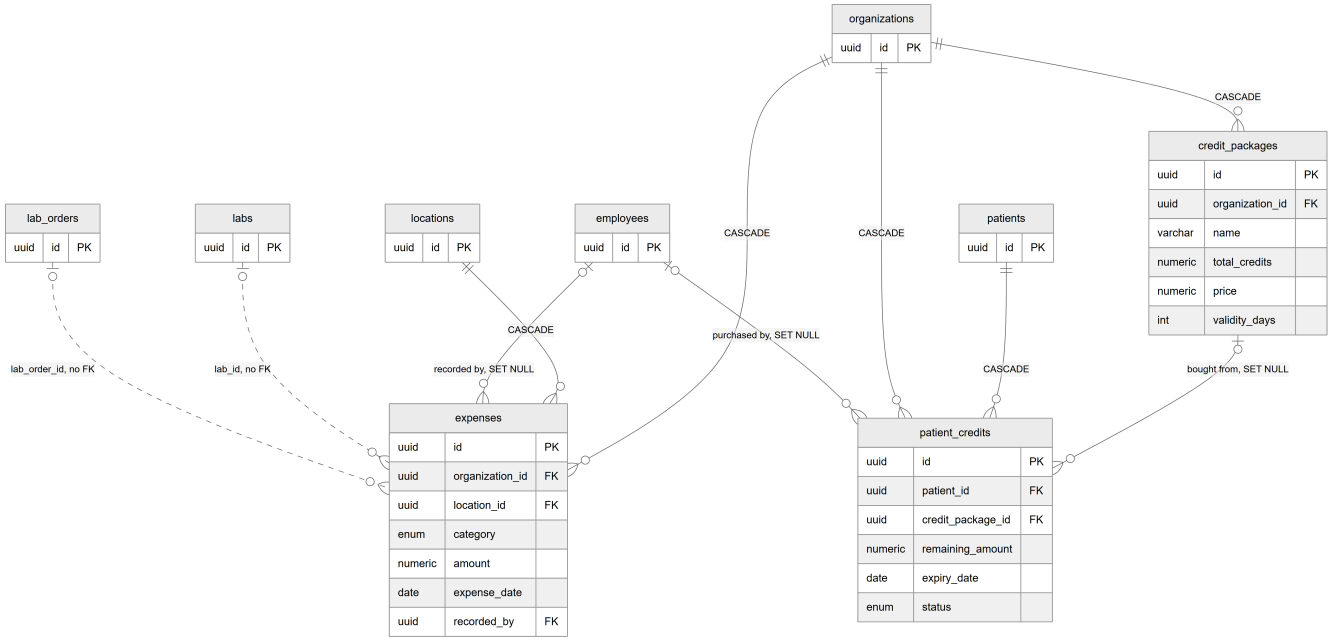


Figure 21.3 — Clinic expenses, credit packages and the credits patients buy.

The figure shows `expenses` with solid lines from `organizations` and `locations` (both CASCADE) and from `employees` ("recorded by, SET NULL"). Dotted lines from `labs` and `lab_orders` reach `expenses` (lab_id and lab_order_id, no FK). On the right, `organizations` connects to `credit_packages` (CASCADE). `credit_packages` connects to `patient_credits` with "bought from, SET NULL". `patients` and `organizations` both connect to `patient_credits` with CASCADE, and `employees` connects with "purchased by, SET NULL".

`expenses` — clinic running costs

Status: [Implemented]. Shown on the expenses page and used by finance reports.

One row is one clinic expense at one location: rent, utilities, supplies, salary, equipment, marketing, maintenance or other (`category`). Key columns: `description` , `amount` (`numeric(10,2)`), `expense_date` , `payment_method` , `reference_number` , `receipt_url` and `recorded_by` . The optional `lab_id` and `lab_order_id` mark an expense as the cost of a lab case. Timestamps in this table are stored without time zone, unlike the rest of billing.

`credit_packages` and `patient_credits` — prepaid credit

Status: [Partial]. The backend has endpoints to manage packages and to record a patient's purchase (feature `billing.credit_packages`), but no web page uses them, and credits are not a payment method at the billing desk.

`credit_packages` is an organization's offer: `name` , `total_credits` , `price` and `validity_days` , with `is_active` as a soft delete. `patient_credits` is one purchase by one patient: `total_amount` , `used_amount` , `remaining_amount` , `purchase_date` , `expiry_date` and `status` (ACTIVE, EXPIRED or EXHAUSTED). Money

columns are `numeric(10,2)` and timestamps have no time zone.

21.5 Relationships explained

Foreign-key relationships:

- **Cash close → cash close lines.** Each line belongs to exactly one cash close; a close has one line per counted account. A close with lines cannot be deleted (RESTRICT).
- **Journal → journal lines.** Each line belongs to exactly one journal; a journal has two or more lines. A journal with lines cannot be deleted (RESTRICT).
- **Organization → expenses** and **location → expenses.** Each expense belongs to one organization and one location. Deleting the organization or the location deletes its expenses (CASCADE).
- **Employee → expenses.** An expense may name the employee who recorded it; deleting that employee keeps the expense and clears the link (SET NULL).
- **Organization → credit packages.** Deleting the organization deletes its packages (CASCADE).
- **Credit package → patient credits.** A patient credit may come from one package; deleting the package keeps the credit and clears the link (SET NULL).
- **Patient → patient credits** and **organization → patient credits.** Deleting the patient or the organization deletes the credits (CASCADE).
- **Employee → patient credits.** The employee who sold the credit; deleting that employee clears the link (SET NULL).

Logical relationships (no foreign key):

- `ledger_entries.patient_id`, `invoice_id`, `payment_id` and `refund_id` point at the patient and at the billing record behind each line.
- `cash_close_lines.payment_account_id` → `payment_accounts`.
- `billing_audit_logs.entity_id` is polymorphic, discriminated by `entity_type`.
- `ledger_journals.source_id` is polymorphic, discriminated by `source_type`; `reversal_of_id` points at another journal.
- `ledger_journal_lines.patient_id`, `lab_id`, `doctor_id` (an employee) and `lab_order_id` are optional tags.
- `vendor_bills.journal_id` → `ledger_journals` (the Labs chapter, Chapter 25, covers vendor bills).
- `expenses.lab_id` → `labs` and `expenses.lab_order_id` → `lab_orders`.
- Actor columns (`created_by`, `closed_by`, `actor_id`) hold a user id with no foreign key.

21.6 Tenant scope

- **Organization and location:** `ledger_entries`, `cash_closes`, `document_sequences`, `billing_audit_logs`, `ledger_journals`, `ledger_journal_lines` (location is optional on the journal tables) and `expenses`.
- **Organization only:** `credit_packages`, `patient_credits`.
- **No tenant column:** `cash_close_lines`, which is scoped only through its cash close. Any query on it must join `cash_closes` to enforce isolation.
- Only `expenses`, `credit_packages` and `patient_credits` have foreign keys on their tenant columns; every other table here stores the organization and location ids without a constraint.

Chapter 22 — Compensation

This chapter describes the four tables that decide how a doctor or other employee is paid: compensation plans, commission rules, the commission ledger and commission settlements. It is for the product owner who needs to understand how commission is recorded, why its amounts can never be edited, and how it reaches payroll. The tables live in the billing module (`backend/src/billing/commissions/`) and connect billing charges, lab invoice lines and payroll records. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

Status: [Implemented]. Plans and rules are edited from the employee screens; commission entries are written automatically when money is collected; settlements are created and paid through payroll; the commission pages show earnings.

22.1 The idea in one paragraph

Pay is set per **assignment**, which is an employee's link to one organization and location, because the same doctor can be paid differently at two branches. A **compensation plan** says how the assignment is paid during a date range: salary, commission, or both. **Commission rules** override the plan's default rate for one service or one service category. Commission is earned only on money actually collected. Every time collection on a charge changes, the Platform writes a **commission entry** with only the difference, positive or negative. When payroll is generated, the unsettled entries of the assignment are grouped into a **settlement**, which is paid through one **payroll record**.

Money convention. All amounts here are `bigint` paisa (integer minor units). Rates are basis points: 4000 means 40.00%.

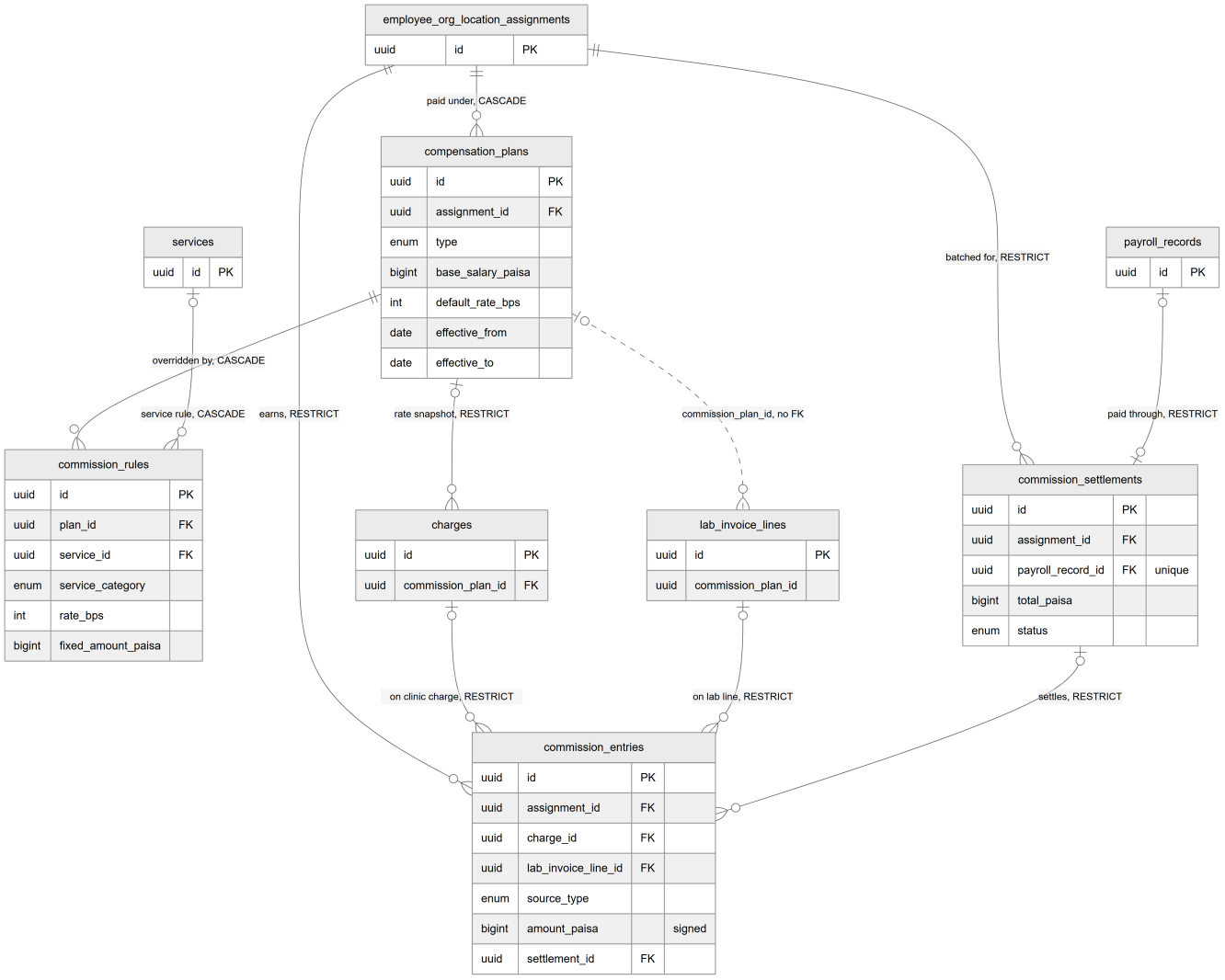


Figure 22.1 — Compensation plans, commission rules, the commission ledger and settlements, with their links to billing and payroll.

The figure shows `employee_org_location_assignments` with three solid lines: to `compensation_plans` ("paid under, CASCADE"), to `commission_entries` ("earns, RESTRICT") and to `commission_settlements` ("batched for, RESTRICT"). `compensation_plans` connects to `commission_rules` ("overridden by, CASCADE") and to `charges` ("rate snapshot, RESTRICT"), and has a dotted line to `lab_invoice_lines` (commission_plan_id, no FK). `services` connects to `commission_rules` ("service rule, CASCADE"). `commission_entries` receives solid lines from `charges` ("on clinic charge"), `lab_invoice_lines` ("on lab line") and `commission_settlements` ("settles"), all RESTRICT. `payroll_records` connects one-to-one with `commission_settlements` ("paid through, RESTRICT"). Each main box shows its key columns, and `amount_paisa` on the entry is marked "signed".

22.2 Tables

`compensation_plans` — how an assignment is paid

One row is one version of an assignment's pay, valid from `effective_from` to `effective_to` (both inclusive; an empty `effective_to` means "current"). `type` is SALARY, COMMISSION or SALARY_PLUS_COMMISSION. `base_salary_paisa` holds the salary, `default_rate_bps` the default commission rate, and `commission_basis` (NET by default, or GROSS) which charge amount the commission rate is applied to. A change of pay closes the current version and adds a new one, so old charges keep the rate that was copied onto them.

Constraints that matter:

- `excl_compensation_plans_overlap` (an EXCLUDE constraint using the `btree_gist` extension): two versions of the same assignment can never have overlapping date ranges.
- `chk_compensation_plans_fields`: a SALARY plan must have a salary and no rate; a COMMISSION plan must have a rate and no salary; SALARY_PLUS_COMMISSION must have both.
- `chk_compensation_plans_dates`, `chk_compensation_plans_salary`, `chk_compensation_plans_rate`: the end date is not before the start, the salary is not negative, and the rate is between 0 and 10,000 basis points.

`commission_rules` — per-service and per-category overrides

One row overrides the plan's default for either one catalog service (`service_id`) or one service category (`service_category`: CONSULTATION, PROCEDURE or OTHER), never both. It holds either a rate (`rate_bps`) or a fixed amount per fully paid charge (`fixed_amount_paisa`), never both. When the Platform looks up the rate for a charge, it tries the service rule first, then the category rule, then the plan default.

Constraints that matter:

- `chk_commission_rules_target` and `chk_commission_rules_amount`: exactly one target and exactly one kind of amount.
- `chk_commission_rules_rate` and `chk_commission_rules_fixed`: valid ranges.
- `uq_commission_rules_service` and `uq_commission_rules_category` (partial unique indexes): at most one rule per plan and service, and at most one per plan and category.

`commission_entries` — the commission ledger

One row is one change in what an assignment has earned on one source line. The source line is either a clinic charge (`charge_id`) or a lab invoice line (`lab_invoice_line_id`), never both. `source_type` says what caused the entry: ALLOCATION (new money), UNAPPLY, CHARGE_ADJUST, VOID or MANUAL; `source_id` is the payment or operation behind it. `base_paisa` is how much had been collected on the line (capped at its net amount) when the entry was written, `rate_bps` the rate used, and `amount_paisa` the signed difference. For each line, the sum of its entries always equals the commission due on what has been collected so far. `settlement_id` is empty until the entry is settled.

Constraints that matter:

- `chk_commission_entries_target`: exactly one of `charge_id` and `lab_invoice_line_id` is set.
- `uq_commission_entries_source` (source type, source id, charge) and `uq_commission_entries_lab_source` (partial, for lab lines): the same operation cannot write commission twice for the same line.
- `idx_commission_entries_unsettled`: a partial index over unsettled entries, used when payroll collects them.

The append-only trigger. This table is protected in the database by the trigger `trg_commission_entries_append_only`, installed by the hand-written migration `1792403456699-doctor-compensation`. The trigger:

- refuses every DELETE;
- refuses any UPDATE that changes a column other than `settlement_id`;
- allows `settlement_id` to be set once, from empty to a settlement;
- allows `settlement_id` to be cleared back to empty only while that settlement is still PENDING. Once the settlement is PAID, the entry can no longer move.

It is one of only three tables in the Platform whose append-only rule is enforced by the database itself (the others are `audit_logs` and `status_history`).

`commission_settlements` — a payout batch

One row groups unsettled commission entries of one assignment for a period (`period_start` , `period_end`) and links them to one payroll record. `total_paisa` is the sum of the entries; it is signed, because a period with only reversals can be negative. `status` is PENDING, then PAID, with `paid_at` and `paid_by` .

How it is used in the code:

- When payroll is generated, the unsettled entries created before the cut-off are locked (`SELECT … FOR UPDATE`), so one entry can never land in two settlements. A PENDING settlement is created and the entries are pointed at it.
- When the payroll record is marked paid, the settlement becomes PAID in the same transaction.
- If the payroll record is deleted before payment, the entries are released back to the unsettled pool and the settlement row is deleted. Once PAID, release is refused.

Constraint that matters: `payroll_record_id` is unique, so one payroll record has at most one settlement.

22.3 Relationships explained

- **Assignment → compensation plans.** Each plan belongs to exactly one assignment; an assignment has zero or more plan versions. Deleting an assignment deletes its plans (ON DELETE CASCADE). In practice this cascade is blocked as soon as any charge has copied one of those plans, because charges point at plans with RESTRICT.
- **Compensation plan → commission rules.** Each rule belongs to exactly one plan; deleting the plan deletes its rules (CASCADE).
- **Service → commission rules.** A rule may target one catalog service; deleting the service deletes its rules (CASCADE). Services are normally switched off with `is_active` rather than deleted.
- **Compensation plan → charges.** A charge may record the plan whose rate it copied; that plan cannot be deleted (RESTRICT). The rate and fixed amount themselves are also copied onto the charge.
- **Assignment → commission entries.** Each entry belongs to exactly one assignment; an assignment with entries cannot be deleted (RESTRICT).
- **Charge → commission entries.** An entry may belong to one charge; a charge has zero or more entries. A charge with entries cannot be deleted (RESTRICT).
- **Lab invoice line → commission entries.** An entry may belong to one lab invoice line instead; that line cannot be deleted (RESTRICT). Lab invoice lines are covered in the Labs chapter (Chapter 25).
- **Settlement → commission entries.** An entry belongs to zero or one settlement; a settlement groups many entries. A settlement that still has entries cannot be deleted (RESTRICT); the release step clears the entries first.
- **Assignment → commission settlements.** Each settlement belongs to exactly one assignment (RESTRICT).
- **Payroll record → commission settlement.** One-to-one: a settlement may be paid through one payroll record, and a payroll record has at most one settlement. A payroll record with a settlement cannot be deleted at database level (RESTRICT); the payroll service releases the settlement first, and only while it is PENDING.

Logical relationships (no foreign key):

- `lab_invoice_lines.commission_plan_id` → `compensation_plans` and `lab_invoice_lines.performing_assignment_id` → `employee_org_location_assignments` : the same snapshot as on charges, but without constraints.
- `commission_entries.source_id` is polymorphic, discriminated by `source_type` .
- Actor columns `compensation_plans.created_by` and `commission_settlements.paid_by` hold a user id.

22.4 Tenant scope

- **Organization and location:** `commission_entries`, `commission_settlements`.
- **Organization only:** `compensation_plans` (the location is implied by the assignment).
- **No tenant column:** `commission_rules`, scoped only through its plan. Queries must join `compensation_plans`.
- None of the tenant columns here has a foreign key; isolation is enforced by the application.

22.5 What an owner should remember

- Commission follows cash, not invoices: nothing is earned until money is allocated to a charge or lab line.
- Commission amounts cannot be edited or deleted by any ordinary SQL statement, because the trigger rejects it. Corrections are new signed entries.
- Tax is not part of the commission base. The base is the collected amount capped at the line's net amount.
- A rate change never rewrites history: old charges keep the rate copied onto them.

Chapter 23 — Inventory

This chapter describes the eleven tables of the inventory module: stock items and their categories, stock movements, suppliers, purchase orders, consumption templates and the record of materials used on a patient. It is for the product owner who wants to know how stock is tracked and where the module is incomplete. The module lives in `backend/src/inventory/` and is reached in the web app at `/inventory`. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

23.1 Two things to know first

camelCase column names. Unlike every other module, the inventory tables use camelCase column names in the database, for example `organizationId`, `locationId`, `currentStock` and `treatmentPlanId`. In SQL these names must be written in double quotes. The legacy `inventory` table is the only exception: it uses snake_case.

A note on the catalog. In the entity catalog, the "Notes" column of the inventory tables is shifted one row down: the comment shown next to a column belongs to the column listed above it. For example, the comment "NEVER update directly — always through StockMovement" belongs to `currentStock`, not to `unit`. This chapter was checked against the entity files in `backend/src/inventory/entities/`.

Money convention. Prices and totals are `numeric(10,2)` in rupees. Quantities are `numeric(10,2)`, so fractional quantities such as 1.5 ml are allowed. Timestamps are stored without time zone.

Status overview:

- Stock items, categories, suppliers, purchase orders, stock movements and consumption templates: [Partial]. The backend and the `/inventory` page (tabs for items, suppliers, purchase orders and templates) exist and are permission-gated, but `/inventory` has no sidebar entry, so users reach it only by URL.
- Recording materials used on a patient (`patient_consumptions`): [Partial]. The backend endpoints (`POST /patient-consumptions` and its read endpoints) exist, and the web app has a `ConsumptionRecorder` component, but no page renders that component.
- The legacy `inventory` table: [Not implemented]. It is not registered by any module.

23.2 Stock and purchasing

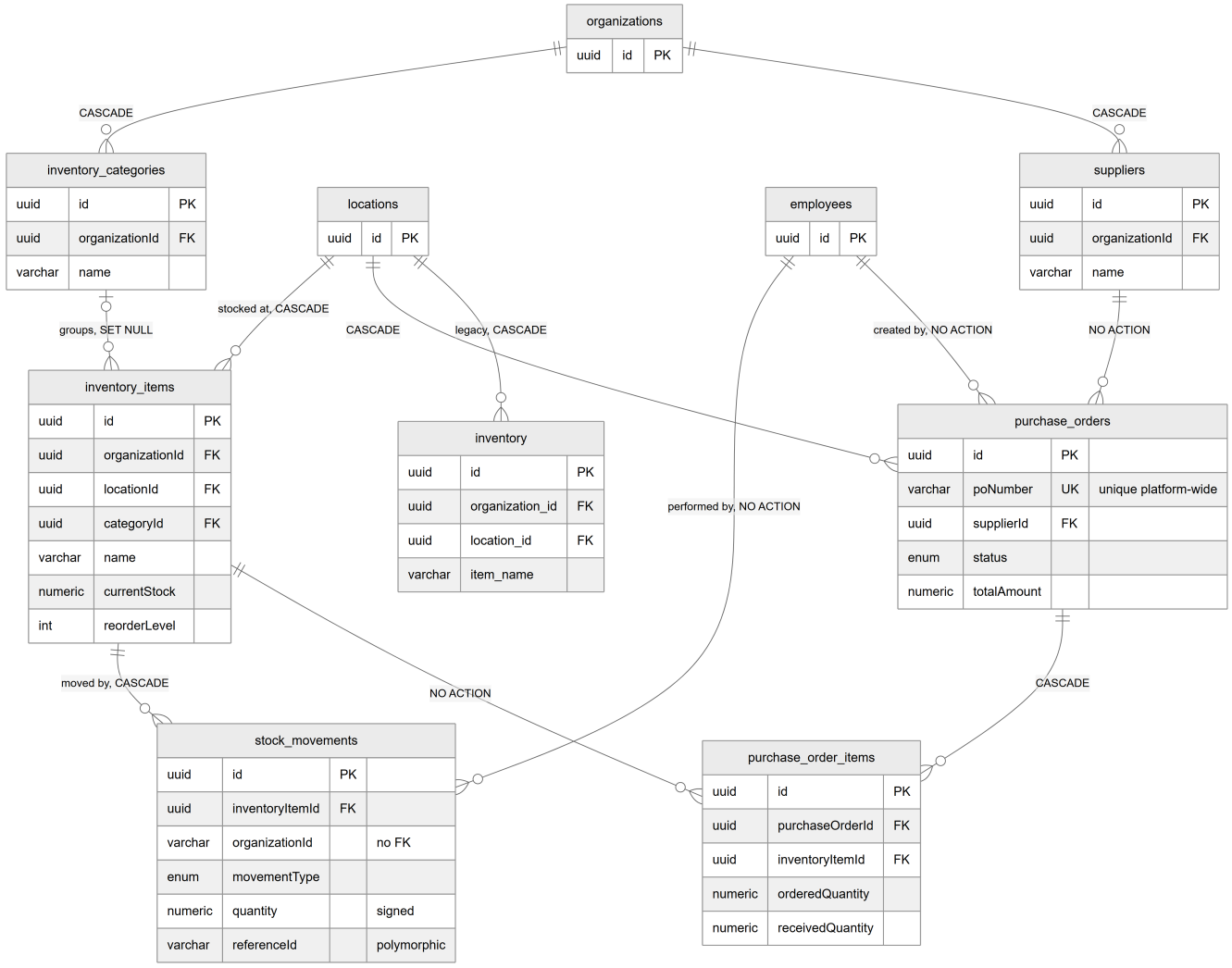


Figure 23.1 — Items, categories, stock movements, suppliers, purchase orders and the legacy inventory table.

The figure shows `organizations` connected to `inventory_categories` and `suppliers` (both CASCADE). `inventory_categories` connects to `inventory_items` with "groups, SET NULL", and `locations` connects to `inventory_items` with "stocked at, CASCADE". `inventory_items` connects to `stock_movements` ("moved by, CASCADE") and to `purchase_order_items` (NO ACTION). `employees` connects to `stock_movements` ("performed by") and to `purchase_orders` ("created by"), both NO ACTION. `suppliers` connects to `purchase_orders` (NO ACTION), `locations` to `purchase_orders` (CASCADE), and `purchase_orders` to `purchase_order_items` (CASCADE). A separate box, `inventory`, hangs from `locations` with "legacy, CASCADE". The `poNumber` column on `purchase_orders` carries the note "unique platform-wide".

`inventory_categories` — item groups

An organization's grouping of items, such as disposables, dental materials, medicines, instruments or office supplies. Key columns: `name`, `description`, `sortOrder`, and `isActive` as a soft delete.

`inventory_items` — one stocked item at one location

One row is one item kept at one location, with its current stock. Key columns:

- Identity: `name`, optional `sku` (an internal stock code) and `barcode` (kept for a future scanning feature).

- Stock: `currentStock` and `unit` (pieces, boxes, pairs, rolls, bottles, packets, cartridges, tubes, ml, gm, kg or liters). The code comment says `currentStock` must never be updated directly, only through a stock movement.
- Price: `unitPrice` is the last purchase cost per unit and is updated when purchase-order items are received; `sellingPrice` is used when the item is billed to a patient.
- Reorder: `reorderLevel` (alert when stock falls to or below it, default 10), `reorderQuantity` (suggested order quantity, default 50) and optional `maxStock`.
- Expiry: `expiryDate` (the nearest expiry among current batches) and `expiryAlertDays` (alert this many days before, default 30).
- Flags: `isBillable` (the item may be added to a patient invoice) and `isActive`.

There is no unique constraint on the SKU. The code comment says SKU uniqueness per organization is enforced in the service, not in the database.

`stock_movements` — the stock history

One row is one change to one item's stock. `movementType` is PURCHASE_IN, MANUAL_ADJUSTMENT, CONSUMPTION, WASTAGE, TRANSFER_IN, TRANSFER_OUT, RETURN_TO_SUPPLIER or INITIAL_STOCK. `quantity` is signed: positive for stock coming in, negative for stock going out. `previousStock` and `newStock` snapshot the item's stock before and after, so `newStock = previousStock + quantity`. `referenceType` names what triggered the movement ('purchase_order', 'appointment', 'treatment_plan', 'manual' or 'transfer') and `referenceId` holds that record's id. `batchNumber` and `expiryDate` describe the batch.

Two oddities: `organizationId` and `locationId` here are `varchar`, not `uuid`, and have no foreign key; `referenceId` is also `varchar`.

`suppliers` — vendors of stock

An organization's supplier: name, contact person, phone, email, address, city, website, notes, and `isActive`.

`purchase_orders` and `purchase_order_items` — buying stock

A purchase order is one order from one supplier for one location. `status` is DRAFT, ORDERED, PARTIALLY_RECEIVED, RECEIVED or CANCELLED. Money columns are `subtotal`, `taxRate`, `taxAmount` and `totalAmount`; dates are `orderDate`, `expectedDeliveryDate` and `receivedDate`. Each purchase-order item is one inventory item on the order, with `orderedQuantity`, `receivedQuantity`, `unitPrice`, `totalPrice`, and optional `batchNumber` and `expiryDate`.

Constraint that matters, and a risk. `poNumber` has a unique constraint on its own, across the whole Platform. The service generates numbers per organization in the form `PO-<year>-<five digits>`, starting from the organization's last order. Two organizations will therefore both try to create `PO-<year>-00001`, and the second insert will fail on the unique constraint. This is recorded as a defect for the owner; the module chapter for inventory (Chapter 39) covers its user impact.

`inventory` — legacy single table

Status: [Not implemented]. A simple item table (`item_name`, `quantity`, `unit_price`, `expiry_date`) for an organization and location. No module registers it, so the running application never creates or uses it in development. It exists only because the command-line schema tools load every entity file, so a production schema built with `schema:sync:prod` may contain it while a development database does not. It looks superseded by `inventory_items` and `stock_movements`.

23.3 Consumption

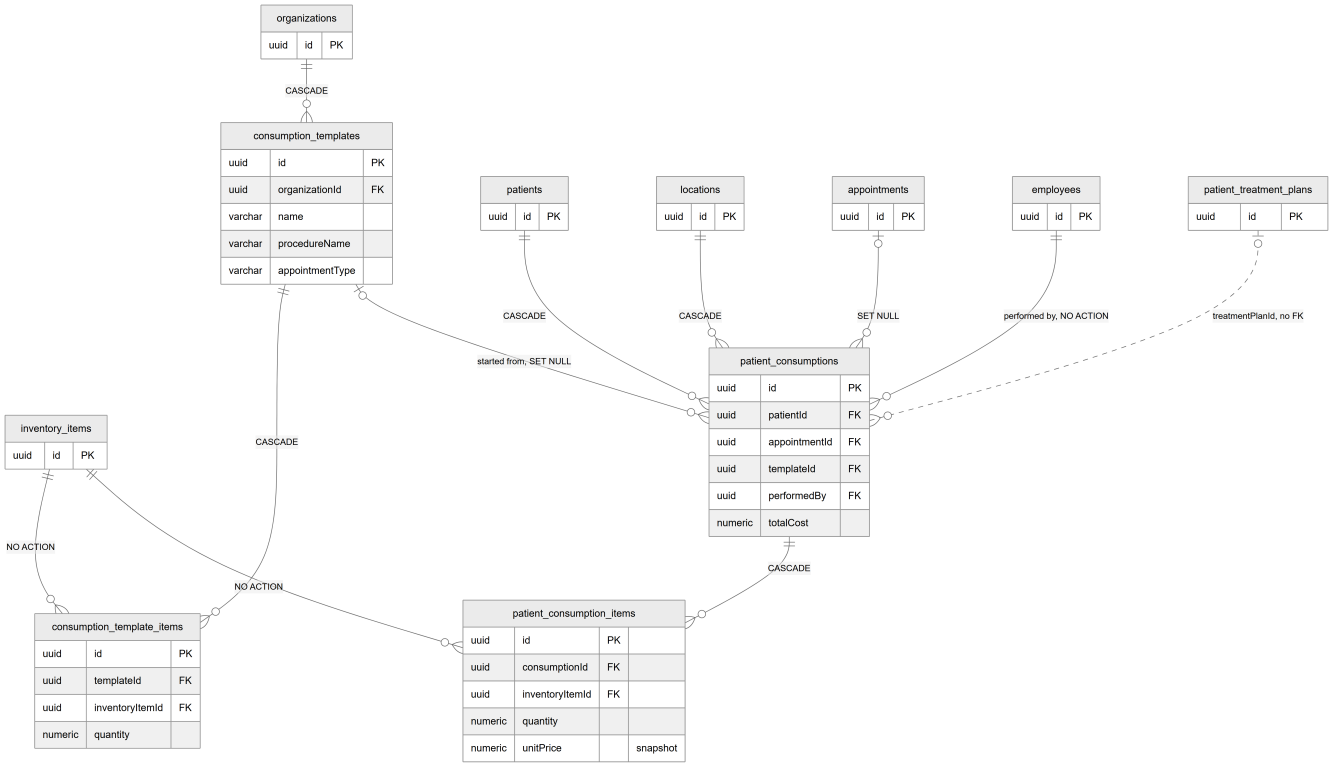


Figure 23.2 — Consumption templates and the record of materials used on a patient.

The figure shows `organizations` connected to `consumption_templates` (CASCADE). `consumption_templates` connects to `consumption_template_items` (CASCADE) and to `patient_consumptions` ("started from, SET NULL"). `inventory_items` connects to both `consumption_template_items` and `patient_consumption_items` with NO ACTION. `patients` and `locations` connect to `patient_consumptions` with CASCADE, `appointments` with SET NULL, and `employees` with "performed by, NO ACTION". `patient_consumptions` connects to `patient_consumption_items` (CASCADE). A dotted line from `patient_treatment_plans` to `patient_consumptions` is labelled "treatmentPlanId, no FK". The `unitPrice` column on the consumption item is marked "snapshot".

`consumption_templates` and `consumption_template_items` — standard kits

A consumption template is an organization's standard kit for a procedure, for example a root canal kit. Key columns: `name`, `procedureName`, optional `appointmentType` (used to suggest the template automatically when that type of visit starts) and `isActive`. Each template item names one inventory item and a default `quantity`, with optional `notes` as hints for the doctor and a `sortOrder`.

`patient_consumptions` and `patient_consumption_items` — what was used on a patient

A patient consumption records the materials used on one patient at one location, optionally for one appointment, and optionally based on a template. `performedBy` is the doctor (an employee). `totalCost` is the sum of the item totals, calculated by the service. Each consumption item records one inventory item, the `quantity` actually used, `unitPrice` copied from the item at the time of use, `totalPrice` (quantity × unit price, calculated by the service) and `wasFromTemplate` (true if the item was suggested by the template, false if added by hand).

`treatmentPlanId` has no foreign key. The code comment says this was done "to avoid circular" imports. The intended target is probably `patient_treatment_plans`, but the code does not say so.

23.4 Relationships explained

Foreign-key relationships:

- **Organization → categories, items, suppliers, purchase orders, consumption templates, patient consumptions.** Each belongs to one organization; deleting the organization deletes them (CASCADE).
- **Location → items, purchase orders, patient consumptions.** Each belongs to one location; deleting the location deletes them (CASCADE).
- **Category → items.** An item belongs to zero or one category; deleting the category keeps the items and clears the link (SET NULL).
- **Item → stock movements.** Each movement belongs to exactly one item; deleting the item deletes its whole stock history (CASCADE).
- **Employee → stock movements, purchase orders, patient consumptions.** Each names the employee who did it. Deleting that employee is blocked while such rows exist (NO ACTION, which behaves like RESTRICT at the end of the statement).
- **Supplier → purchase orders.** Each order belongs to exactly one supplier; a supplier with orders cannot be deleted (NO ACTION). Suppliers are deactivated with `isActive`.
- **Purchase order → purchase-order items.** Deleting an order deletes its items (CASCADE).
- **Item → purchase-order items, template items, consumption items.** Each refers to exactly one inventory item; an item used on any of these cannot be deleted (NO ACTION). Note the asymmetry: deleting an item would cascade its stock movements, but the delete is blocked first if the item appears on any order, template or consumption.
- **Template → template items.** Deleting a template deletes its items (CASCADE).
- **Template → patient consumptions.** A consumption may have started from one template; deleting the template keeps the consumption and clears the link (SET NULL).
- **Patient → patient consumptions.** Deleting the patient deletes the consumption records (CASCADE).
- **Appointment → patient consumptions.** Deleting the appointment keeps the consumption and clears the link (SET NULL).
- **Patient consumption → consumption items.** Deleting a consumption deletes its items (CASCADE).
- **Legacy `inventory`.** Belongs to one location and optionally one organization; both CASCADE.

Logical relationships (no foreign key):

- `patient_consumptions.treatmentPlanId` → probably `patient_treatment_plans` (not verified in the code).
- `stock_movements.referenceId` is polymorphic, discriminated by `referenceType`.
- `stock_movements.organizationId` and `locationId` → `organizations` and `locations`.

23.5 Tenant scope

- **Organization and location:** `inventory_items`, `stock_movements`, `purchase_orders`, `patient_consumptions`, and the legacy `inventory`.
- **Organization only:** `inventory_categories`, `suppliers`, `consumption_templates`.
- **No tenant column:** `purchase_order_items`, `consumption_template_items` and `patient_consumption_items`, scoped only through their parent order, template or consumption. Queries must join the parent to enforce isolation.
- Unlike the billing tables, most inventory tenant columns do have foreign keys. The exception is `stock_movements`, whose tenant columns are unconstrained `varchar`.

Chapter 24 — Notifications

This chapter describes the seven tables of the notification system: the platform catalog of events, each clinic's on/off rules and wording, the outbox of messages to send, the in-app inbox (the bell), and each patient's messaging preferences. It is for the product owner who needs to know what is stored when "something notifies someone" and which channels really work today. The module lives in `backend/src/notifications/`; the sending mechanics are covered in the Notification mechanics chapter (Chapter 59). The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

24.1 The idea in one paragraph

The Platform defines a fixed list of **notification events** in code, such as an order being submitted or an item being ready. Each event can reach one or more **audiences**: patient, doctor, coordinator or lab. A clinic decides per event, audience and channel whether to send (**notification rules**), can switch a whole audience off or set quiet hours (**audience settings**), and can replace the default wording (**templates**). When something happens, the Platform writes one **outbox** row per recipient and channel in the same database transaction as the change itself, including rows it decided to skip, so the log can show why a message never went. A background relay then sends the queued rows. An in-app send also creates a row in the recipient's **bell**.

Channels in practice. The tables allow four channels: `in_app`, `whatsapp`, `sms` and `email`. In the code, only `in_app` is switched on for the doctor, coordinator and lab audiences; WhatsApp, SMS and email are commented out because no real provider is connected. The patient audience has no channel at all, so patients currently receive nothing.

Status:

- In-app notifications to staff and lab users (outbox, bell, settings, templates): [Implemented].
- WhatsApp, SMS and email delivery: [Not implemented]. The channel values exist in the tables and checks, but no provider sends them.
- Patient notifications: [Not implemented]. Patient preferences can be stored, but no channel reaches patients.

Keys. Every table here except `notification_events` and `patient_comm_prefs` uses a UUID version 7 primary key generated by the database function `uuid_generate_v7()`. Version 7 ids start with a timestamp, so they sort by creation time (the Labs chapter, Chapter 25, explains them in more detail).

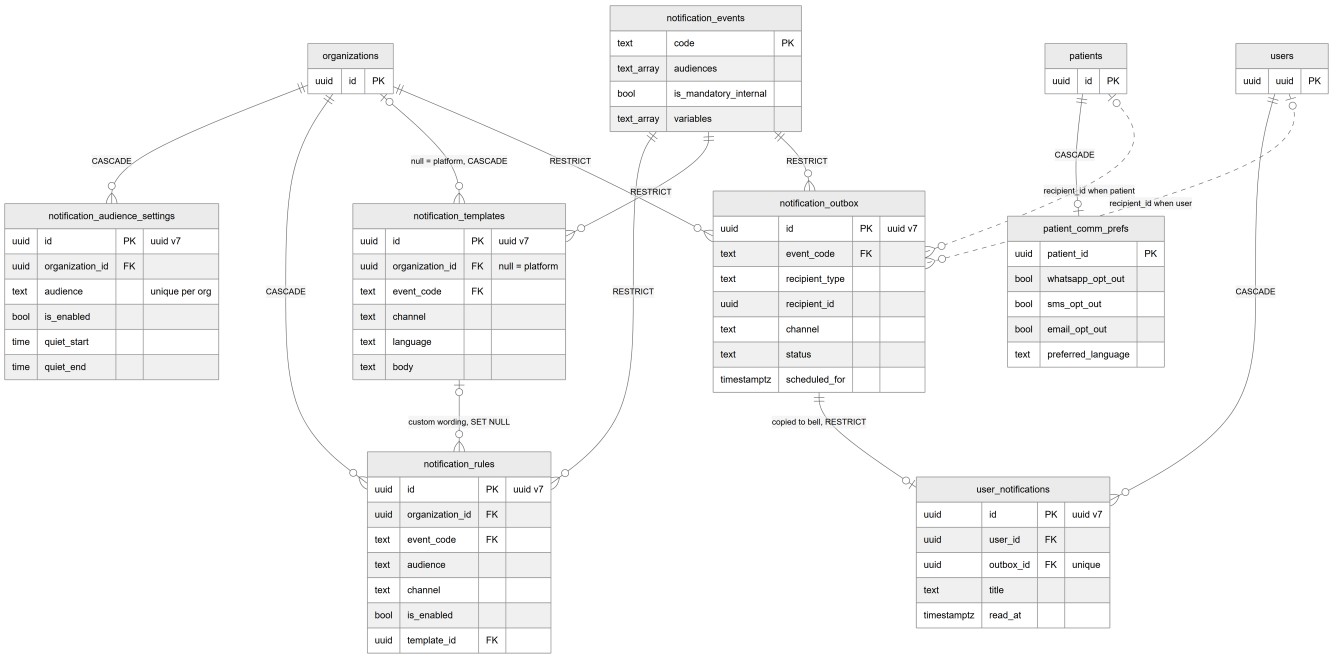


Figure 24.1 — The event catalog, clinic rules, templates, audience settings, the outbox, the bell and patient preferences.

The figure shows `notification_events` at the centre, keyed by a text `code`. It connects with RESTRICT to `notification_rules`, `notification_templates` and `notification_outbox`. `organizations` connects to `notification_audience_settings` and `notification_rules` (CASCADE), to `notification_templates` ("null = platform, CASCADE") and to `notification_outbox` (RESTRICT). `notification_templates` connects to `notification_rules` with "custom wording, SET NULL". `notification_outbox` connects one-to-one to `user_notifications` ("copied to bell, RESTRICT"), and `users` connects to `user_notifications` (CASCADE). `patients` connects one-to-one to `patient_comm_prefs` (CASCADE). Two dotted lines run from `users` and from `patients` to `notification_outbox`, labelled "recipient_id when user" and "recipient_id when patient", showing that the recipient column has no foreign key.

24.2 Tables

`notification_events` — the platform event catalog

One row is one kind of event that can notify someone. The primary key is a text `code`, for example `order.submitted` or `item.ready`. There is no organization column: this is a platform catalog, seeded from code on every start (missing rows are inserted, existing ones are kept). Key columns: `name`, `description`, `audiences` (the audiences it can reach), `is_mandatory_internal` (internal audiences always get it in-app and clinics cannot switch it off), `variables` (placeholders the event supplies) and `sort_order`.

`notification_rules` — per-clinic on/off switches

One row says whether one event is sent to one audience on one channel for one organization, and optionally which template to use. Rows are created only when a clinic changes something. A missing row means the default from the code: on for an internal audience (doctor, coordinator, lab) on the in-app channel, off for everything else. The table has an optimistic `version` column, so two admins saving at once cannot silently overwrite each other.

Constraints that matter:

- Unique per organization, event, audience and channel.

- CHECKs limit `channel` to in_app, whatsapp, sms or email, and `audience` to patient, doctor, coordinator or lab.

`notification_templates` — wording

One row is the wording of one event on one channel in one language (`en`, `ur` or `roman_ur`). If `organization_id` is empty, it is the platform default; a clinic's own row with the same event, channel and language overrides it. Key columns: `subject` (in-app title or email subject), `body` with `{placeholder}` markers, `variables` (computed on save) and `provider_template_ref` (for WhatsApp, the provider-approved template id).

Constraints that matter: two partial unique indexes allow exactly one platform template and one clinic template per event, channel and language.

`notification_audience_settings` — master switch and quiet hours

One row per organization and audience. `is_enabled` is the master switch for that audience. `quiet_start` and `quiet_end` define quiet hours as local wall-clock times, which may wrap past midnight (for example 22:00 to 08:00). `timezone` is the zone for quiet hours; if empty, the location's timezone is used. A missing row means the default: patients off, internal audiences on.

Constraints that matter: unique per organization and audience; quiet start and end must both be set or both be empty.

`notification_outbox` — one message to one recipient on one channel

One row is one message. It is written in the same transaction as the change that caused it. Key columns:

- Who: `audience`, `recipient_type` (user, patient or lab) and `recipient_id`.
- What: `event_code`, `channel`, `payload` (the rendered data), and `entity_type` with `entity_id` (the record that caused it, so a screen can show "messages about this order").
- State: `status`, `scheduled_for`, `attempts`, `provider_message_id`, `last_error` and `sent_at`.

`status` values: `queued`, `held_quiet_hours`, `sent`, `delivered`, `failed`, `skipped_disabled` and `skipped_opt_out`. The relay polls about every two seconds. When the worker claims a row, it increases `attempts` and moves `scheduled_for` five minutes ahead as a lease, so a crashed send is retried later.

Constraints that matter:

- CHECKs on `channel`, `audience` and `status`.
- A partial index over rows in `queued` or `held_quiet_hours`, which keeps the worker's search fast.
- The table is described as never deleted, but no database trigger enforces that.

`user_notifications` — the bell

One row is a user's in-app copy of one sent `in_app` outbox row. Key columns: `title`, `body`, `link`, `event_code`, `organization_id` (so a user who works for several clinics can filter) and `read_at`.

Constraints that matter: `outbox_id` is unique, so a re-delivered send never shows twice; a partial index over unread rows serves the unread count.

`patient_comm_prefs` — a patient's opt-outs

The primary key is the patient id, so each patient has at most one row. Key columns: `whatsapp_opt_out`, `sms_opt_out`, `email_opt_out`, `preferred_language` and `opted_out_at`. It has an optimistic `version` column.

24.3 Relationships explained

- **Organization → audience settings, rules, templates.** Each belongs to one organization (templates: zero or one, because platform defaults have none). Deleting the organization deletes them (CASCADE).
- **Organization → outbox.** Each outbox row belongs to one organization. An organization with outbox rows cannot be deleted (RESTRICT), which keeps the message log.
- **Event → rules, templates, outbox.** Each refers to exactly one event code. An event that is referenced cannot be deleted (RESTRICT). Events are seeded insert-if-missing and are never removed by the code.
- **Template → rules.** A rule may point at one template; deleting the template keeps the rule and clears the link (SET NULL), so the default wording is used again.
- **Outbox → bell entry.** One-to-one: a bell entry belongs to exactly one outbox row, and an outbox row has at most one bell entry. An outbox row with a bell entry cannot be deleted (RESTRICT).
- **User → bell entries.** Each bell entry belongs to one user; deleting the user deletes their bell entries (CASCADE).
- **Patient → preferences.** One-to-one; deleting the patient deletes the preferences (CASCADE).

Logical relationships (no foreign key):

- `notification_outbox.recipient_id` is polymorphic, discriminated by `recipient_type` (a user, a patient or a lab).
- `notification_outbox.entity_id` is polymorphic, discriminated by `entity_type`.
- `notification_outbox.location_id` → `locations`.
- `user_notifications.organization_id` and `patient_comm_prefs.organization_id` → `organizations`.
- Actor columns (`created_by`, `updated_by`) hold a user id.

24.4 Tenant scope

- **Global (platform catalog):** `notification_events`.
- **Organization and location:** `notification_outbox` (location optional).
- **Organization only:** `notification_audience_settings`, `notification_rules`, `notification_templates` (or platform-wide when empty), `user_notifications`, `patient_comm_prefs`.
- `user_notifications` belongs to a user, not a clinic, but carries the organization id so the bell can be filtered per clinic.

Chapter 25 — Labs

This chapter describes the data model of dental-lab work: the lab directory and how clinics connect to labs, the lab's service catalog and prices, lab orders and their items, production (try-in stages, technicians, shipments, remakes, warranties), and the money on both sides: what the patient pays the clinic for lab work, and what the clinic owes the lab. It is for the product owner who needs to know how the roughly fifty lab tables fit together. "Lab" here always means a dental laboratory that makes crowns, bridges and dentures, not a diagnostic test lab. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

25.1 Three ideas that run through every lab table

A lab is not a tenant. A clinic is an organization (the tenant). An external lab works for many clinics, so `labs` has no organization column. A clinic connects to a lab through a **lab link** (`lab_clinic_links`). Lab users are **lab members**: they have no clinic membership at all, so no clinic query can return their data. An internal lab, run by one clinic, records its owner in `owner_organization_id` .

Snapshots and versions. A lab order item copies the service name, options and prices when the order is submitted, and pins the exact form version and workflow version it runs on. Later catalog or workflow changes never alter work already ordered. Changes after submit become a new order revision, never an overwrite.

UUID version 7 keys. Every lab table, and the workflow engine, notification and general-ledger tables, uses a primary key generated by the database function `uuid_generate_v7()` , installed at start-up and by the hand-written migration `1792490000000-lab-management-foundation` . A version 7 UUID is still a normal 128-bit `uuid` , but its first 48 bits are the creation time in milliseconds. Ids therefore sort roughly by creation time, and indexes on append-heavy tables stay compact because new keys land at the end of the index instead of at random places. The remaining bits are still random. Older modules use random version 4 UUIDs.

Money convention. All lab money is `bigint` paisa. Tax rates are basis points (`tax_rate_bp` , `tax_bp`).

Status. The lab module is [Implemented] end to end: clinic screens (labs, lab price list, order templates, lab orders, lab billing, lab payables) and the lab portal for lab members. Exceptions are labelled where they occur.

25.2 Directory, links, members and magic links

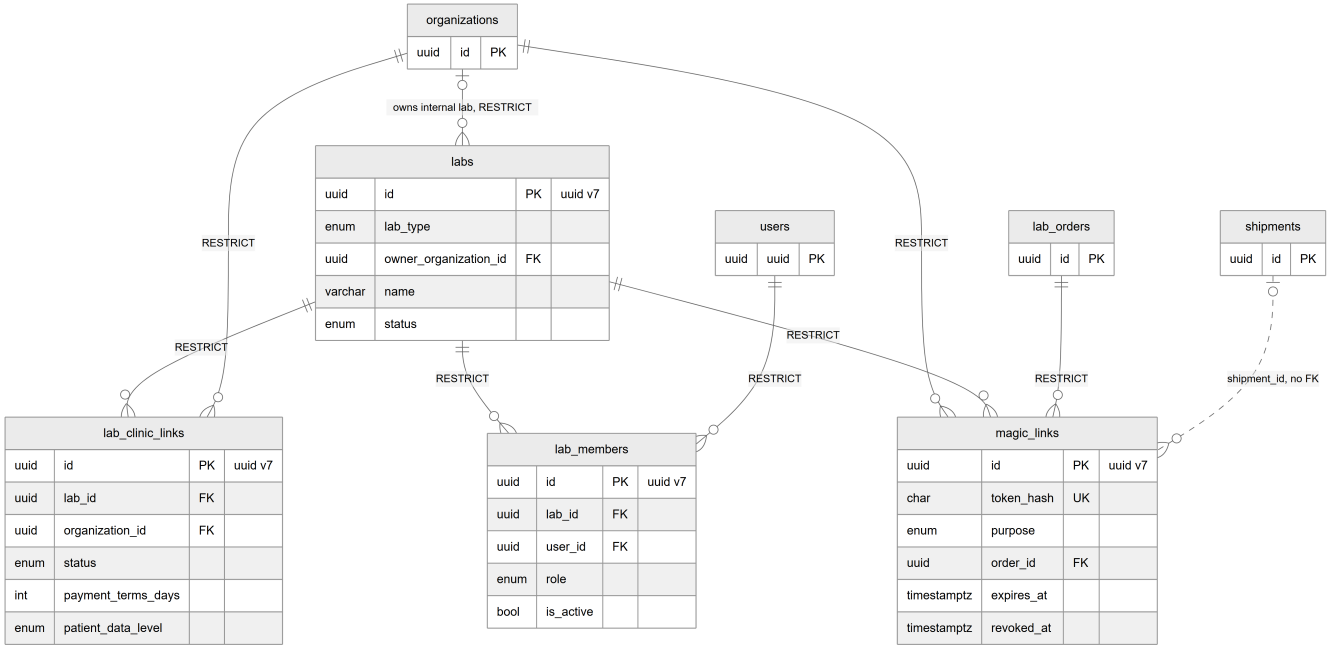


Figure 25.1 — Labs, the clinic-to-lab link, lab members and magic links.

The figure shows `labs` with solid lines to `lab_clinic_links`, `lab_members` and `magic_links`, all RESTRICT. `organizations` connects to `labs` ("owns internal lab, RESTRICT"), to `lab_clinic_links` and to `magic_links` (RESTRICT). `users` connects to `lab_members` (RESTRICT), and `lab_orders` connects to `magic_links` (RESTRICT). A dotted line from `shipments` to `magic_links` is labelled "shipment_id, no FK". All four main boxes note UUID version 7 keys, and `token_hash` on `magic_links` is marked unique.

`labs` — the lab directory

One row is one dental lab. `lab_type` is external or internal; `status` is active or suspended. Contact columns (`phone`, `whatsapp_phone`, `email`, `city`, `address`) hold the lab's business contact. Constraint that matters: `chk_labs_owner` requires `owner_organization_id` to be set exactly when the lab is internal.

`lab_clinic_links` — a clinic working with a lab

One row connects one organization to one lab. `status` is invited, active or suspended. `payment_terms_days` (0–365, default 30) sets the clinic's payment terms. `patient_data_level` decides how much patient identity the lab sees: `full_name` or `code_only`. `activated_at` records the first activation. Status changes go through one service and are logged in `status_history`. Constraint that matters: one link per lab and organization.

`lab_members` — people who work at a lab

One row makes a user a member of a lab, with `role` lab_admin or lab_technician, and `is_active` as a soft delete. Lab-side access is resolved from these rows only. Constraint that matters: one membership per lab and user.

`magic_links` — no-login links for a lab

A magic link lets a lab view or update one order without logging in. The token itself is never stored: the table keeps only `token_hash` (a SHA-256 hash, unique) and a random `nonce` from which the server can re-derive the token at send time. Key columns: `purpose` (lab_view_order, lab_update_order or receive_shipment), `lab_id`,

`order_id`, `delivery` (sent as a notification or copied by the user), `expires_at`, `max_uses` with `used_count`, `last_used_at`, `last_viewed_at`, and the revocation columns `revoked_at`, `revoked_reason` and `revoked_by`. Constraint that matters: `chk_magic_links_uses` keeps `used_count` within `max_uses`.

Status of shipment links: [Not implemented]. The `receive_shipment` purpose and the `shipment_id` column exist, but no code creates such links; clinics receive shipments by scanning the shipment's own QR token. The column comment says the foreign key would be added "until that table exists here"; the `shipments` table now exists but the foreign key was never added.

25.3 Lab catalog

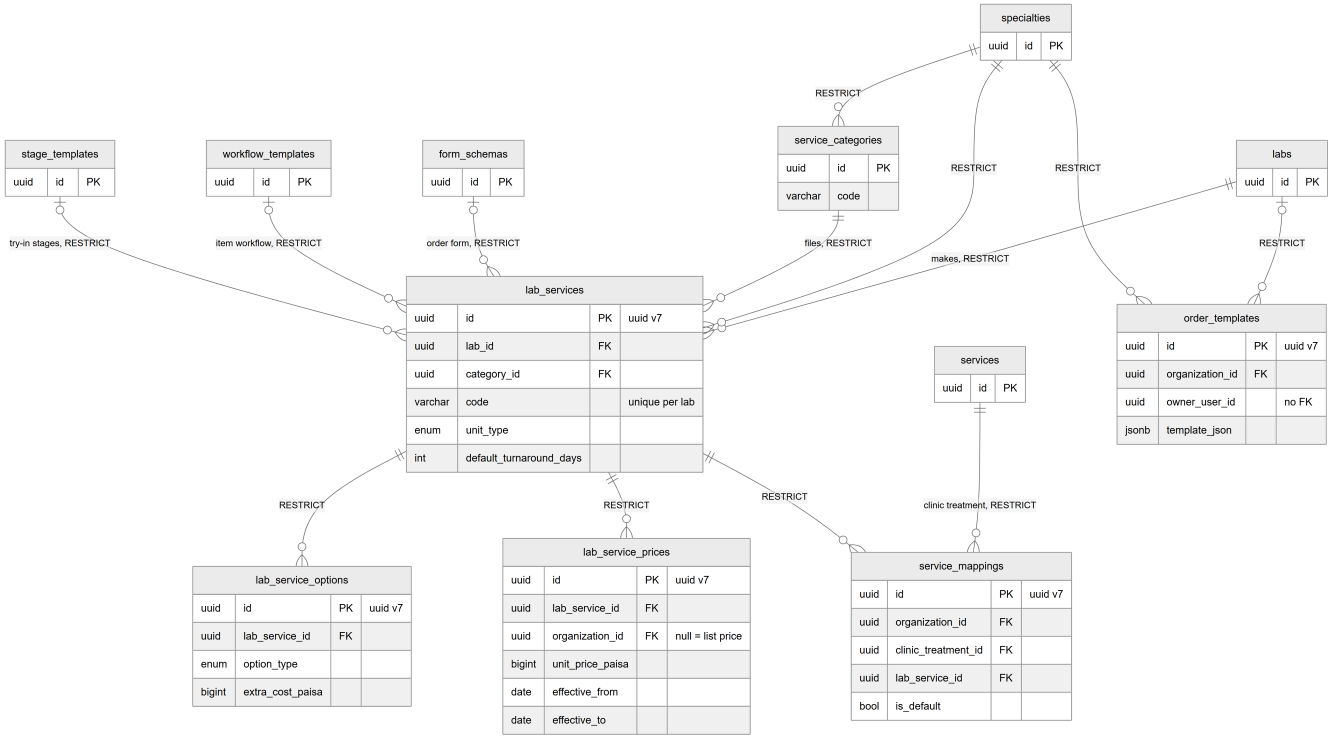


Figure 25.2 — Lab services, options, prices, categories, mappings to clinic services and order templates.

The figure shows `lab_services` at the centre. Solid RESTRICT lines run into it from `labs` ("makes"), `specialties`, `service_categories` ("files"), `form_schemas` ("order form"), `workflow_templates` ("item workflow") and `stage_templates` ("try-in stages"). `specialties` also connects to `service_categories` and to `order_templates`. `lab_services` connects down to `lab_service_options`, `lab_service_prices` and `service_mappings`. `services` (the clinic's billing catalog) connects to `service_mappings` with "clinic treatment, RESTRICT". `labs` connects to `order_templates`. Notes mark `lab_services.code` as "unique per lab", `lab_service_prices.organization_id` as "null = list price", and `order_templates.owner_user_id` as "no FK".

`service_categories` — shared categories

A platform-level grouping of lab services within a specialty, such as CROWN, BRIDGE or DENTURE. Every lab files its services under the same categories, so clinics can compare labs like for like. Unique per specialty and code.

`lab_services` — what a lab makes

One row is one product of one lab, for example a zirconia crown. It has no organization column; clinics see it through their lab link. Key columns: `code` (upper-case, unique per lab), `name`, `unit_type` (per_tooth, per_unit, per_arch or per_case), `default_turnaround_days` (0–365) and `is_active` (archived services are hidden from new

orders; old orders keep them). Three optional links point at the workflow engine: the order form the doctor fills (`form_schema_id`), the item workflow (`workflow_template_id`) and the try-in stage template (`stage_template_id`).

`lab_service_options` — materials and variants

A material or variant of a service, such as "Emax" on a crown, with `extra_cost_paisa` (not negative) added to the resolved price. Unique per service and code.

`lab_service_prices` — price windows

One row is a price for a service over a date window (`effective_from` to `effective_to` , inclusive; an empty end means open-ended). If `organization_id` is empty, it is the lab's list price; if set, it is a negotiated price for that clinic, which wins over the list price. Columns include `unit_price_paisa` , `rush_surcharge_paisa` and `currency` (default PKR). A price entered by mistake is voided (`voided_at` , `void_reason`), not deleted.

Constraint that matters: `excl_lab_service_prices_overlap` , an EXCLUDE constraint (using `btree_gist`), forbids two non-voided windows for the same service and clinic (or the same list price) from overlapping.

`service_mappings` — which lab service makes which clinic treatment

One row says that a lab service fulfils one of the clinic's billable treatments. The clinic treatment is a row of the billing catalog `services` , held in a column named `clinic_treatment_id` . Constraints that matter:

- One row per organization, treatment and lab service.
- At most one active default lab service per treatment (partial unique index).
- An active lab service can be mapped to only one treatment per organization (partial unique index).

`order_templates` — saved orders

A doctor's saved starting point for an order ("my usual zirconia crown"). `owner_user_id` empty means shared with every doctor in the clinic; `lab_id` empty means the doctor picks the lab when ordering. `template_json` holds the order body; `use_count` and `last_used_at` track use.

25.4 Lab orders

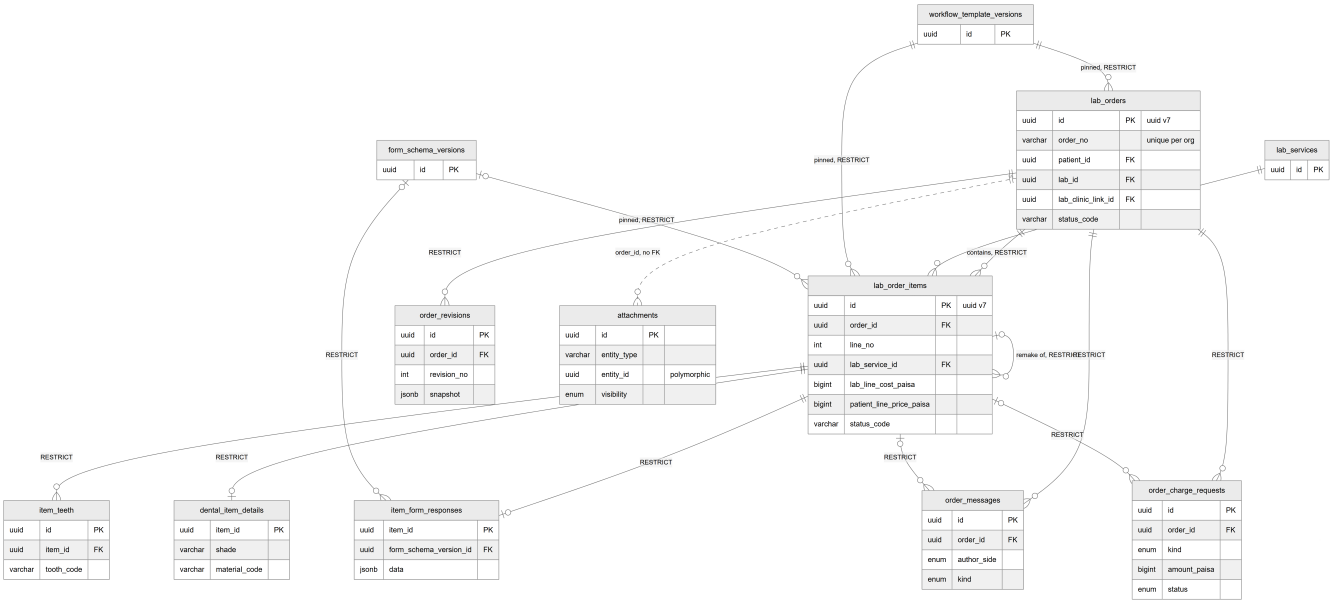


Figure 25.3 — Lab orders, items and their teeth, details, form answers, messages, revisions, charge requests and attachments.

The figure shows `lab_orders` connected to `lab_order_items` ("contains, RESTRICT"). `lab_order_items` has a loop to itself labelled "remake of, RESTRICT". `lab_services` connects to `lab_order_items`; `workflow_template_versions` connects to both `lab_orders` and `lab_order_items` ("pinned"); `form_schema_versions` connects to `lab_order_items` ("pinned") and to `item_form_responses`. From `lab_order_items`, lines run to `item_teeth` (many), `dental_item_details` (one-to-one) and `item_form_responses` (one-to-one). `lab_orders` connects to `order_messages`, `order_revisions` and `order_charge_requests`; items also connect optionally to messages and charge requests. A dotted line from `lab_orders` to `attachments` is labelled "order_id, no FK". All lines are RESTRICT.

`lab_orders` — a lab case

One row is one case: a container of items for one patient, one doctor and one lab, sent from one location. Key columns: `order_no` (for example LAB-0001, assigned at submit from the organization's number sequence), `status_code` (a state of the order workflow version pinned at creation, which never changes), `priority` (normal or urgent), `requested_on` and `lab_expected_on`, the time stamps `submitted_at`, `accepted_at`, `completed_at` and `cancelled_at`, `current_revision_no` (0 for drafts, 1 at submit, +1 per revision) and `idempotency_key`. Optional logical links point at the appointment, the patient's treatment record and the order template the draft started from.

Constraints that matter: `order_no` is unique per organization; the idempotency key is unique per organization, so repeating a create request returns the same draft.

`lab_order_items` — one piece of work

One row is one piece of work in an order, such as a crown or a bridge, numbered by `line_no` (unique per order). Key columns:

- Snapshots frozen at submit: `service_code_snap`, `service_name_snap`, `unit_type`, `options_snap`.
- Prices frozen at submit: `lab_unit_cost_paisa`, `lab_line_cost_paisa` (what the lab charges the clinic), `patient_unit_price_paisa`, `patient_line_price_paisa` (what the clinic charges the patient), `priced_on`,

`currency`.

- Draft inputs: `lab_service_id`, `option_ids`, `rush`, `quantity` (above zero), `clinic_treatment_id`.
- Engine: the pinned `form_schema_version_id` and `workflow_template_version_id`, the copied `stage_template_id`, `status_code` (empty while the order is a draft) and `current_stage_seq`.
- `remake_of_item_id`: set when this item remakes an earlier one.

Only a revision or an approved charge request may change the frozen columns. CHECKs keep all paisa columns non-negative and the quantity above zero.

Item parts: `item_teeth`, `dental_item_details`, `item_form_responses`

- `item_teeth`: one row per tooth of an item; a three-unit bridge has three rows. `numbering_system` is FDI (default), UNIVERSAL or PALMER; `tooth_code` is unique per item; `surfaces` lists tooth surfaces. Rows are written at submit from the form answers, which remain the source of truth.
- `dental_item_details`: one row per item (its primary key is the item id) with typed columns copied from the form answers at submit, such as shade, material and implant details, so reports read columns instead of JSON.
- `item_form_responses`: one row per item (primary key = item id) with the form answers in `data` (JSON) against the pinned form version. Drafts store whatever was typed; `validated_at` is set when submit validates them.

`order_messages`, `order_revisions`, `order_charge_requests`, `attachments`

- `order_messages`: the clinic-to-lab conversation on an order or one item. `author_side` is clinic or lab; `kind` is note, clarification_request or reply; a clarification is resolved by the clinic's reply (`resolved_at`).
- `order_revisions`: what the lab was asked to make, as of one revision. Revision 1 is written at submit; each later change adds a revision with a full `snapshot` and a `reason`. The lab acknowledges each one (`lab_acknowledged_at`). Unique per order and revision number.
- `order_charge_requests`: an extra charge the lab asks for after submit (`kind` rush, material, shipping or other) with `amount_paisa` above zero, a required `reason`, and `status` pending, approved or rejected. An approved item-level request is added to the item's lab line cost.
- `attachments`: a file on an order or item. The file itself is in private storage; the row keeps `storage_key`, `file_name`, `mime_type`, `size_bytes` and `sha256`. `entity_type` is limited by a CHECK to `lab_order` or `lab_order_item`. `visibility` is internal, lab_shared or patient_shared (the lab sees only lab_shared), and `uploaded_side` is clinic or lab. Downloads use short-lived signed URLs.

25.5 Production

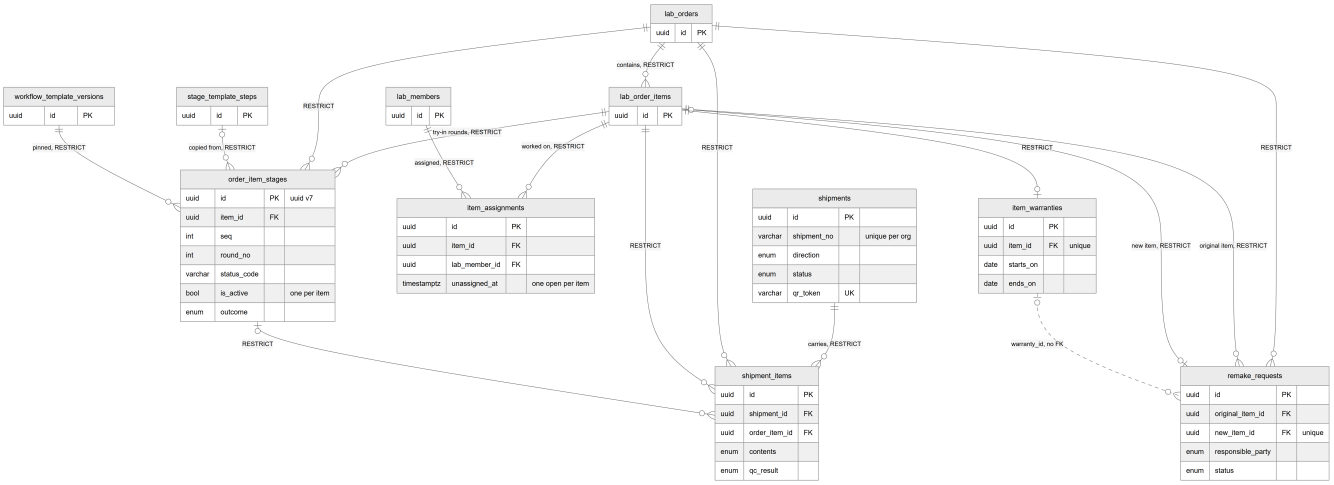


Figure 25.4 — Try-in stages, technician assignments, shipments, remakes and warranties.

The figure shows `lab_orders` and `lab_order_items` at the top. `lab_order_items` connects to `order_item_stages` ("try-in rounds"), `item_assignments` ("worked on"), `shipment_items`, `remake_requests` twice ("original item" many, and "new item" one-to-one) and `item_warranties` (one-to-one). `lab_orders` also connects to `order_item_stages`, `shipment_items` and `remake_requests`. `stage_template_steps` connects to `order_item_stages` ("copied from"), `workflow_template_versions` connects to it ("pinned"), and `lab_members` connects to `item_assignments` ("assigned"). `shipments` connects to `shipment_items` ("carries"), and `order_item_stages` connects optionally to `shipment_items`. A dotted line from `item_warranties` to `remake_requests` is labelled "warranty_id, no FK". All solid lines are RESTRICT. Notes mark `is_active` "one per item" on stages and `unassigned_at` "one open per item" on assignments.

`order_item_stages` — try-in rounds

One row is one round of one stage of an item, for example "metal try-in, round 2". Rows are created from the item's stage template when the lab accepts the order. A correction closes the round and adds a new row with `round_no + 1`; rounds are never overwritten. Key columns: `seq`, `round_no`, copied `code_snap` and `name_snap`, `is_final` (the last stage, finished work rather than a try-in), `requires_patient_visit`, `status_code`, `lab_expected_on`, an optional try-in `appointment_id`, `outcome` (approved, correction or rejected), notes, and who decided.

Constraints that matter: unique per item, sequence and round; at most one active stage per item (partial unique index); round and sequence above zero.

`item_assignments` — which technician works on an item

One row assigns a lab member to an item. Reassigning closes the open row (`unassigned_at`) and adds a new one. Constraint: at most one open assignment per item (partial unique index).

`shipments` and `shipment_items` — parcels between clinic and lab

A shipment is one parcel in either direction (`direction` to_lab or to_clinic) between one clinic and one lab; it can carry items of several orders. `shipment_no` (for example SHP-0001) is unique per organization. `status` is dispatched, partially_received, received or lost. `qr_token` is unique platform-wide and printed on the slip; scanning it opens the receive screen. A shipment item is one item in a parcel, with `contents` (impression, model, framework, tryin, final or other), `quantity`, `received_qty` (CHECK: not above quantity), `condition`, `qc_result` and `qc_notes`. An item appears at most once per shipment.

`remake_requests` — asking for work to be redone

One row asks to remake an item. `reason_code` is shade, fit, fracture, design, damaged or other. `responsible_party` (lab, clinic, patient or undetermined) decides pricing: if the lab is at fault, both the lab charge and the patient charge are zero. `status` is requested, agreed, disputed or closed. If agreed, a new item is created in the same order and linked through `new_item_id`. An active warranty marks the request as a warranty claim automatically.

Constraints that matter: `new_item_id` is unique; at most one open (requested or disputed) request per original item.

`item_warranties` — warranty after delivery

One row per item (the item id is unique) with `starts_on`, `ends_on` (CHECK: not before the start), `terms`, and void columns. The warranty starts when the item is delivered.

25.6 Lab finance: what the patient owes the clinic

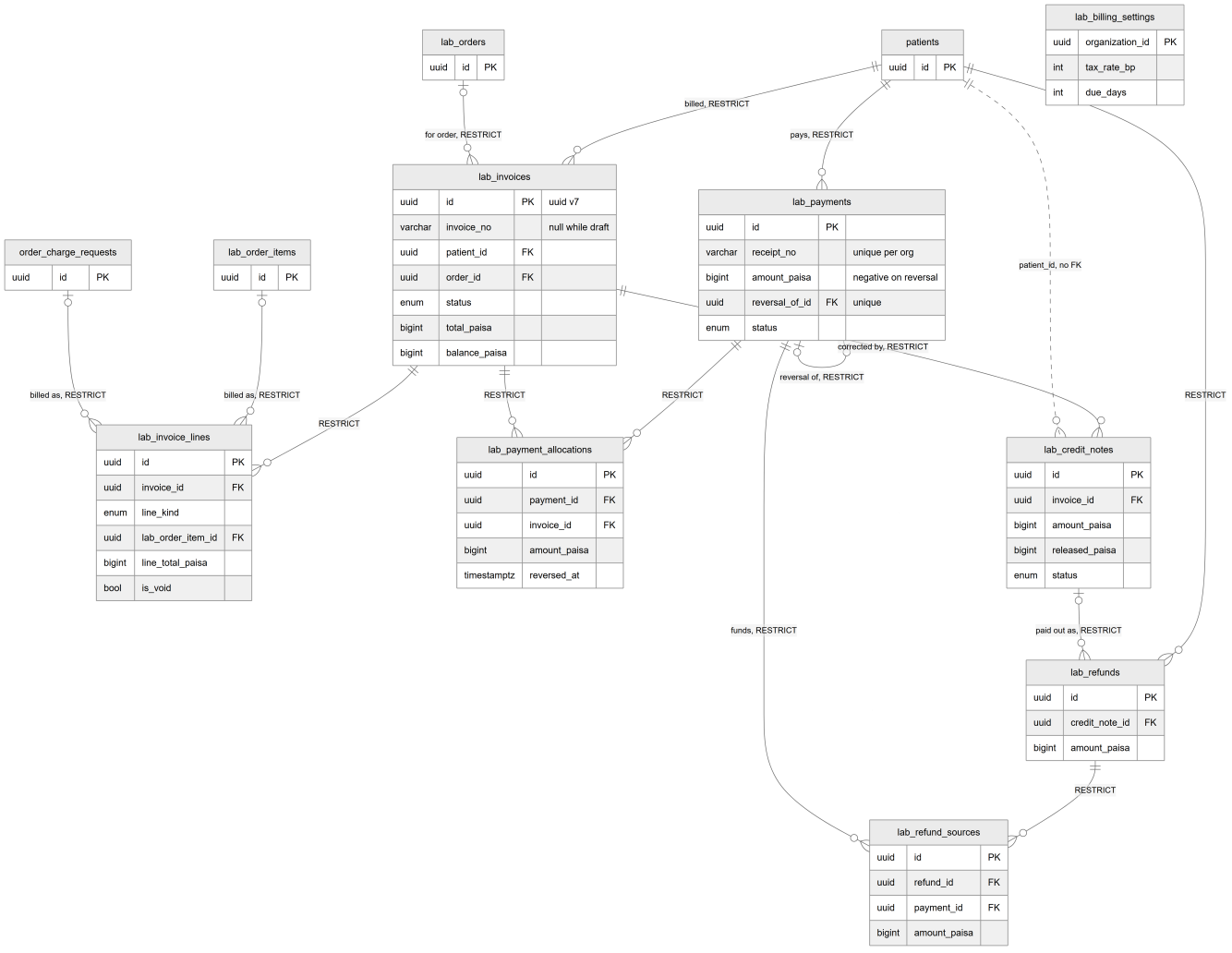


Figure 25.5 — Patient lab invoices, lines, payments, allocations, credit notes, refunds and settings.

The figure shows `patients` with solid lines to `lab_invoices` ("billed"), `lab_payments` ("pays") and `lab_refunds`, and a dotted line to `lab_credit_notes` (patient_id, no FK). `lab_orders` connects to `lab_invoices` ("for order"). `lab_invoices` connects to `lab_invoice_lines`, `lab_payment_allocations` and `lab_credit_notes` ("corrected by"). `lab_order_items` and `order_charge_requests` connect to `lab_invoice_lines` ("billed as"). `lab_payments`

has a one-to-one loop to itself ("reversal of") and connects to `lab_payment_allocations` and `lab_refund_sources` ("funds"). `lab_credit_notes` connects to `lab_refunds` ("paid out as"), and `lab_refunds` connects to `lab_refund_sources`. `lab_billing_settings` stands alone with its key `organization_id`. All solid lines are RESTRICT.

This side mirrors the clinic billing design (the Billing core chapter, Chapter 20) but with its own tables and in paisa. The UI calls these **patient lab invoices**.

- **`lab_invoices`.** A patient invoice for lab work, optionally for one order. A `draft` is an estimate: it has no number, can be edited and owes nothing. Issuing gives it an `invoice_no` (unique per organization) and freezes the lines. `status` is draft, issued, partially_paid, paid or void. Cached totals (`subtotal_paisa`, `discount_paisa`, `tax_paisa`, `total_paisa`, `credited_paisa`, `paid_paisa`, `balance_paisa`) are recomputed from source rows by one service. CHECK `chk_lab_invoices_math`: total = subtotal − discount + tax, and balance, paid and credited are not negative.
- **`lab_invoice_lines`.** One billed thing: `line_kind` lab_item, lab_charge or manual. CHECK `chk_lab_invoice_lines_math` enforces quantity × unit − discount + tax = line total. Two partial unique indexes make sure a lab order item or charge request sits on at most one non-void line, so it is never billed twice; voiding the invoice copies `is_void` onto the lines and frees them. Lines also carry a commission snapshot (`doctor_id`, `performing_assignment_id`, `commission_plan_id`, rate and fixed amount), all without foreign keys; the commission ledger links to these lines (the Compensation chapter, Chapter 22).
- **`lab_payments`.** Money received from a patient for lab work. It belongs to the patient; the unallocated, unrefunded part is the patient's advance. A wrong payment is never deleted: a reversal row with a negative amount points at it through `reversal_of_id` (unique). `received_on` is the location-local day. `receipt_no` and `idempotency_key` are each unique per organization. CHECK `chk_lab_payments_amounts` ties the sign of the amount to whether it is a reversal and keeps allocated + refunded within the amount.
- **`lab_payment_allocations`.** "This much of payment P settles invoice I", always positive. A reversal or credit note stops it counting by setting `reversed_at` and `release_reason`; rows are not deleted.
- **`lab_credit_notes`.** Corrects an issued invoice downwards. Money the patient already paid above the new total is released back to advance (`released_paisa`) and may then be refunded (`refunded_paisa`). `credit_note_no` is unique per organization. CHECK: refunded ≤ released ≤ amount.
- **`lab_refunds`** and **`lab_refund_sources`.** A refund pays money back out of advance; refund sources say which payments funded it. `credit_note_id` set means the refund pays out a credit note. `refund_no` and `idempotency_key` are unique per organization.
- **`lab_billing_settings`.** One row per organization (primary key = organization id): `tax_rate_bp` (0–10,000, default 0 until the accountant confirms) and `due_days`. A missing row means defaults.

25.7 Lab finance: what the clinic owes the lab

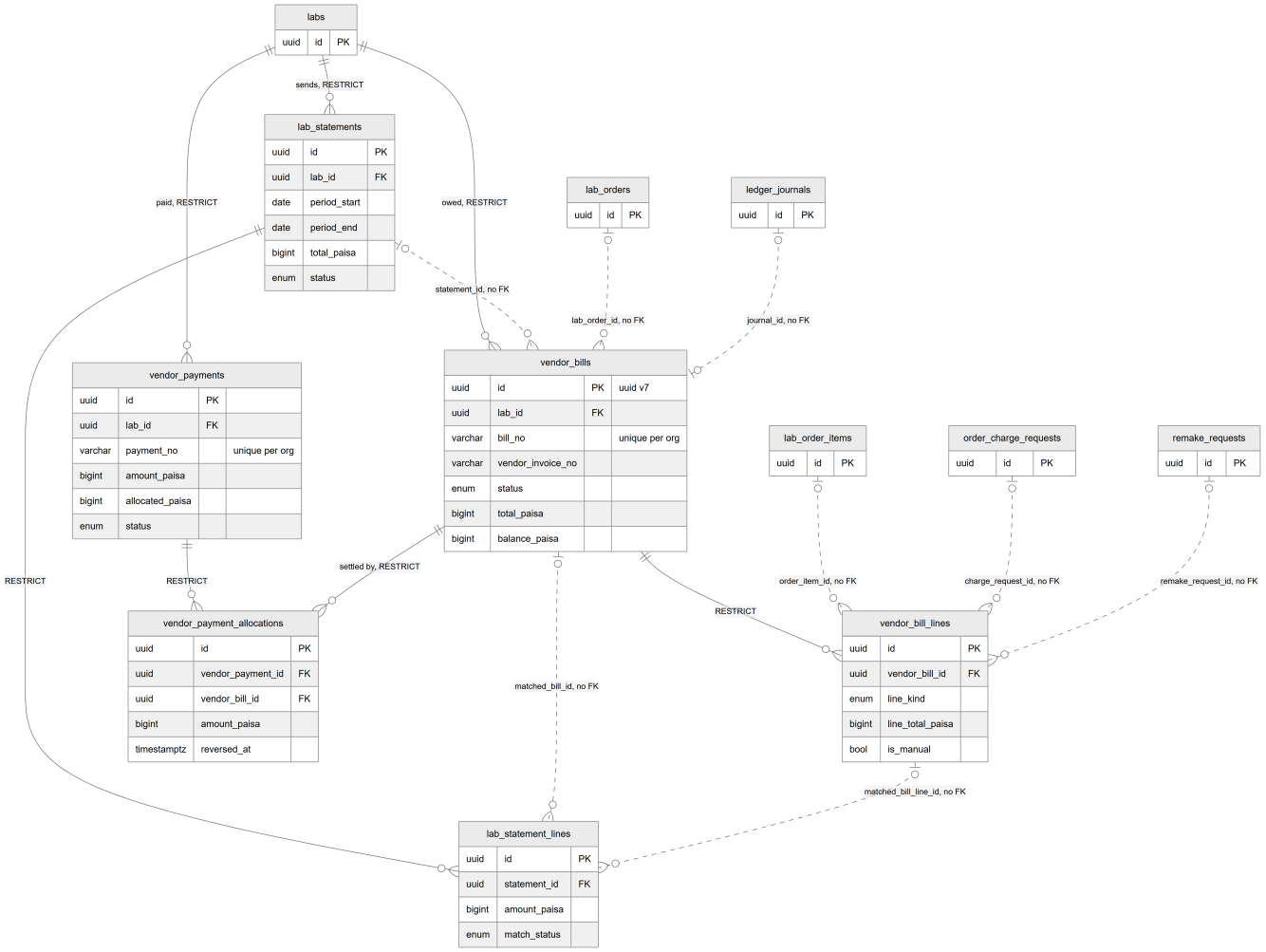


Figure 25.6 — Vendor bills, vendor payments, allocations and lab statements.

The figure shows `labs` with solid lines to `vendor_bills` ("owed"), `vendor_payments` ("paid") and `lab_statements` ("sends"). `vendor_bills` connects to `vendor_bill_lines` and `vendor_payment_allocations` ("settled by"); `vendor_payments` connects to `vendor_payment_allocations`; `lab_statements` connects to `lab_statement_lines`. Dotted lines (no foreign key) run from `lab_statements` to `vendor_bills` (statement_id), from `vendor_bills` and `vendor_bill_lines` to `lab_statement_lines` (matched bill and matched bill line), from `lab_orders` to `vendor_bills` (lab_order_id), from `ledger_journals` to `vendor_bills` (journal_id), and from `lab_order_items`, `order_charge_requests` and `remake_requests` to `vendor_bill_lines`. All solid lines are RESTRICT.

- **`vendor_bills`.** What the clinic owes a lab. One draft per lab order is created when the lab accepts the order, from the items' frozen lab costs (a partial unique index allows only one draft per order). Nothing reaches the general ledger until approval, which posts a Lab Cost / Lab Payable journal (`journal_id`). `status` is draft, approved, partially_paid, paid or void. `bill_no` is unique per organization; the lab's own invoice number (`vendor_invoice_no`), entered at approval, is unique per organization and lab among non-void bills. CHECK: total = subtotal + tax, and paid and balance are not negative. Bills are voided, never deleted.
- **`vendor_bill_lines`.** One cost on a bill: `line_kind` service, rush, shipping, material, remake or other. Lines of a draft follow the order and are re-synced; after approval they are frozen. `is_manual` marks lines added by hand. Links to the order, item, charge request and remake request have no foreign keys.
- **`vendor_payments`** and **`vendor_payment_allocations`.** Money the clinic paid a lab, split over approved bills by allocations. An unallocated rest is an advance to the lab. `payment_no` is unique per organization. A mistake is

reversed (`status` reversed, `reversed_at`), with a mirror journal; allocations are reversed with `reversed_at`, never deleted.

- **`lab_statements` and `lab_statement_lines`.** A lab's monthly statement, typed in by the clinic and reconciled against the bills of the period. `status` is uploaded, reconciling, reconciled or disputed. Each line has the lab's reference, date and amount, the clinic's `system_amount_paisa` for the same reference, and `match_status` (matched, amount_mismatch, missing_in_system or missing_in_statement).

Status of statement files: [Not implemented]. `lab_statements.attachment_id` exists, but the column comment says there is no storage for statement files yet, because attachments are order-scoped.

25.8 Relationships explained

Every foreign key in the lab tables uses ON DELETE RESTRICT: no lab record can be deleted while anything refers to it. Records are voided, reversed, archived or switched off instead.

Directory, links and magic links:

- An internal lab belongs to zero or one owner organization; an organization can own many labs.
- Each lab link belongs to exactly one lab and one organization; one link per pair.
- Each lab member belongs to exactly one lab and one user; one membership per pair.
- Each magic link belongs to exactly one organization, one lab and one lab order; an order can have many links.

Catalog:

- Each lab service belongs to exactly one lab, one specialty and one service category, and optionally to one form schema, one workflow template and one stage template.
- Each service category belongs to exactly one specialty.
- Each option and each price row belongs to exactly one lab service. A price row belongs to zero or one organization (empty = list price).
- Each service mapping belongs to exactly one organization, one clinic service (`services`) and one lab service.
- Each order template belongs to exactly one organization and one specialty, and optionally to one lab.

Orders:

- Each lab order belongs to exactly one organization, location, specialty, patient, lab, lab link and pinned workflow version, and optionally to one doctor (employee).
- Each order item belongs to exactly one order, one lab service and one pinned workflow version, and optionally to one form version and to one earlier item it remakes.
- Each tooth row belongs to exactly one item; each item has at most one details row and at most one form-response row (both keyed by the item id). A form response may point at one form version.
- Each message, revision and charge request belongs to exactly one order; messages and charge requests may also point at one item.

Production:

- Each stage row belongs to exactly one order, one item and one pinned workflow version, and optionally to one stage-template step.
- Each assignment belongs to exactly one item and one lab member.
- Each shipment item belongs to exactly one shipment, one order and one item, and optionally one stage.

- Each remake request belongs to exactly one order and one original item, and to zero or one new item (one-to-one).
- Each warranty belongs to exactly one item (one-to-one through the unique item id).

Finance:

- Each patient lab invoice belongs to exactly one patient and optionally one order; each line belongs to exactly one invoice and optionally one item or one charge request.
- Each lab payment belongs to exactly one patient and may reverse exactly one earlier payment (one-to-one).
- Each allocation belongs to exactly one payment and one invoice. Each credit note belongs to exactly one invoice.
- Each lab refund belongs to exactly one patient and optionally one credit note; each refund source belongs to exactly one refund and one payment.
- Each vendor bill and vendor payment belongs to exactly one lab; each bill line belongs to exactly one bill; each vendor allocation belongs to exactly one vendor payment and one bill.
- Each lab statement belongs to exactly one lab; each statement line belongs to exactly one statement.

Logical relationships (no foreign key):

- `magic_links.shipment_id` → `shipments` (unused, see 25.2).
- `order_templates.owner_user_id` → `users`.
- `attachments.entity_id` is polymorphic (order or item); `attachments.order_id` → `lab_orders`.
- `lab_order_items.clinic_treatment_id` → `services`, `option_ids` → `lab_service_options` (array), `lab_price_id` → `lab_service_prices`, `stage_template_id` → `stage_templates`. Note that `service_mappings.clinic_treatment_id` has a foreign key to the same `services` table, but the item's copy does not.
- `lab_orders.appointment_id` → `appointments`, `treatment_id` → `treatments`, `order_template_id` → `order_templates`.
- `order_messages.author_user_id` → `users`.
- `item_assignments.lab_id`, `order_item_stages.lab_id`, `remake_requests.lab_id`, `shipments.lab_id` → `labs`. These copies let the lab portal filter by lab quickly.
- `order_item_stages.appointment_id` → `appointments`; `remake_requests.warranty_id` → `item_warranties`.
- `lab_credit_notes.patient_id` → `patients`.
- `lab_invoice_lines.clinic_treatment_id` → `services`, `doctor_id` → `employees`, `performing_assignment_id` → `employee_org_location_assignments`, `commission_plan_id` → `compensation_plans`.
- `vendor_bills.lab_order_id` → `lab_orders`, `statement_id` → `lab_statements`, `journal_id` → `ledger_journals`.
- `vendor_bill_lines.lab_order_id`, `order_item_id`, `charge_request_id`, `remake_request_id` → the matching lab tables.
- `lab_statement_lines.matched_bill_id` → `vendor_bills` and `matched_bill_line_id` → `vendor_bill_lines`.
- `lab_statements.attachment_id` → `attachments` (unused).
- Actor columns (`created_by`, `updated_by`, `decided_by`, `received_by` and others) hold a user id.

25.9 Tenant scope

- **Global, lab-side:** `labs` (internal labs carry an owner organization), `lab_members`, `lab_services`, `lab_service_options`. These are scoped by `lab_id`, not by organization.

- **Global, platform catalog:** `service_categories`.
- **Organization and location:** `lab_orders`, `lab_invoices`, `lab_payments`, `lab_credit_notes`, `lab_refunds`, `vendor_bills`, `vendor_payments`.
- **Organization only:** every other lab table, including items, item parts, messages, revisions, charge requests, attachments, all production tables, invoice lines, allocations, refund sources, settings, bill lines, vendor allocations, statements and statement lines. `lab_service_prices` is organization-scoped when a negotiated price, global when a list price.
- Only `lab_clinic_links`, `magic_links`, `lab_service_prices`, `service_mappings`, `order_templates` and `lab_orders` have foreign keys on their tenant columns. Everywhere else the organization id is stored without a constraint and the application enforces isolation.

Chapter 26 — Workflow engine and common tables

This chapter describes two groups of tables. The first is the workflow engine: twelve tables that store specialties, versioned forms, try-in stage templates and versioned state machines, which the lab module runs on. The second is the shared infrastructure in `backend/src/common/`: the generic audit log, the status history and the per-organization number counters, plus the unused `chairs` table. It is written for the product owner who needs to know which behaviour is configured as data rather than code, and which history can never be changed. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

26.1 The workflow engine in one paragraph

A **specialty** (for example DENTAL) is the root. Under it, the Platform keeps **form schemas** (what the doctor fills in when ordering a lab item), **stage templates** (the try-in rounds a piece of work goes through) and **workflow templates** (state machines for lab orders, lab order items and stages). Forms and workflows are versioned: a version is a draft while it is edited, becomes published and then frozen, and is finally retired. Every lab order and item pins the exact version it started on, so editing a form or workflow never breaks work in progress. When a workflow version is published, its JSON definition is turned into rows of states, transitions and transition effects, which the runtime reads. Effects are code handlers listed in a registry; configuration can only choose among them, never add new code.

Status: [Implemented]. The super admin edits forms, workflows and stage templates in Config Studio (sidebar entries "Form schemas", "Workflows" and "Stage templates", super admin only). The lab order and production services run on the published versions.

Keys. All engine tables with an `id` use UUID version 7 keys (`uuid_generate_v7()`); the Labs chapter (Chapter 25) explains them. `organization_specialties`, `transition_effects` and `workflow_effects` have natural keys supplied by the application.

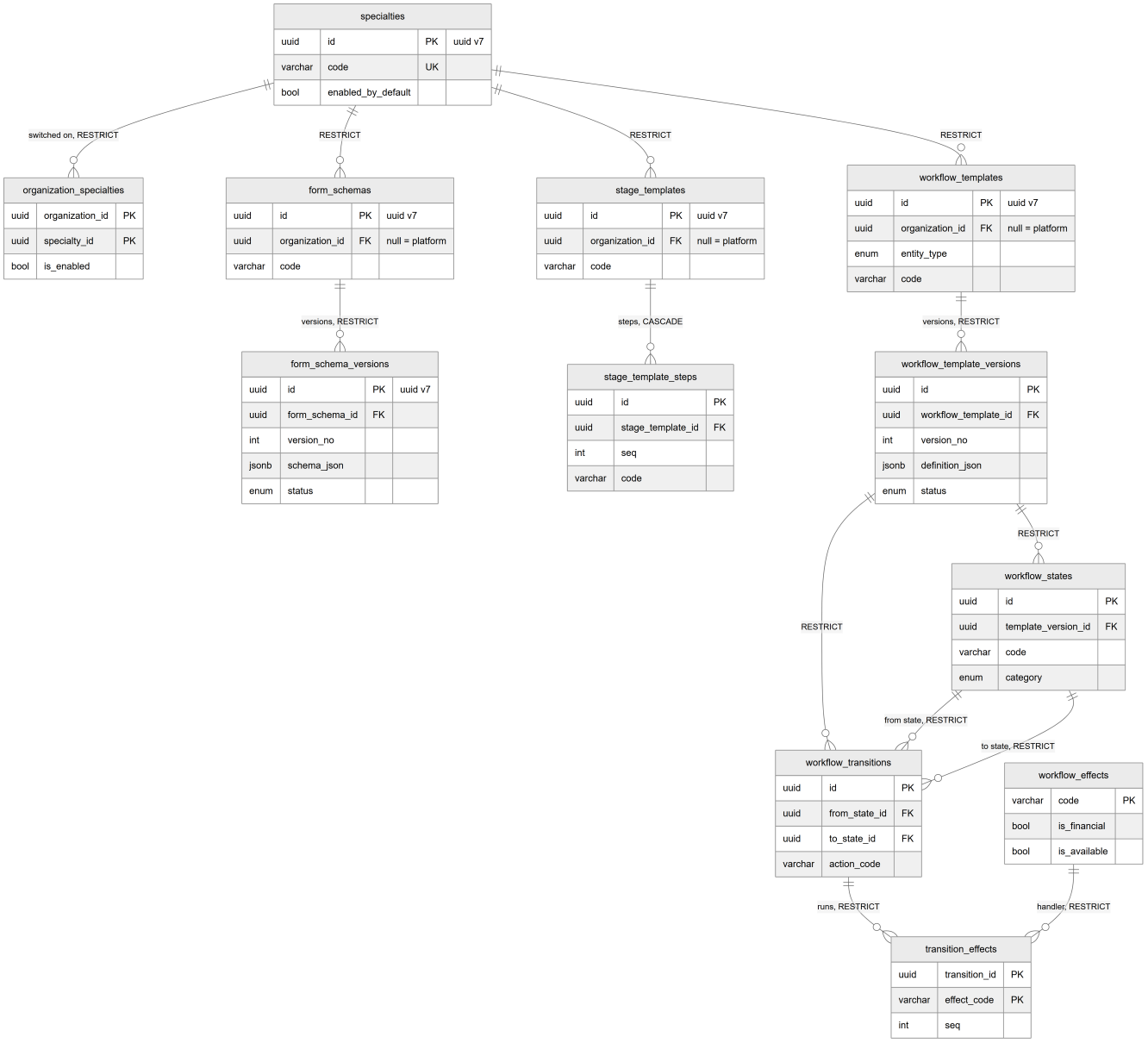


Figure 26.1 — Specialties, form schemas and versions, stage templates and steps, workflow templates, versions, states, transitions and effects.

The figure shows `specialties` at the top with four lines down: to `organization_specialties` ("switched on"), `form_schemas`, `stage_templates` and `workflow_templates`, all RESTRICT. `form_schemas` connects to `form_schema_versions` ("versions, RESTRICT"). `stage_templates` connects to `stage_template_steps` ("steps, CASCADE"). `workflow_templates` connects to `workflow_template_versions` ("versions, RESTRICT"). `workflow_template_versions` connects to `workflow_states` and to `workflow_transitions`. `workflow_states` connects twice to `workflow_transitions`, labelled "from state" and "to state". `workflow_transitions` and `workflow_effects` both connect to `transition_effects` ("runs" and "handler"). All lines except the stage steps are RESTRICT. The template boxes mark `organization_id` as "null = platform".

26.2 Engine tables

`specialties` and `organization_specialties`

`specialties` is a platform catalog of clinical specialties, each with a unique upper-case `code`, a `name`, `is_active`, and `enabled_by_default` (new organizations get it switched on; DENTAL does). `organization_specialties` records which specialties one organization works in, with `is_enabled`. Its primary key is the pair (organization, specialty). Rows are created lazily from `enabled_by_default` the first time an organization is read.

`form_schemas` and `form_schema_versions` — versioned order forms

A form schema is the stable identity of a form, such as CROWN or BRIDGE, within a specialty. If `organization_id` is empty, it is the platform default; an organization's own schema with the same code overrides the default for that organization. Partial unique indexes allow one platform schema and one per organization for each specialty and code.

A form schema version holds the content: `schema_json` (the field definitions), `ui_json` (layout, such as tooth-chart settings), `promotions` (which answers are copied into typed columns, for example into `dental_item_details`) and `status` (draft, published or retired) with `published_at` and `retired_at`.

Constraints that matter:

- `version_no` is unique per form and above zero.
- At most one draft and at most one published version per form (two partial unique indexes).
- `chk_form_schema_versions_published_at`: a draft has no publish date, and every non-draft has one.

Correction to the catalog. The entity's code comment, repeated in the catalog, says a database trigger makes published versions immutable. No such trigger exists. The rule is enforced by the form schemas service, which allows content changes only on drafts.

`stage_templates` and `stage_template_steps` — try-in rounds

A stage template lists the try-in rounds a kind of work goes through, for example metal, then bisque, then final for a porcelain-fused-to-metal crown. It follows the same platform-or-organization pattern as forms. Each step has a `seq` (1-based, unique per template), a `code` (unique per template), a `name`, `requires_patient_visit`, `requires_doctor_approval` and `default_lab_days` (0–365). Lab orders copy step names into their own stage rows, so editing a template never changes work in progress.

`workflow_templates`, `workflow_template_versions`, `workflow_states`, `workflow_transitions`

A workflow template is a state machine for one kind of record: `entity_type` is order, item or stage. It follows the platform-or-organization pattern. A workflow template version holds the draft definition as JSON (`definition_json`) and a `status` of draft, published or retired; as with forms, at most one draft and one published version exist per template, and `version_no` is unique and above zero. Published versions never change; this is enforced by the service, not the database.

Publishing a version writes its rows once:

- `workflow_states`: one row per state, with a `code` unique per version, a `name`, a `category` (initial, active, waiting, done or cancelled), `is_terminal` and `sort_order`.
- `workflow_transitions`: one row per allowed move from one state to another under an `action_code`, unique per version, from-state, action and to-state. `allowed_sides` and `allowed_roles` say who may perform it (an

empty role list means any role of an allowed side), `required_fields` lists data that must be present and `requires_reason` forces a reason.

`workflow_effects` and `transition_effects`

`workflow_effects` is a registry of effect handlers that exist in code, keyed by `code`. It is synced from the in-memory registry on every start. `is_financial` marks effects that validate money themselves; `is_available` is false when a handler was removed from code but old versions still mention it. `transition_effects` lists which effects a transition runs, in `seq` order (unique per transition), with optional JSON `params`; its primary key is the pair (transition, effect code). Effects run inside the transition's transaction.

26.3 Common tables

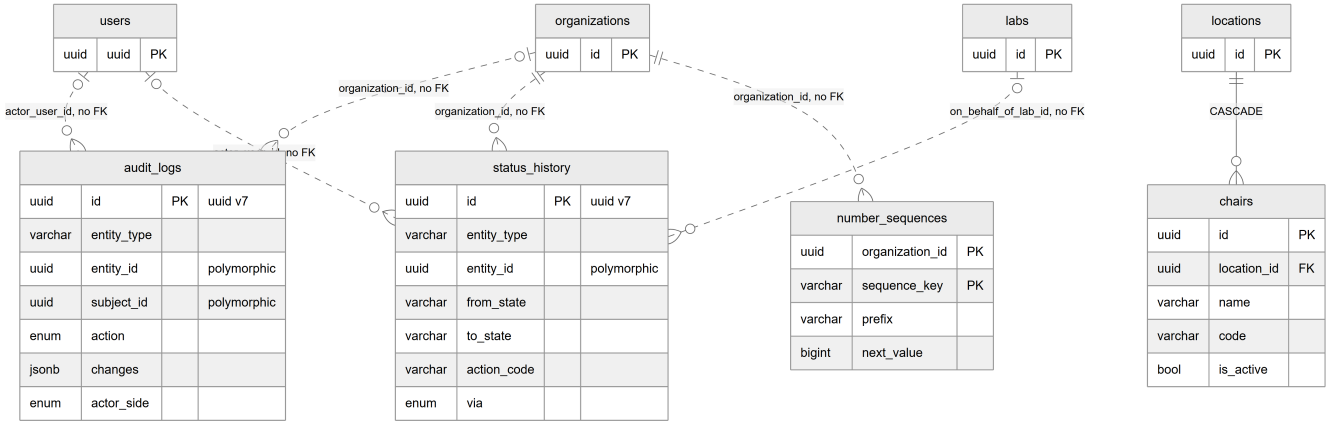


Figure 26.2 — The audit log, status history, number sequences and the unused chairs table.

The figure shows four main boxes. `audit_logs` receives dotted lines from `organizations` (organization_id) and `users` (actor_user_id). `status_history` receives dotted lines from `organizations`, `users` and `labs` (on_behalf_of_lab_id). `number_sequences` receives a dotted line from `organizations`. All dotted lines mean "no FK". `chairs` is the only box with a solid line, from `locations`, labelled CASCADE. `audit_logs` and `status_history` both show `entity_type` and an `entity_id` marked "polymorphic".

`audit_logs` — the generic audit trail

Status: [Implemented]. Used by about eighteen modules, including patients, appointments, staff, payroll, organizations, the engine and every lab module.

One row records who changed what. `entity_type` and `entity_id` name the changed record. `subject_id` names the record whose history this row joins when that is different, for example a patient for a change to the patient's vitals. `action` is create, update, void, delete, export or view_sensitive. `changes` holds `{ before, after }` for the changed fields only. `actor_user_id` and `actor_side` (clinic, lab, platform or system) say who did it, and `ip` and `request_id` record where it came from. `organization_id` is empty for platform-level records, such as a lab edited by a super admin.

Append-only trigger. The hand-written migration `1792490000000-lab-management-foundation` installs `trg_audit_logs_append_only`, which calls `reject_append_only_mutation()` before any UPDATE or DELETE and raises an error. Rows can only be inserted.

`status_history` — every status change

Status: [Implemented].

One row records one status change: `from_state` (empty for the creating transition), `to_state`, `action_code`, who did it (`actor_user_id`, `actor_side`), how (`via`: app, portal, magic_link, proxy or qr), an optional `reason` and `metadata`. `on_behalf_of_lab_id` is set when clinic staff updated something on the lab's behalf (`via = proxy`). States are free text, because workflow states are data, not code enums. The same migration installs `trg_status_history_append_only`, so rows cannot be updated or deleted.

`number_sequences` — human-readable numbers per organization

Status: [Implemented].

One row is a gapless counter for one organization and one `sequence_key`, with a `prefix` and `next_value` (CHECK: above zero). The primary key is the pair (organization, sequence key), supplied by the application. It is only touched through the numbering service inside a transaction, which increments the counter in a single statement. It produces numbers such as LAB-0001 for lab orders and SHP-0001 for shipments, and also numbers patient lab invoices, lab receipts, credit notes, lab refunds, vendor bills and vendor payments. Clinic invoice, receipt and refund numbers use a different mechanism, `document_sequences` (the Billing ledgers chapter, Chapter 21).

`chairs` — dental chairs

Status: [Not implemented]. Entity only.

The table would list the treatment chairs of a location: `name`, `code`, `description` and `is_active`. There is no module, service, controller or screen, and no module registers the entity, so the running application never creates the table in development. A production schema built with the command-line schema tools may contain it. An early link from appointments to chairs no longer exists.

26.4 Relationships explained

Engine:

- **Specialty → organization specialties, form schemas, stage templates, workflow templates.** Each belongs to exactly one specialty; a specialty in use cannot be deleted (RESTRICT).
- **Organization → organization specialties.** Deleting the organization deletes its specialty switches (CASCADE).
- **Organization → form schemas, stage templates, workflow templates.** Each belongs to zero or one organization (empty = platform default). An organization with its own definitions cannot be deleted (RESTRICT).
- **Form schema → versions.** Each version belongs to exactly one form; a form with versions cannot be deleted (RESTRICT).
- **Stage template → steps.** Each step belongs to exactly one template; deleting the template deletes its steps (CASCADE). Lab stage rows that point at a step block that delete in practice (RESTRICT from `order_item_stages`).
- **Workflow template → versions.** RESTRICT.
- **Workflow version → states and transitions.** Each belongs to exactly one version (RESTRICT).
- **State → transitions.** Each transition has exactly one from-state and one to-state (RESTRICT on both).
- **Transition → transition effects** and **effect → transition effects.** Each transition effect belongs to exactly one transition and one effect (RESTRICT on both).
- **Links from the lab tables.** Lab services point at form schemas, workflow templates and stage templates; lab orders, items and stages pin workflow versions; items and form responses pin form versions. All are RESTRICT

(the Labs chapter, Chapter 25, lists them).

Common:

- **Location → chairs.** Each chair belongs to one location; deleting the location deletes its chairs (CASCADE).
- `audit_logs` and `status_history` have no foreign keys at all. `entity_id` and `subject_id` are polymorphic; `organization_id`, `actor_user_id` and `on_behalf_of_lab_id` are logical references. This is deliberate for an audit table: history must survive even if the record it describes is gone.
- `number_sequences.organization_id` has no foreign key.
- Actor columns in the engine (`created_by`, `updated_by`, `published_by`) hold a user id with no foreign key.

26.5 Tenant scope

- **Global, platform catalog:** `specialties`, `workflow_effects`.
- **Platform or organization:** `form_schemas`, `stage_templates`, `workflow_templates` (empty organization = platform default). Their child tables (`form_schema_versions`, `stage_template_steps`, `workflow_template_versions`, `workflow_states`, `workflow_transitions`, `transition_effects`) have no organization column and inherit the parent's scope.
- **Organization only:** `organization_specialties`, `audit_logs` (empty for platform records), `status_history`, `number_sequences`.
- **Location only:** `chairs` (the organization is reachable through `locations`).

Chapter 27 — Overlaps and ambiguities

This chapter collects the places where the Platform's data model is confusing, duplicated or weaker than it looks. It is written for the product owner, who will meet these questions when planning features, writing reports or reviewing a migration. Each item says what it is, why it matters, the risk, and a status label. The facts come from the entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md` , sections 9 and 10) and were checked against the code; the other data-model chapters (Chapters 12 to 26) describe the tables themselves.

27.1 How to read this chapter

Each topic has the same four parts:

- **What it is.** The tables or columns involved.
- **Why it matters.** What goes wrong for a developer, a report writer or a user.
- **Risk.** Low, medium or high, with the reason.
- **Status.** One label. [Implemented] means both sides exist and work as designed; [Partial] means one side is incomplete; [Not implemented] means a planned link was never finished; [Unverified] means the code does not say which answer is right.

27.2 Two prescription tables

What it is. `prescriptions` (prescriptions module) is the current prescription record: draft and finalized states, medicines in their own table, change history, doctor signatures, templates and print layouts. `patient_prescriptions` (patients module) is the older, simpler record: one row with the medicines in a JSON array. The class comment of `prescriptions` says it "coexists with the legacy" table while consumers migrate.

Why it matters. The patient's Clinical tab shows both. The main card lists prescriptions from the new table, and a collapsible "Legacy quick-Rx history" section still reads the old table and still offers a form to add a legacy prescription. A report that counts prescriptions from only one table will be wrong.

Risk. Medium. Data is split, and new legacy rows can still be created.

Status. [Partial]. Migration to the new table is not finished; the legacy write path is still live.

27.3 Four "visit" concepts

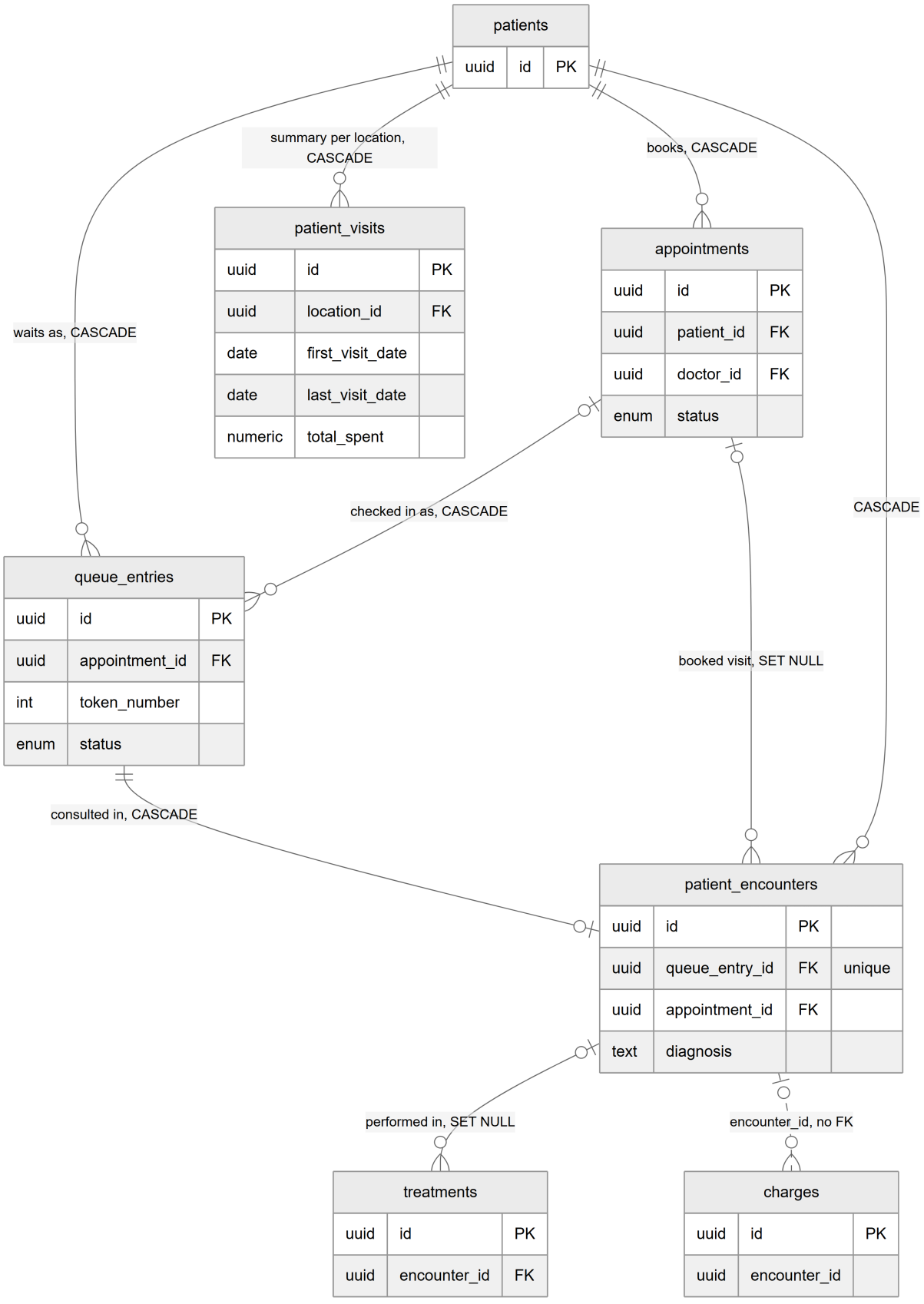


Figure 27.1 — Appointments, queue entries, encounters and the patient visit summary, with treatments and charges.

The figure shows `patients` at the top, connected to `appointments` ("books"), `queue_entries` ("waits as"), `patient_encounters` and `patient_visits` ("summary per location"), all CASCADE. `appointments` connects optionally to `queue_entries` ("checked in as, CASCADE") and to `patient_encounters` ("booked visit, SET NULL"). `queue_entries` connects one-to-one to `patient_encounters` ("consulted in, CASCADE"). `patient_encounters` connects to `treatments` ("performed in, SET NULL") and, by a dotted line with no foreign key, to `charges` (encounter_id). `patient_visits` has no link to any of the other visit tables.

What it is.

- `appointments` : a booking for a future time with a doctor.
- `queue_entries` : a patient's place in one doctor's queue on one day. Created only at check-in or for a walk-in, never at booking.
- `patient_encounters` : the clinical record of one consultation, created when the doctor starts it. Exactly one per queue entry; it points at the appointment for booked visits and has none for walk-ins.
- `patient_visits` : a per-location summary for a patient (first and last visit date, total spent, preferred location).

Why it matters. "How many visits did we have?" has four possible answers. Appointments include no-shows and cancellations; queue entries include walk-ins and people who left; encounters are consultations that actually started. `patient_visits` gives no answer at all: in the code, the patients service only reads it (`GET /patients/:id/visits`), and nothing writes rows into it.

Risk. Medium. Reports and dashboards can disagree depending on which table they count. The visit summary will be empty for every patient.

Status. [Partial]. Appointments, queue entries and encounters are [Implemented]; `patient_visits` is read but never written.

27.4 Treatment-plan table families

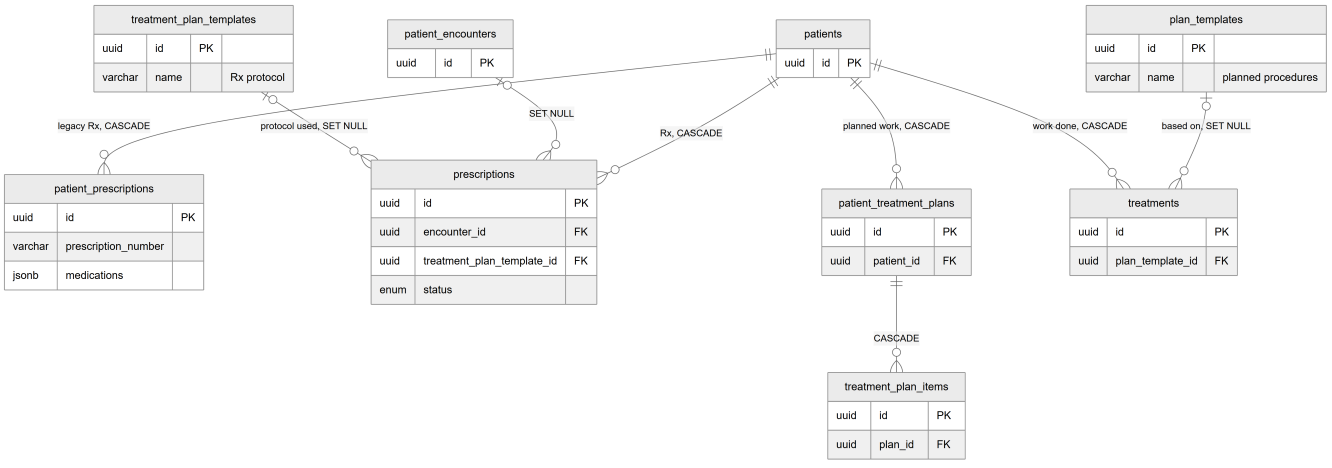


Figure 27.2 — The tables whose names suggest "plan" or "prescription", and what they really hold.

The figure shows `patients` connected to `patient_prescriptions` ("legacy Rx"), `prescriptions` ("Rx"), `patient_treatment_plans` ("planned work") and `treatments` ("work done"), all CASCADE. `patient_treatment_plans` connects to `treatment_plan_items` (CASCADE). `plan_templates` connects to `treatments` ("based on, SET NULL"). `treatment_plan_templates` connects to `prescriptions` ("protocol used, SET NULL"), and `patient_encounters` connects to `prescriptions` (SET NULL). Notes on the template boxes say "Rx protocol" for `treatment_plan_templates` and "planned procedures" for `plan_templates` .

What it is.

- `patient_treatment_plans` with `treatment_plan_items`: planned work for a patient.
- `treatments`: work actually performed, optionally in an encounter.
- `plan_templates`: predefined treatment plans ("Root canal treatment" with its usual steps), copied into a patient's plan.
- `treatment_plan_templates`: despite the name, prescription protocols ("Root canal aftercare"), used by the prescription builder to pre-fill medicines and instructions.

Why it matters. The names invite mistakes. A developer asked to "add a step to the treatment plan template" could easily edit the prescription protocol table. The code comment on `plan_templates` itself warns against confusing the two.

Risk. Low for data, medium for development mistakes.

Status. [Implemented]. All four families work; the issue is naming. In prose the handbook uses **plan template** for `plan_templates` and calls `treatment_plan_templates` "prescription protocol templates".

27.5 Several audit tables

What it is. Change history lives in five places: `audit_logs` (generic, changed fields only), `status_history` (every status change of lab and workflow records), `billing_audit_logs` (billing's own log with full before and after snapshots), `prescription_history` (prescription changes) and `queue_events` (queue actions). The comment on `billing_audit_logs` says it exists because "the repo has no general audit module", which is no longer true.

Why it matters. "Who changed this invoice?" is answered by `billing_audit_logs`; "who changed this patient?" by `audit_logs`. A single audit screen or export must read all five.

Risk. Low. Each log is consistent within its area.

Status. [Implemented]. Consolidation is a design choice not yet made.

27.6 Two ledgers

What it is. `ledger_entries` is the patient statement: one-sided, in rupees. `ledger_journals` with `ledger_journal_lines` is a double-entry general ledger in paisa. Both are described in the Billing ledgers chapter (Chapter 21).

Why it matters. The general ledger is not a complete set of books. In the code, only lab money posts journals: patient lab invoices, lab payments, credit notes, lab refunds, commission on lab lines, vendor bills and vendor payments. Ordinary clinic invoices and payments, and expenses, do not post journals, and no report reads the general ledger.

Risk. Medium. Anyone who builds accounting reports from the journals will miss most clinic revenue.

Status. [Partial]. The patient statement is [Implemented]; the general ledger covers lab money only.

27.7 Two outboxes

What it is. `outbox` (queue module) carries realtime queue events to the Socket.IO relay. `notification_outbox` (notifications module) holds messages for people: in-app today, other channels later.

Why it matters. The names are similar but the purposes are separate. A developer adding a notification should never write to `outbox`, and a realtime screen update should never go through `notification_outbox`.

Risk. Low.

Status. [Implemented]. Both work as designed.

27.8 Two sequence mechanisms

What it is. `document_sequences` (billing) counts clinic invoices, receipts and refunds per organization, location, document type and year, giving numbers like `<location code>-INV-<year>-00001`. `number_sequences` (common) counts per organization and key, giving numbers like LAB-0001 and SHP-0001 for the lab module. The inventory module uses neither: it builds purchase-order numbers by reading the organization's last order.

Why it matters. Numbering rules differ by document: clinic documents restart every year at each location, lab documents run on per organization. The inventory approach reads the last number without a lock, so two orders created at the same moment can get the same number; and its purchase-order number is unique across the whole Platform while it is generated per organization, so a second organization's first purchase order collides with the first organization's (the Inventory chapter, Chapter 23, explains it).

Risk. Low for the two sequence tables; high for inventory purchase-order numbering.

Status. [Implemented] for the two sequence tables. The purchase-order numbering defect is recorded for the owner.

27.9 `services` versus `lab_services`

What it is. `services` (code class `CatalogService`) is the clinic's billable catalog: what the patient is charged for. `lab_services` is a lab's catalog: what the lab makes and charges the clinic for. `service_mappings` connects them. Lab tables call a `services` id `clinic_treatment_id`.

Why it matters. The class name, table name and column name all differ, which makes code search and SQL harder. The clinic price and the lab cost of the same crown live in different tables.

Risk. Low.

Status. [Implemented].

27.10 Logical references with uncertain targets

These columns hold an id but nothing in the code says for certain which table it belongs to:

- `dentist_id` on `patient_teeth_chart`, `patient_treatment_plans`, `patient_recalls` and `patient_feedback`: an employee id or a user id. Only the request formats mention it, as a plain string.
- `doctor_signatures.employee_id`: its code comment says it currently stores the signed-in **user's** id, despite the column name.
- `locations.manager_id`: an employee or a user.
- `patient_consumptions.treatmentPlanId` (inventory): probably a patient treatment plan.
- `notification_outbox.recipient_id` (by `recipient_type`) and `config_values.scope_id` (by `scope`): polymorphic by design.
- `stock_movements.referenceId`: polymorphic and stored as text, not as a UUID.

Why it matters. A join to the wrong table returns nothing or, worse, the wrong person. The employee-versus-user confusion is the most dangerous, because both tables use UUIDs and the mistake is silent.

Risk. Medium.

Status. [Unverified] for the `dentist_id`, `manager_id` and `treatmentPlanId` targets; the `doctor_signatures` case is confirmed by the code comment.

27.11 Foreign keys that look missing

What it is.

- `magic_links.shipment_id` has no foreign key. Its comment says "no FK until that table exists here", but `shipments` now exists. No code creates shipment magic links.
- `lab_order_items.clinic_treatment_id` has no foreign key, while `service_mappings.clinic_treatment_id`, pointing at the same `services` table, has one.
- Almost every billing v2 table, and most lab tables, store `organization_id` and `location_id` without foreign keys, while older modules constrain them. No comment explains the choice.

Why it matters. Without a foreign key the database accepts ids that do not exist, and deletes are not blocked. Tenant isolation in these tables rests entirely on the application.

Risk. Medium. Low today because the application always fills these columns, but a bug or a manual SQL fix can create orphan rows.

Status. [Not implemented] for the shipment link; the others are as designed but undocumented.

27.12 "Append-only" that the database does not enforce

What it is. Class comments call `ledger_entries`, `ledger_journals`, `ledger_journal_lines`, `payment_allocations`, `billing_audit_logs` and `notification_outbox` append-only. Only three tables have a trigger that rejects changes: `commission_entries` (deletes and amount changes refused), `audit_logs` and `status_history` (all updates and deletes refused). A comment on `form_schema_versions` also claims a trigger, but none exists; the service enforces it.

Why it matters. For the unprotected tables, append-only is a promise of the application code only. A manual SQL statement or a future bug can change history without any error.

Risk. Medium for the financial tables.

Status. [Partial]. Enforced in the database for three tables, by the application for the rest.

27.13 Naming inconsistency

What it is. The ten registered inventory tables use camelCase column names (`organizationId`, `currentStock`, `treatmentPlanId`). Every other module, including the legacy `inventory` table, uses snake_case.

Why it matters. In SQL, camelCase names must be quoted ("organizationId"); unquoted names fail. Report writers who move between modules trip over it. In `stock_movements` the tenant columns are also text instead of UUID.

Risk. Low.

Status. [Implemented] as is; renaming would need a migration.

27.14 Employee phone and CNIC are unique across the whole Platform

What it is. `employees` is a person record shared by every clinic the person works for; the link to a clinic is the assignment. Its `phone` and `cnic` columns are unique across the entire Platform, not per organization.

Why it matters. If two clinics enter the same doctor separately, the second insert fails. The design intends the second clinic to reuse the existing person record, but the clinic may not know that record exists, and its contact details are shared with the other clinic.

Risk. Medium. Expect support questions when a doctor works at two unrelated clinics.

Status. [Implemented] by design.

27.15 Primary keys with no default

What it is. These tables have keys that the application must supply: `document_sequences`, `number_sequences`, `organization_specialties`, `transition_effects` (composite keys); `location_billing_settings` (location id), `lab_billing_settings` (organization id), `dental_item_details` and `item_form_responses` (item id), `patient_comm_prefs` (patient id); `workflow_effects` and `notification_events` (a code); and `idempotency_keys`.

Why it matters. Most of these are "one row per parent" tables, where using the parent's id as the key is the point: it makes a second row impossible. An insert that forgets the key fails at once, which is safe.

Risk. Low.

Status. [Implemented] by design.

27.16 Entities the running application does not create

What it is. `chairs` and the legacy `inventory` table have entity files, but no module registers them. The development database, built by the running application, does not contain them; a production database built by the command-line schema tool, which loads every entity file, may contain them.

Why it matters. Development and production schemas can differ by these two tables, which confuses schema comparisons.

Risk. Low.

Status. [Not implemented] for both tables.